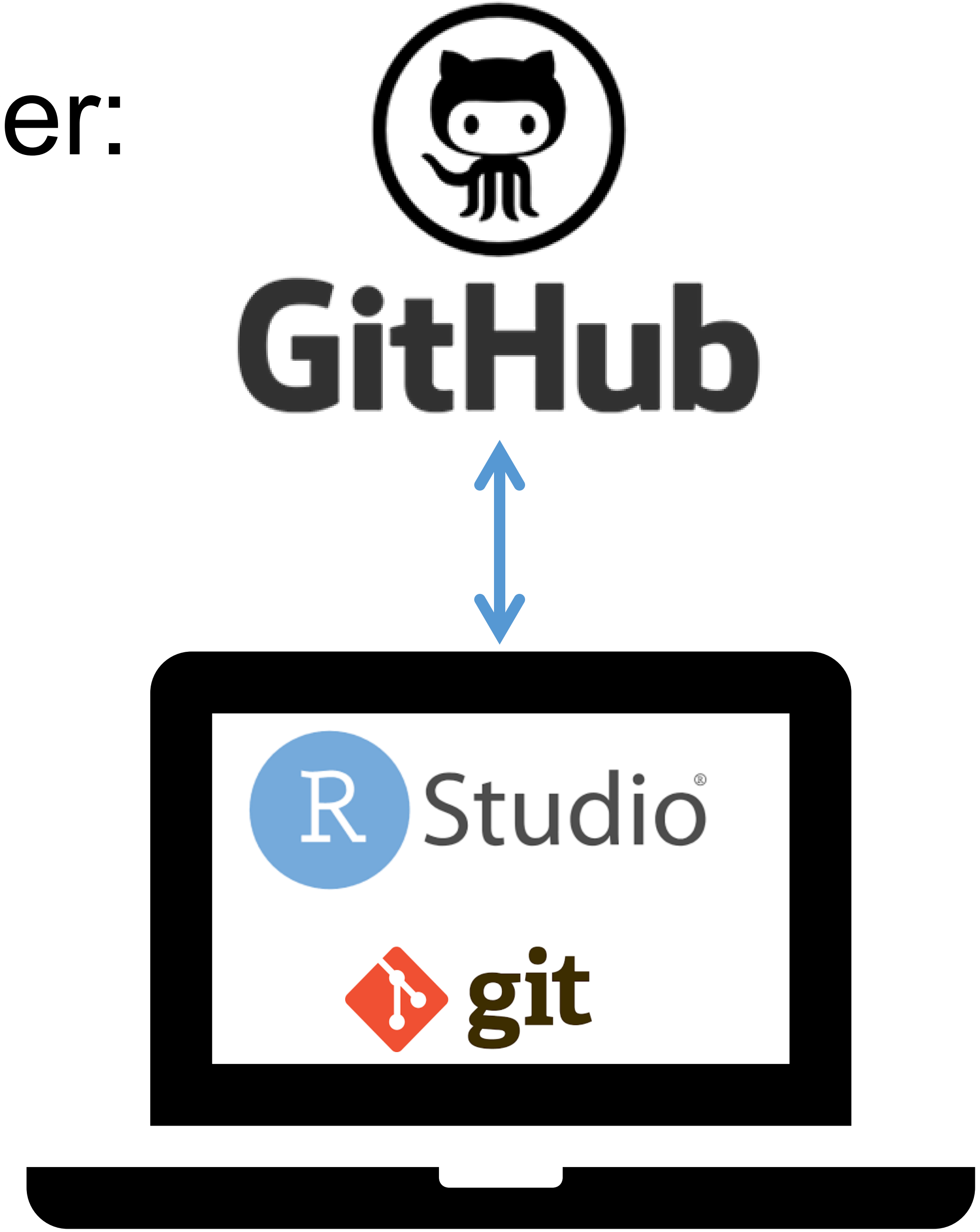
Version control using git in RStudio

Tobin Magle, PhD
Lead data management specialist
Research Computing and Data Services

Today we're going to cover:

- What is **version control**?
- What is **git**?
- Version control using **RStudio**
- Backing up your data to **GitHub**



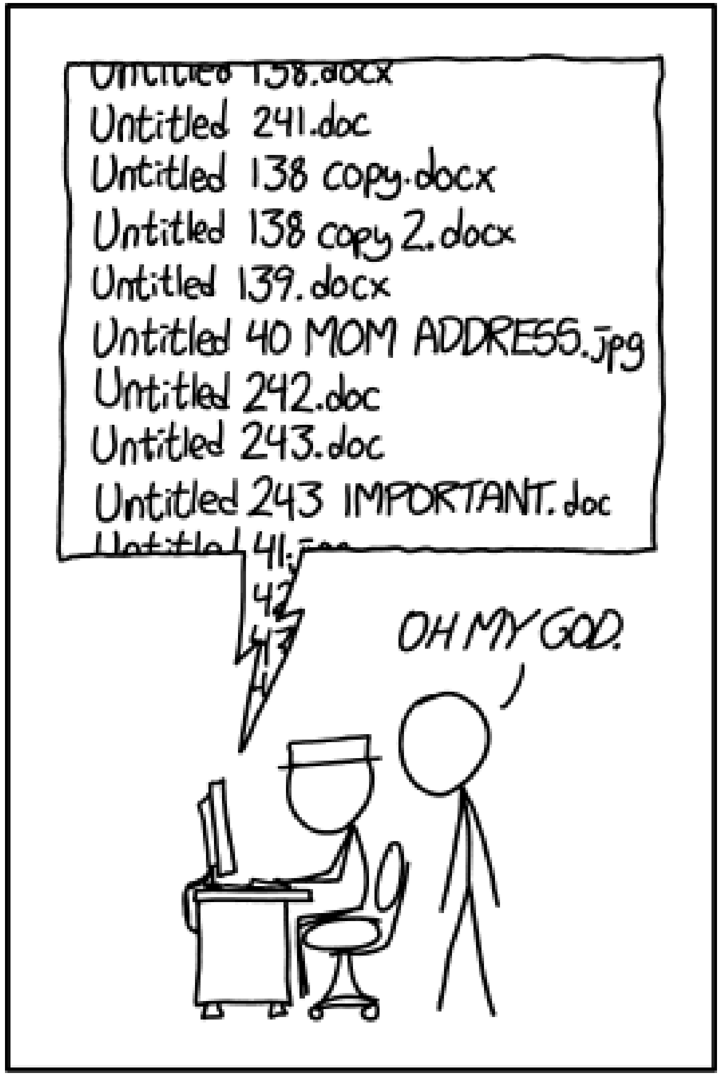
Pre-workshop Setup

- Install R installed
- Install RStudio
- Install git
- Created a GitHub account

Manual Version Control

“Renaming files”

- Creates many files
- Relies on your ability to name files consistently
- Not designed for collaboration



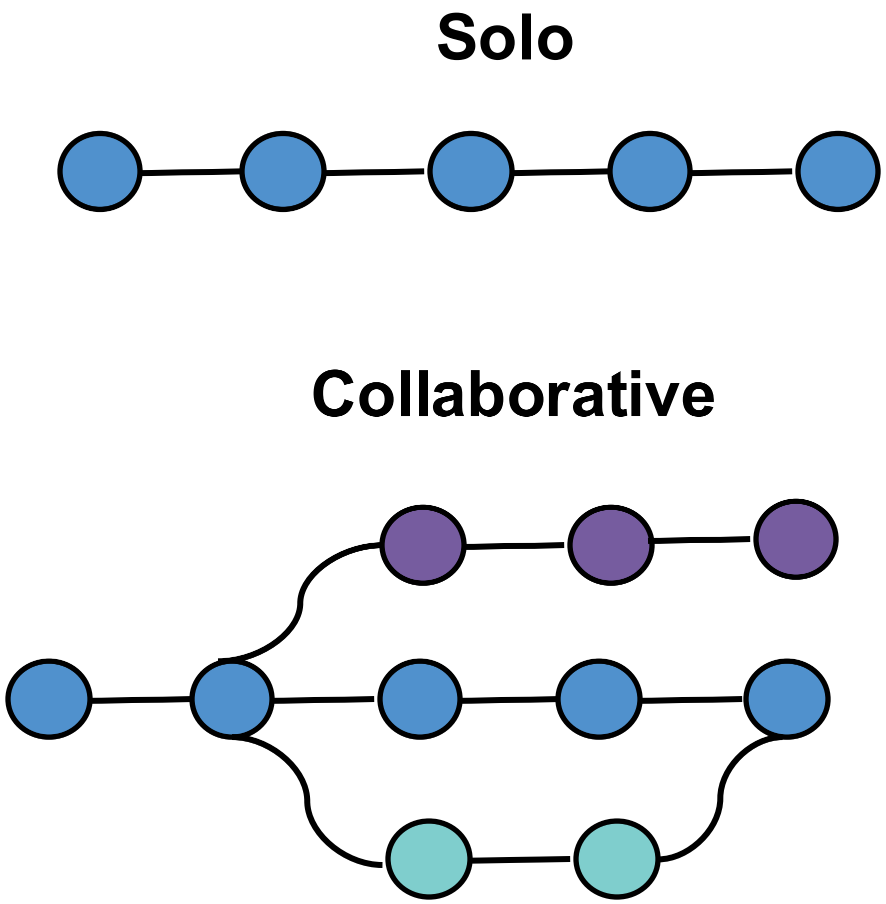
PROTIP: NEVER LOOK IN SOMEONE ELSE'S DOCUMENTS FOLDER.

https://xkcd.com/1459/

Version Control Systems

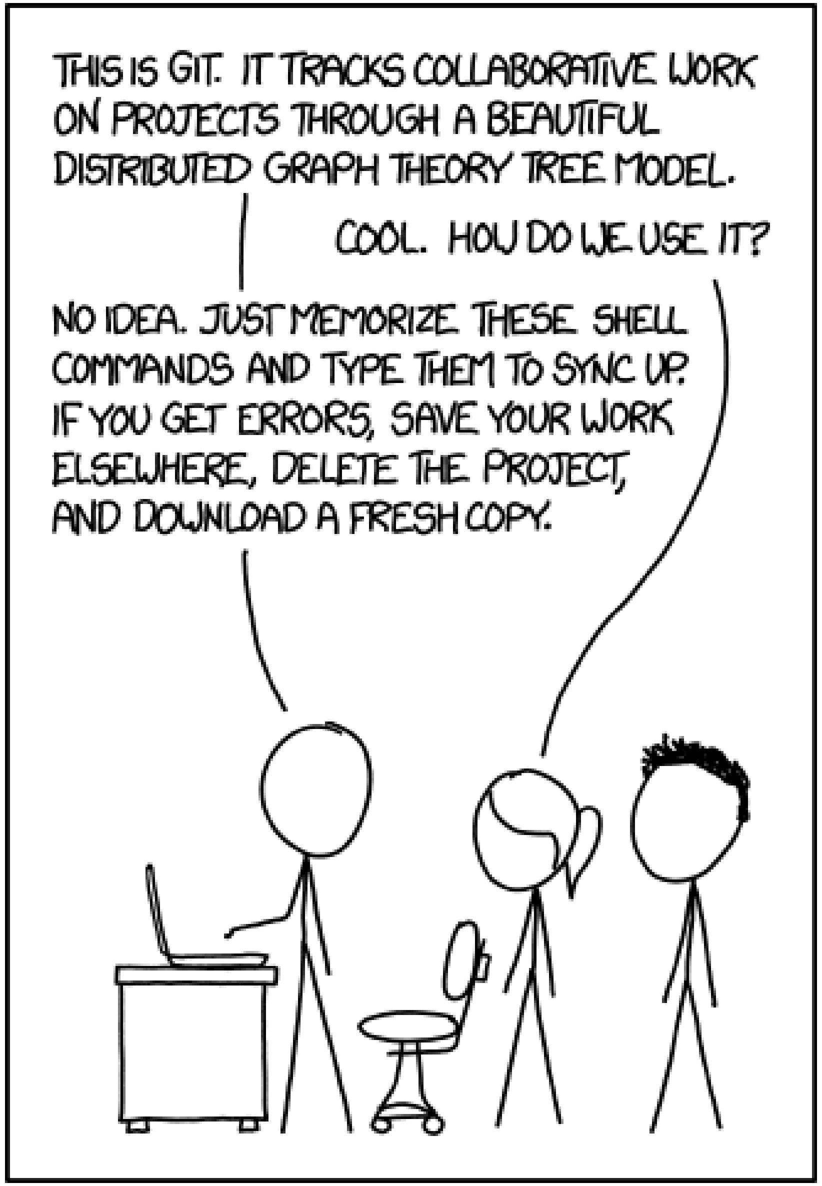
“Unlimited undo”

- One file, many versions
- Records who made what change when
- Good for **collaboration**: identifies and helps resolve conflicting changes



Today we're using **git**

- Built for large software development projects
- Can get complicated, but the basics are simple
- Free, open source
- Many online resources/tutorials



https://xkcd.com/1597/

Types of files git works best with

“Good” formats (text based)

- Documents (.txt, .tex, .rtf, .Rmd)
- Tabular Data (.csv)
- Source code (.R, .py, .c, .sh)

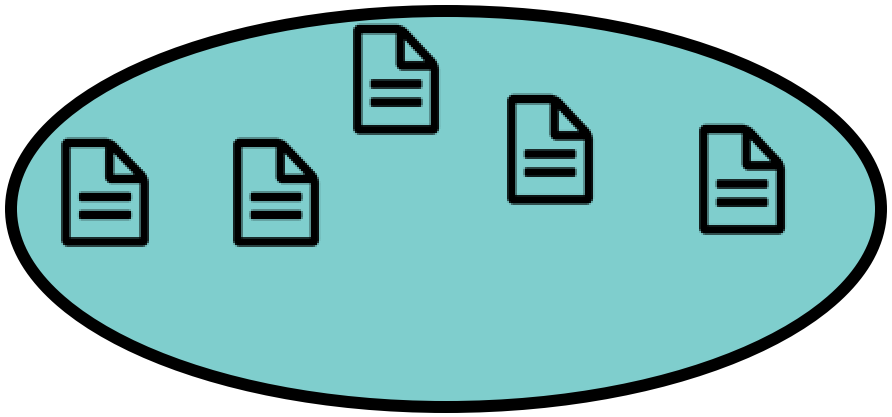
“Bad” formats (binary files)

- Documents (.docx, .pdf, .ppt)
- Excel spreadsheets (.xlsx)
- Media (.jpg, .mp3, .mp4)
- Databases (.mdb, .sqlite)

https://www.nayuki.io/page/what-are-binary-and-text-files

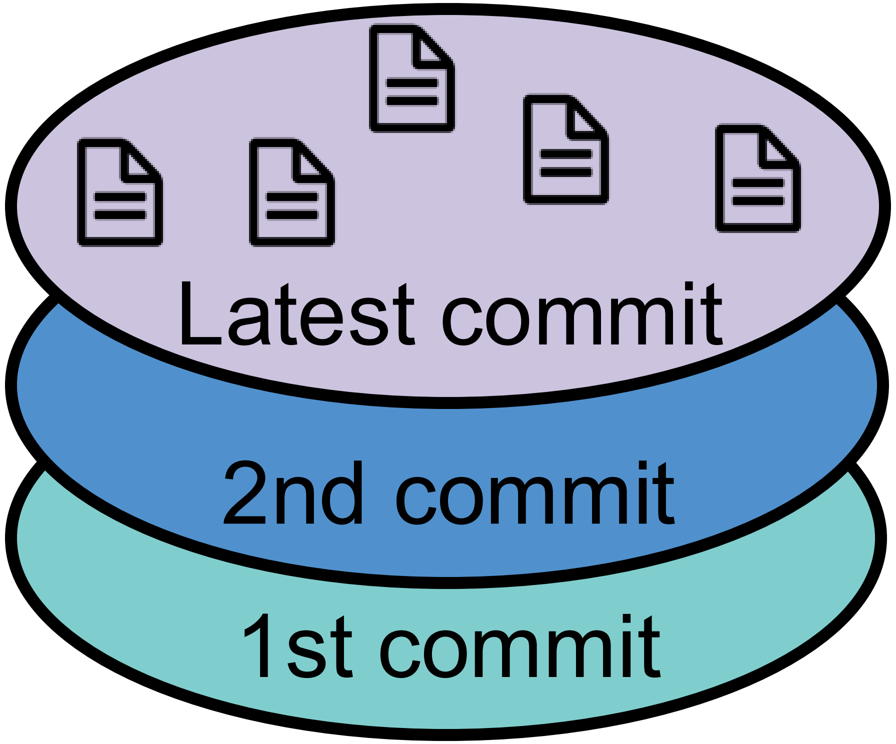
Using git creates a repository

- **Repository** – A folder that is tracked by git



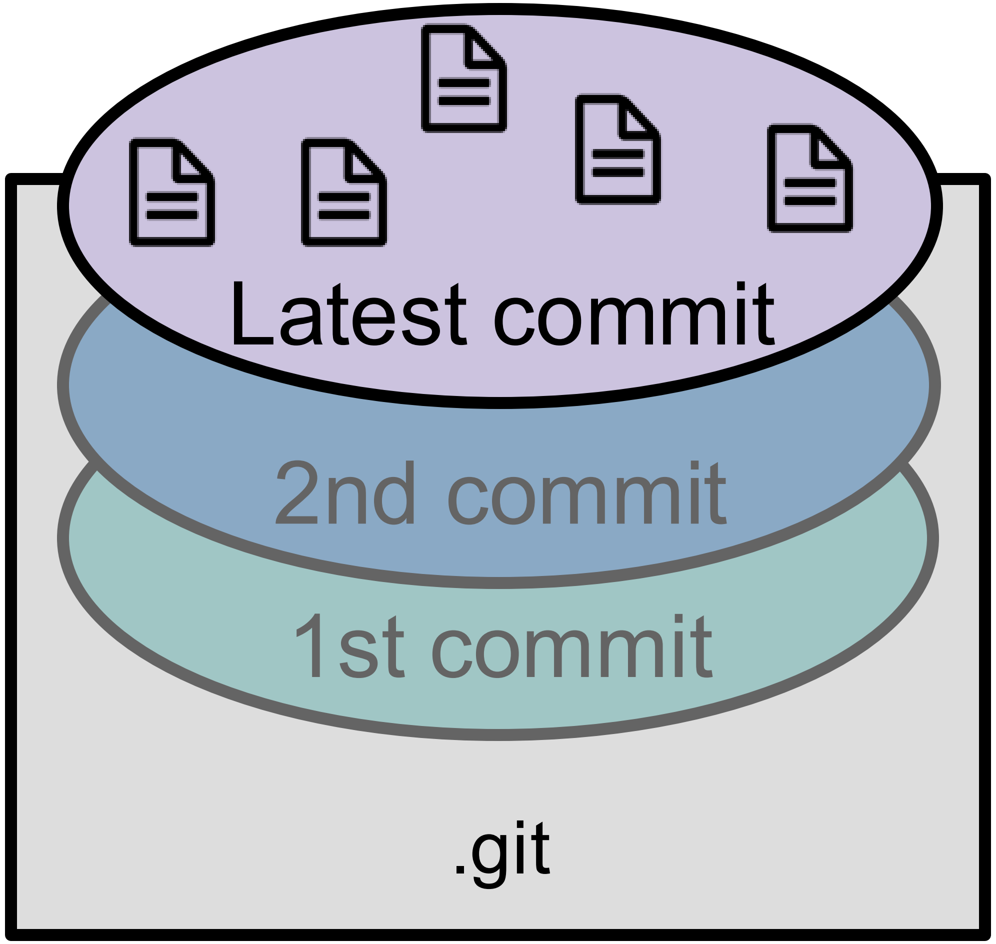
Repositories are made of commits

- **Repository** – A folder that is tracked by git
- **Commit** – a version of what the folder looks like when it was created



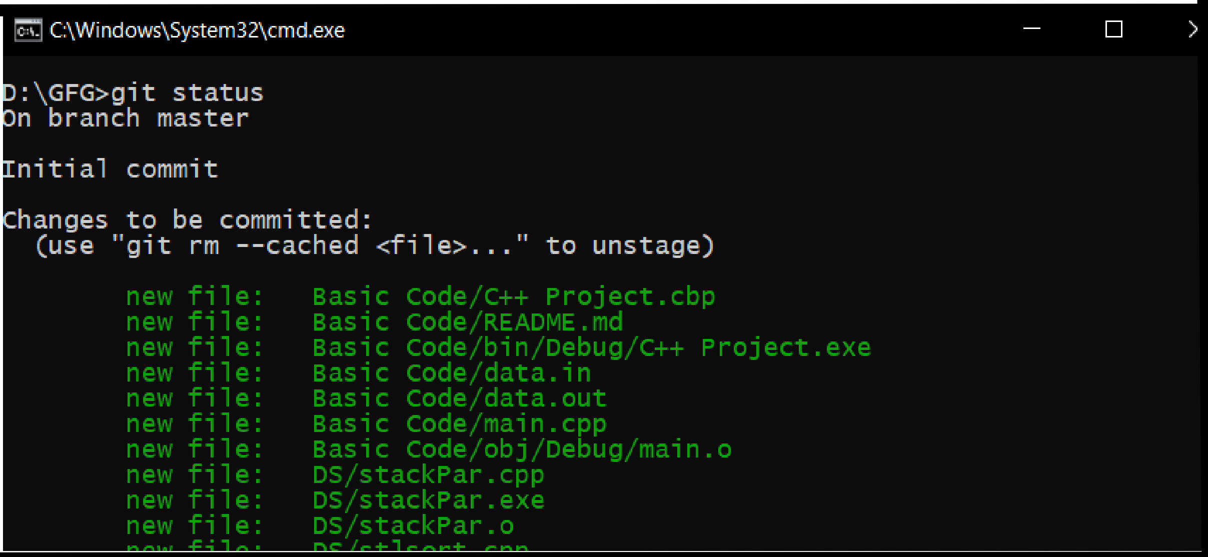
You only see your latest commit

- **Repository** – A folder that is tracked by git
- **Commit** – a version of what the folder looks like when it was created
- **Structure is hidden** (.git), you see the most recent version

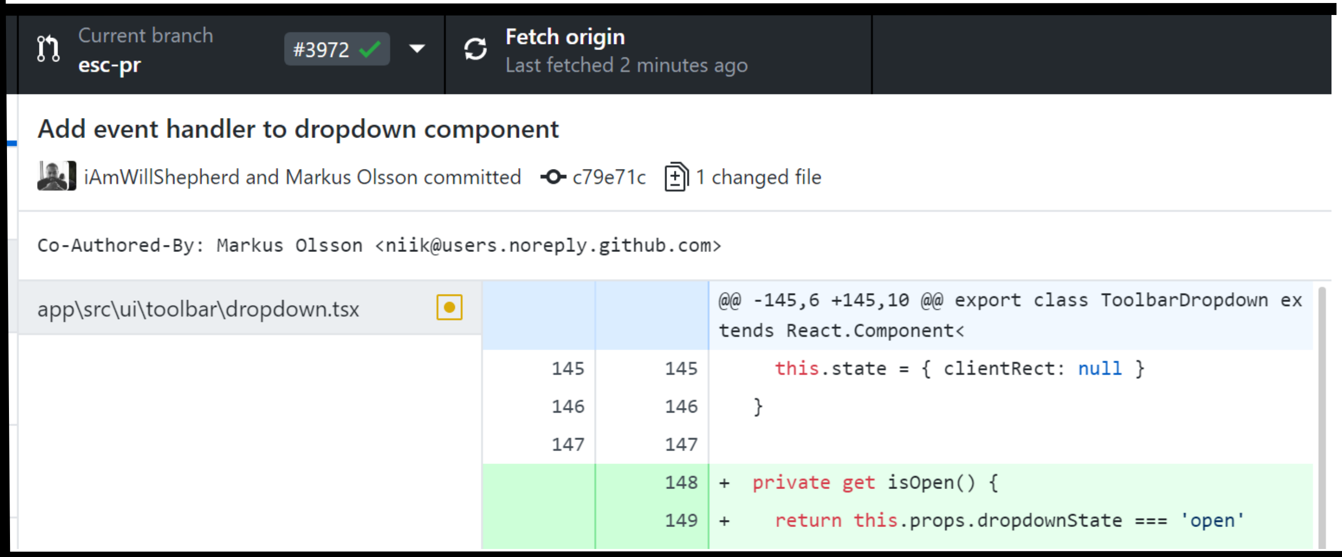


Many ways to use Git

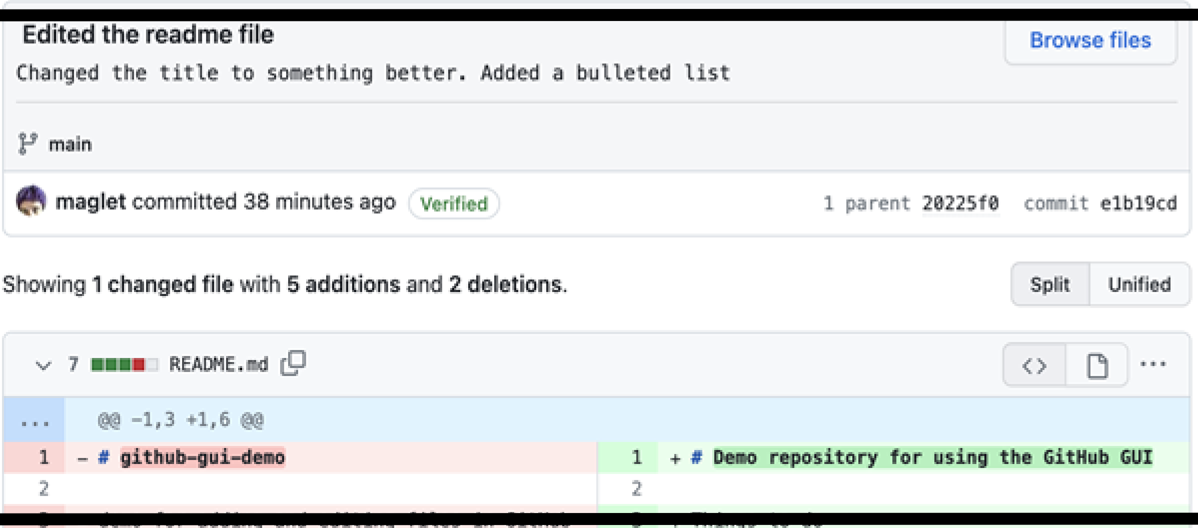
Command line



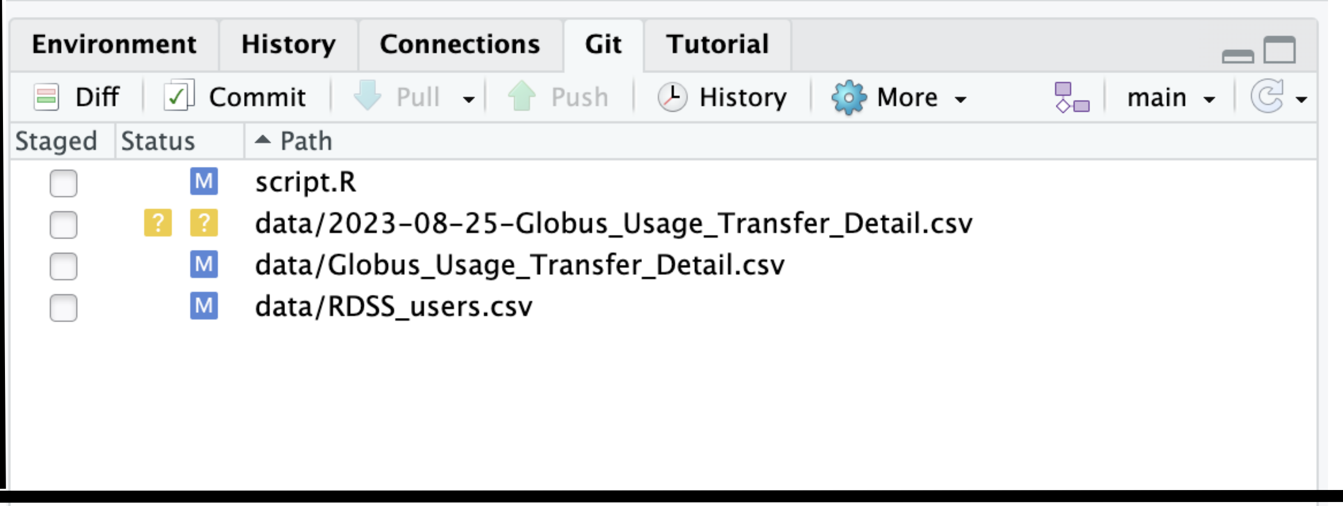
Desktop Clients



GitHub Website

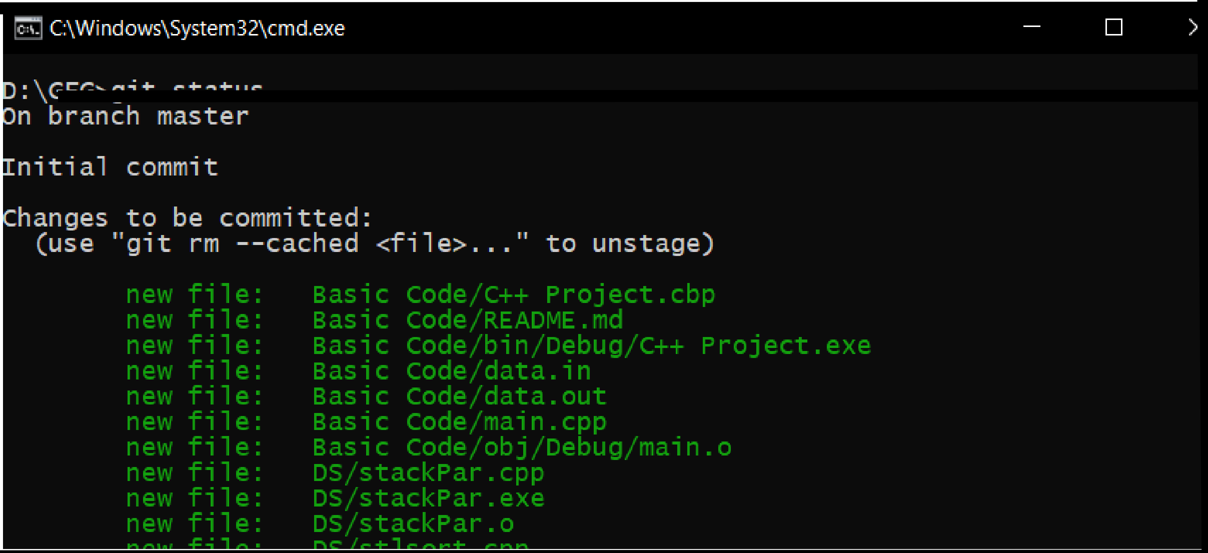


RStudio IDE

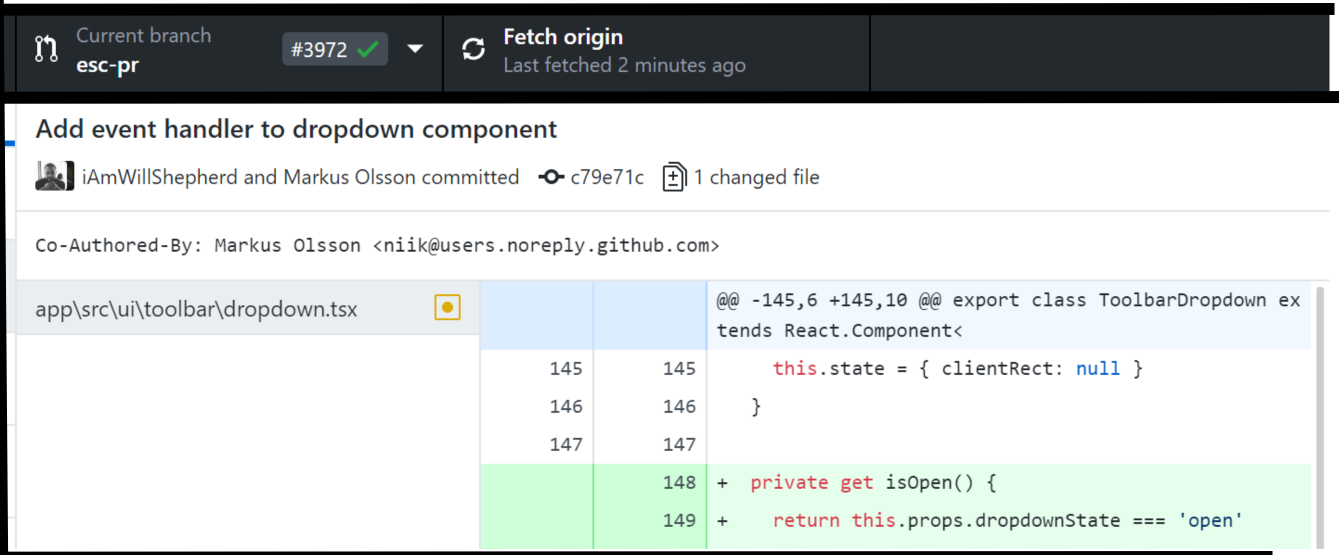


Many ways to use Git

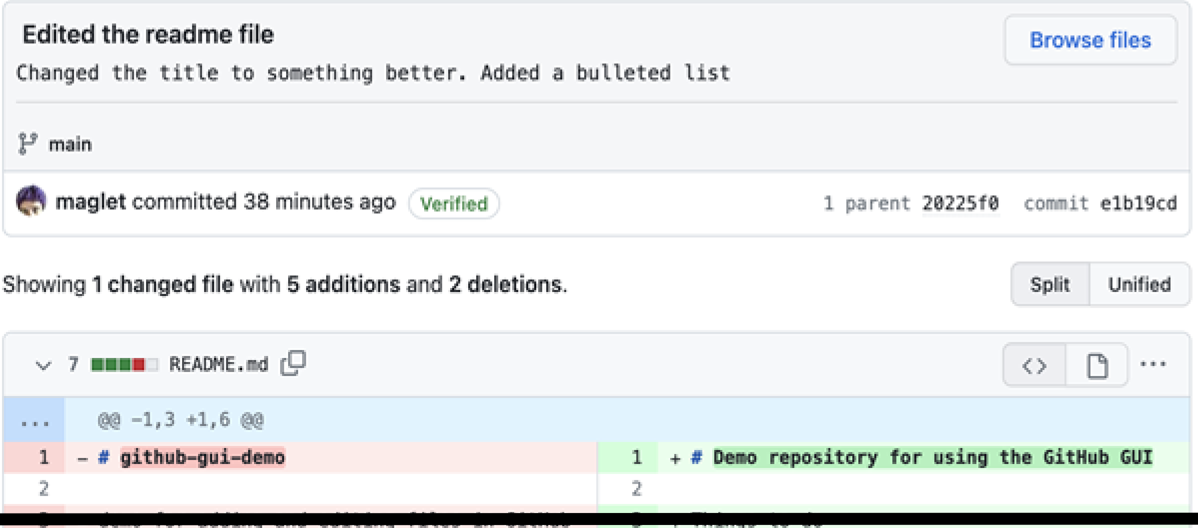
Command line



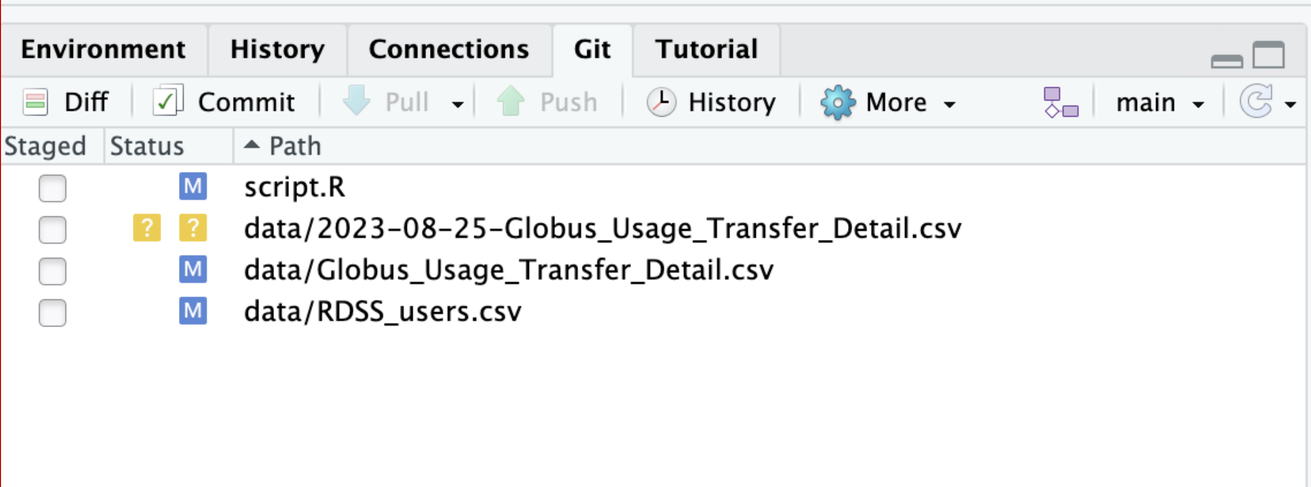
Desktop Clients

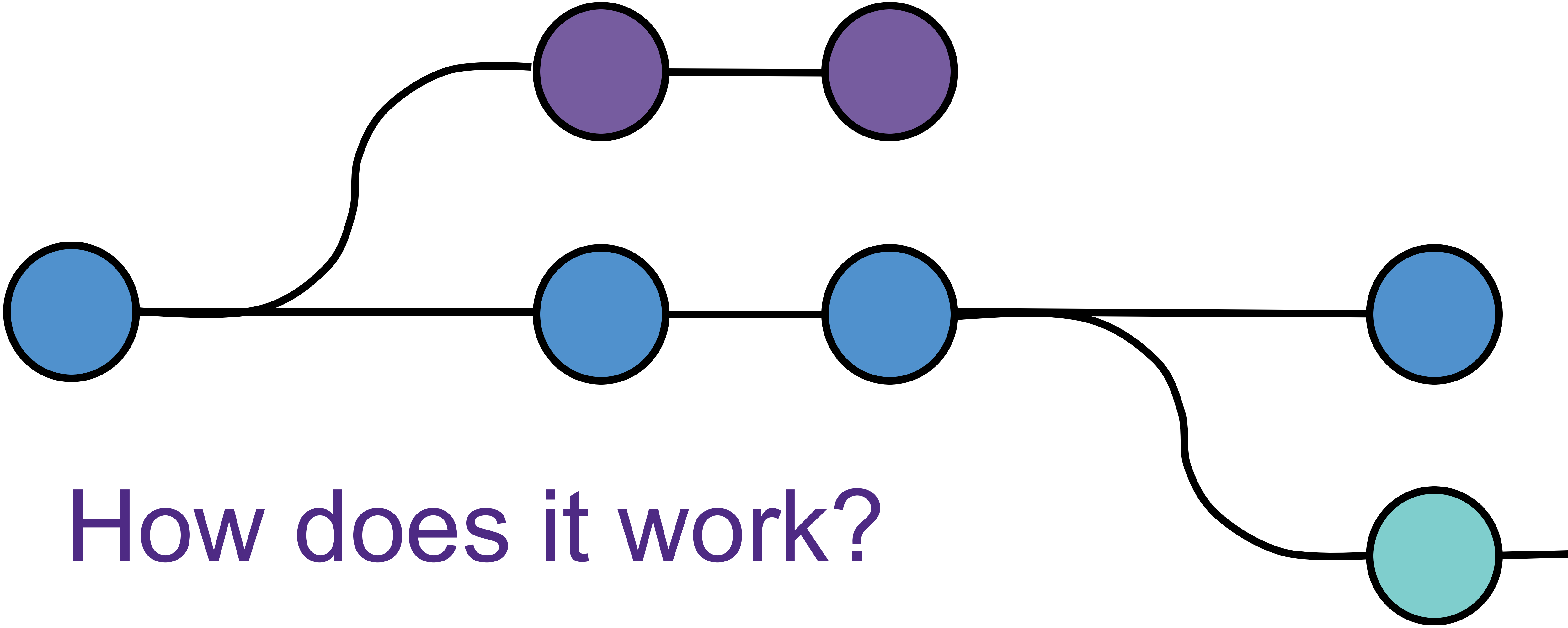


GitHub Website



RStudio IDE





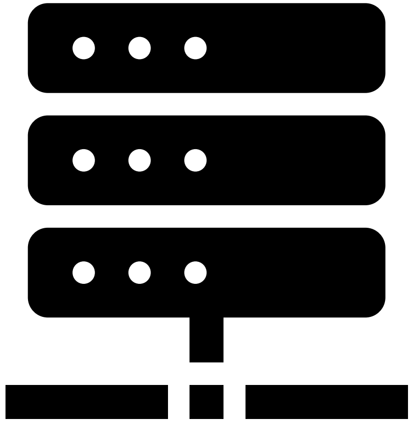
How does it work?

Coding happens on your computer

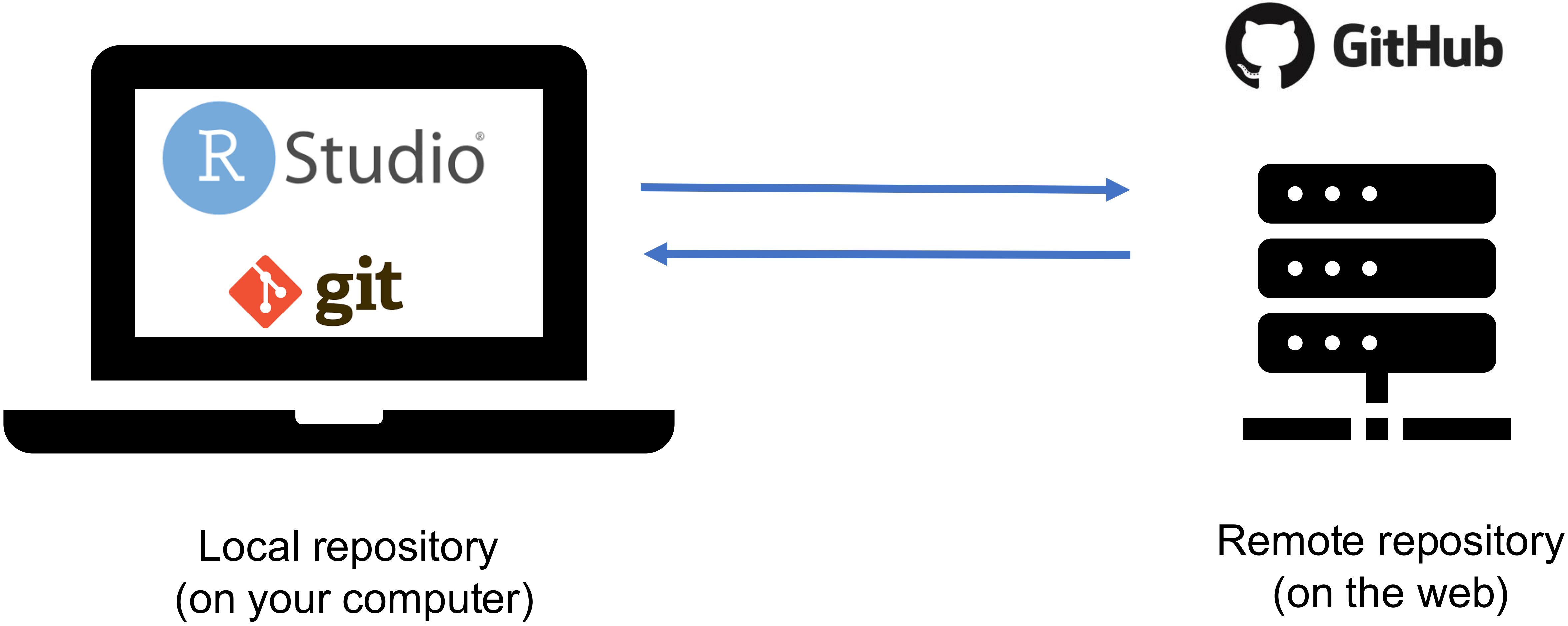
- Create and edit files in RStudio
- Git is also on your computer
- Use git version control in the RStudio interface
- Creates a local repository

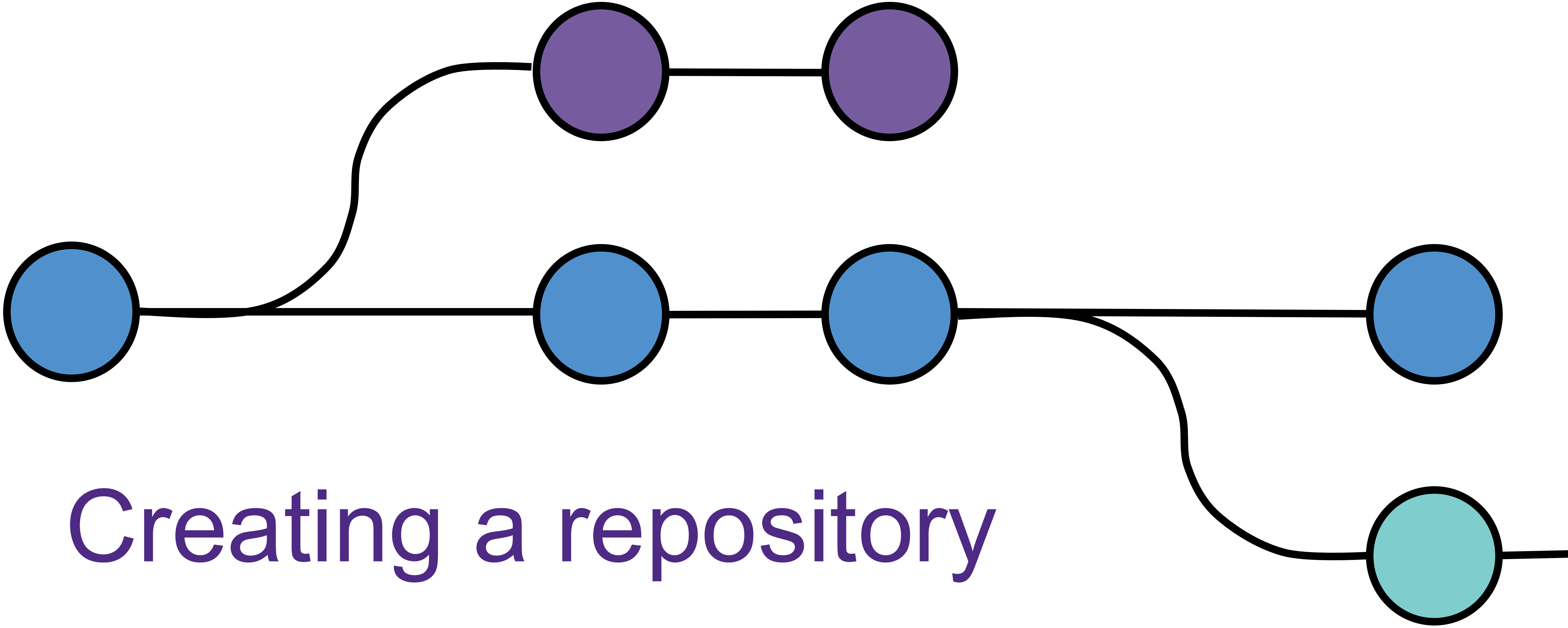
GitHub stores code in the cloud

- Back up your work
- Collaborate with colleagues
- Share your code



Overall workflow

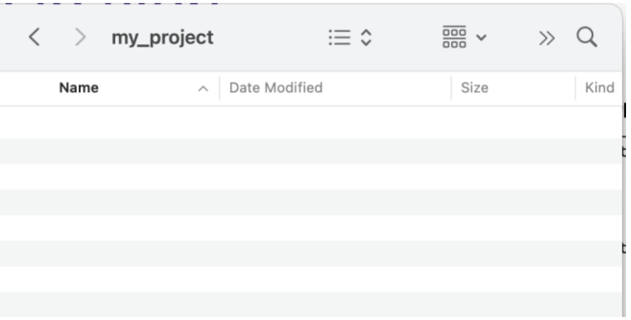




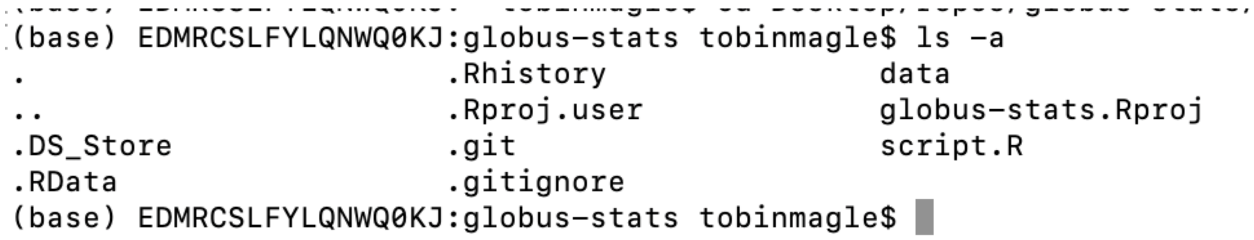
Creating a repository

A few ways to start

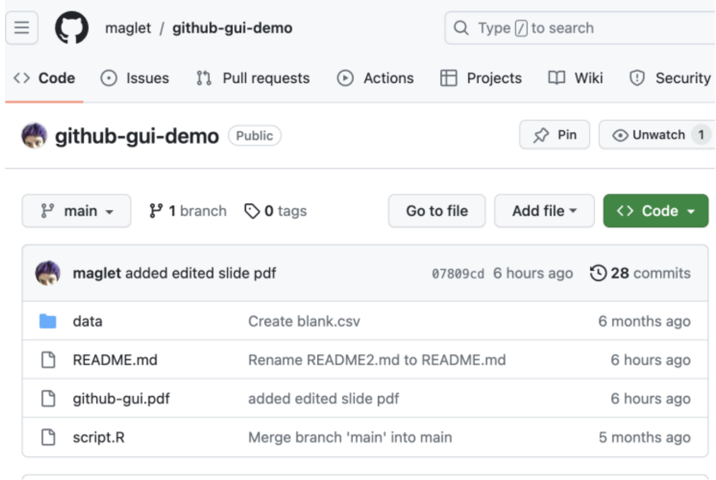
From nothing



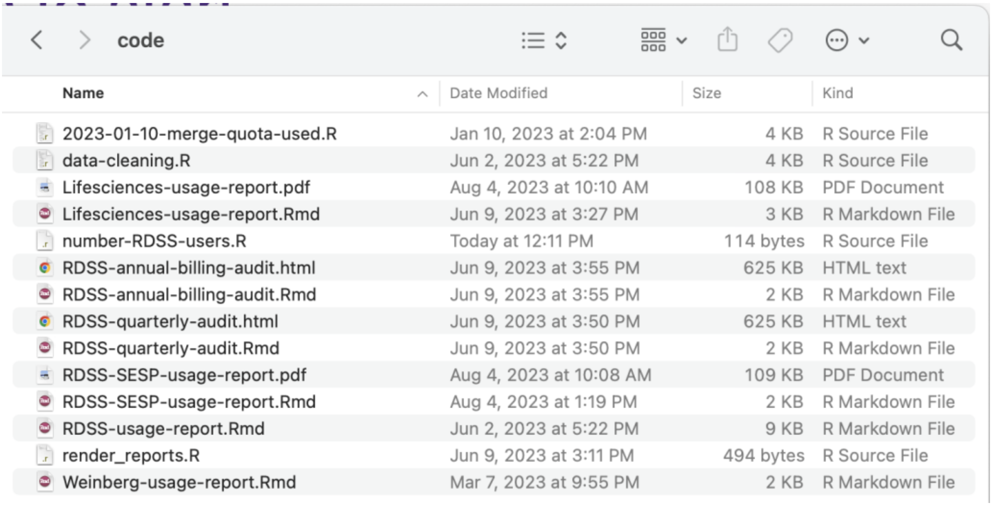
From a local repo



From a GitHub repo



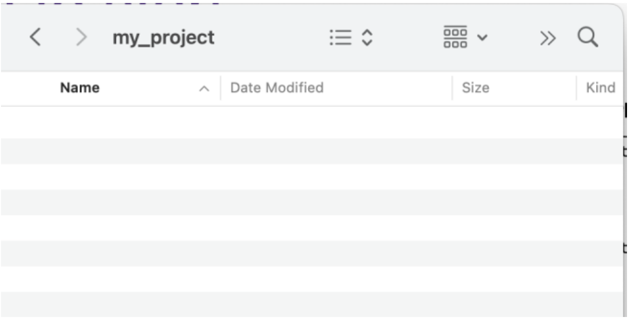
From files outside a repo



https://sites.northwestern.edu/researchcomputing/2022/05/11/git-with-rstudio-order-matters/

A few ways to start

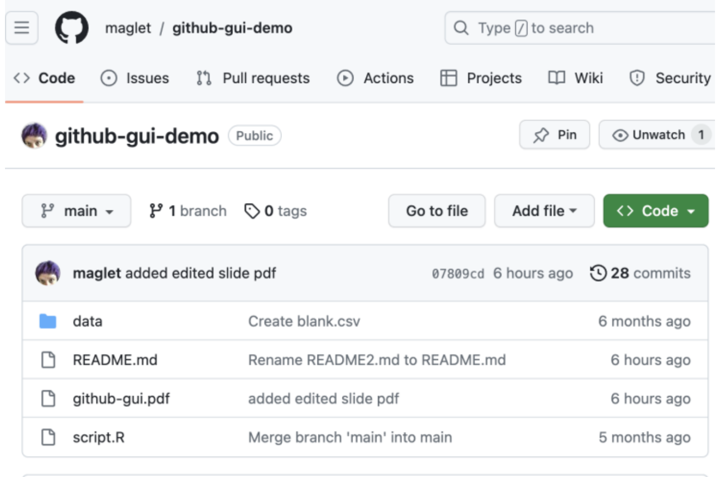
From nothing



From a local repo

```
(base) EDMRCSLFYLQNWQ0KJ:globus-stats tobinmagle$ ls -a
.                            .Rhistory                    data
..                           .Rproj.user                  globus-stats.Rproj
.DS_Store                    .git                         script.R
.RData                       .gitignore
(base) EDMRCSLFYLQNWQ0KJ:globus-stats tobinmagle$
```

From a GitHub repo



From files outside a repo

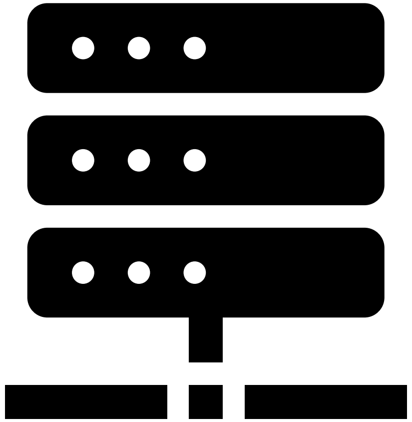
code

Name	Date Modified	Size	Kind
2023-01-10-merge-quota-used.R	Jan 10, 2023 at 2:04 PM	4 KB	R Source File
data-cleaning.R	Jun 2, 2023 at 5:22 PM	4 KB	R Source File
Lifesciences-usage-report.pdf	Aug 4, 2023 at 10:10 AM	108 KB	PDF Document
Lifesciences-usage-report.Rmd	Jun 9, 2023 at 3:27 PM	3 KB	R Markdown File
number-RDSS-users.R	Today at 12:11 PM	114 bytes	R Source File
RDSS-annual-billing-audit.html	Jun 9, 2023 at 3:55 PM	625 KB	HTML text
RDSS-annual-billing-audit.Rmd	Jun 9, 2023 at 3:55 PM	2 KB	R Markdown File
RDSS-quarterly-audit.html	Jun 9, 2023 at 3:50 PM	625 KB	HTML text
RDSS-quarterly-audit.Rmd	Jun 9, 2023 at 3:50 PM	2 KB	R Markdown File
RDSS-SESP-usage-report.pdf	Aug 4, 2023 at 10:08 AM	109 KB	PDF Document
RDSS-SESP-usage-report.Rmd	Aug 4, 2023 at 1:19 PM	2 KB	R Markdown File
RDSS-usage-report.Rmd	Jun 2, 2023 at 5:22 PM	9 KB	R Markdown File
render_reports.R	Jun 9, 2023 at 3:11 PM	494 bytes	R Source File
Weinberg-usage-report.Rmd	Mar 7, 2023 at 9:55 PM	2 KB	R Markdown File

https://sites.northwestern.edu/researchcomputing/2022/05/11/git-with-rstudio-order-matters/

Start in GitHub

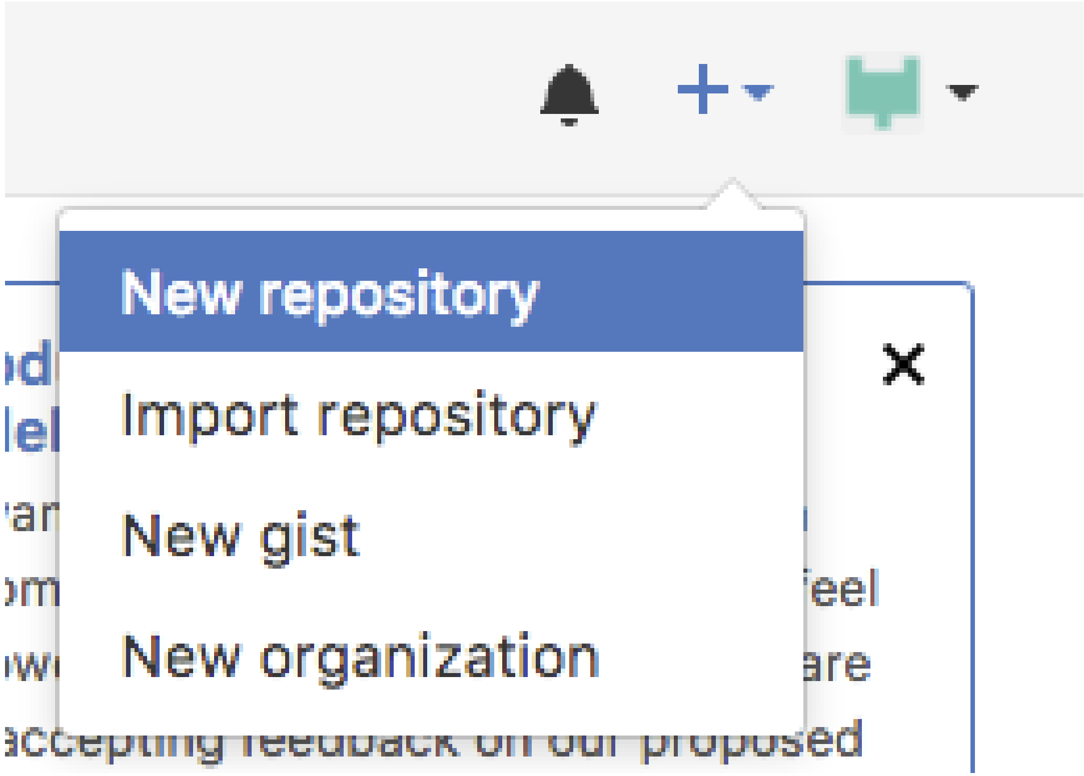
GitHub



Remote

GitHub: Create a repository

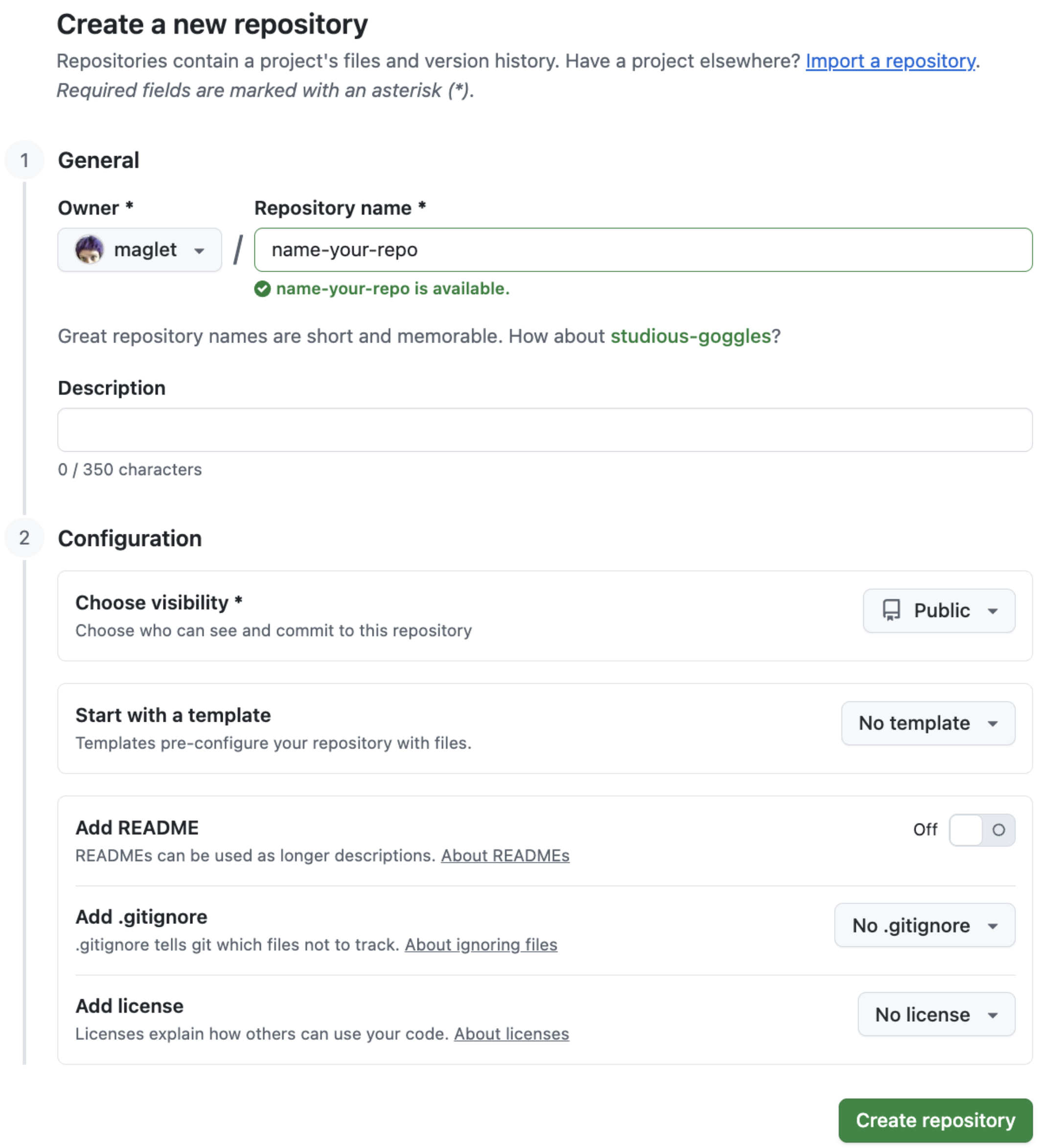
1. Go to http://github.com
2. Click the plus sign on the upper right-hand side of the screen
3. Select **New repository**



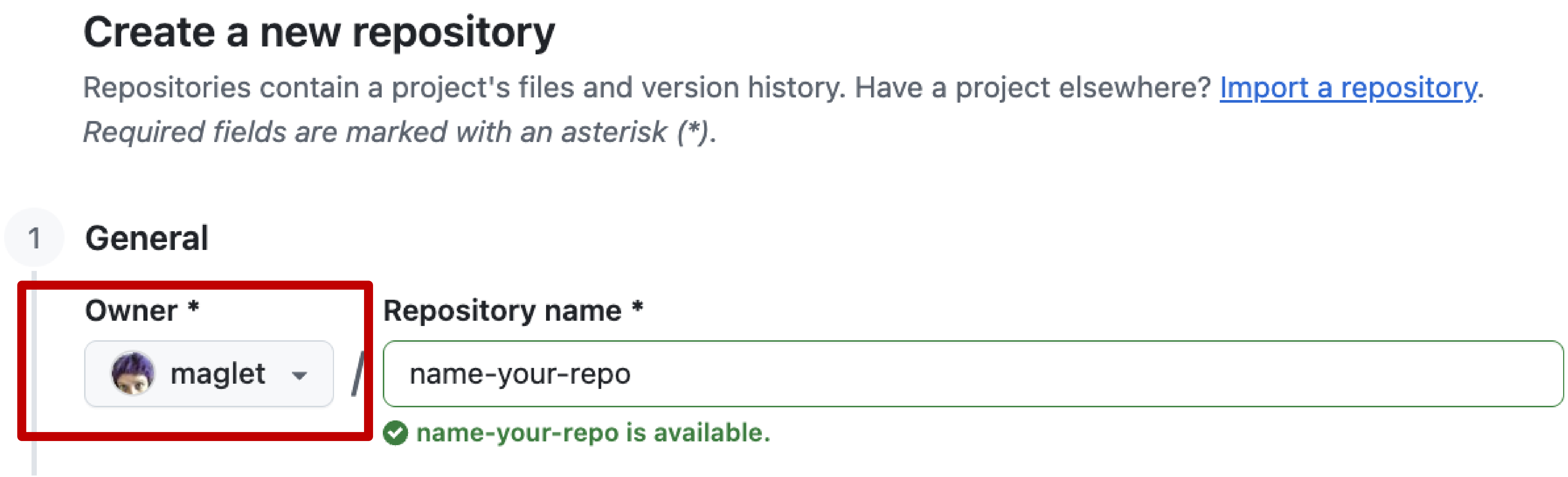
GitHub: Create a repository

A new page will open:

- Choose an owner
- Name your repository
- Provide a description
- Keep it public
- Add a README file

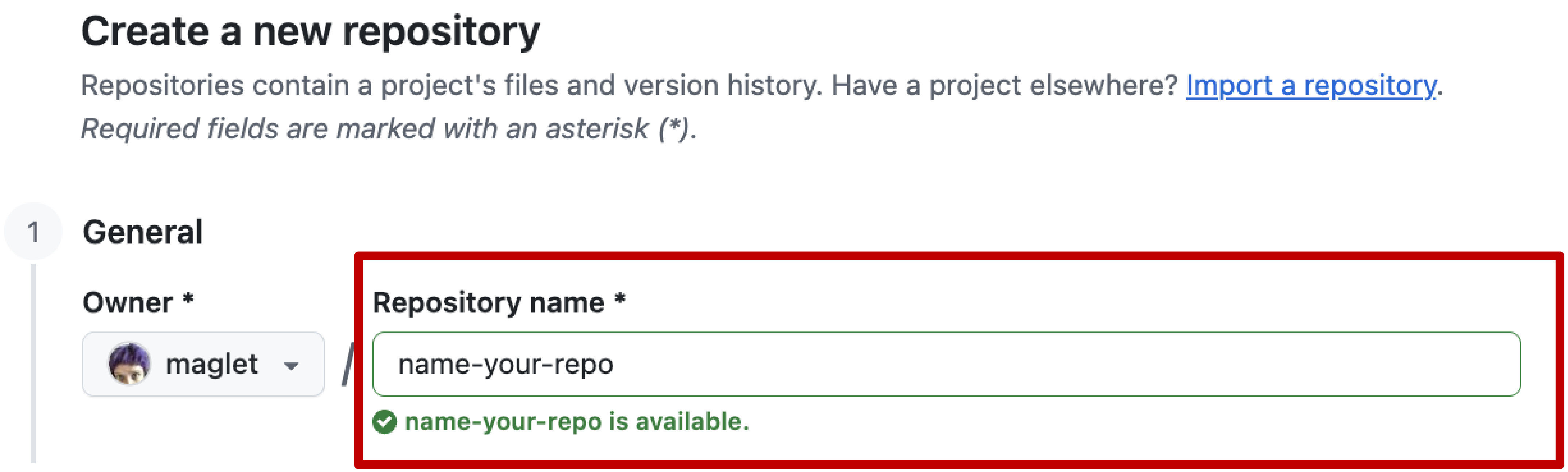


GitHub: Choose an Owner



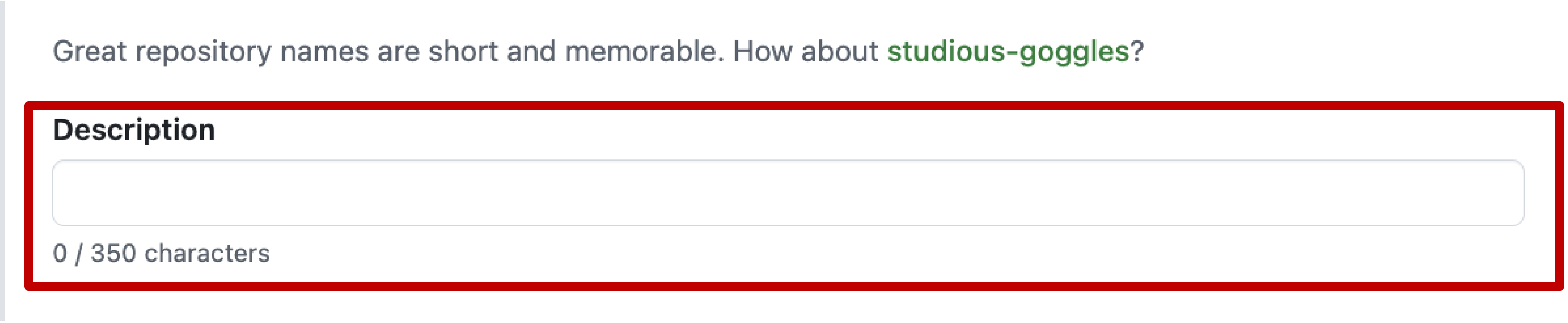
- GitHub repositories can be associated with users or "orgs"
- For this workshop, select your Git username
- If your research group has an org, use that for your research code

GitHub: Name your repository



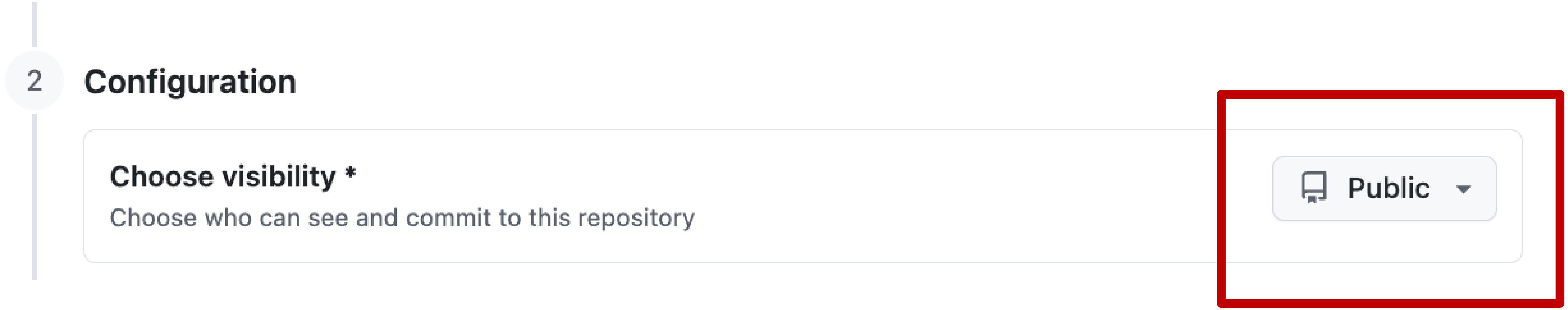
- Make a descriptive name for your repo
- For this workshop, use your last name then "git-workshop"
- Eg: "magle-git-workshop"

GitHub: Provide a description



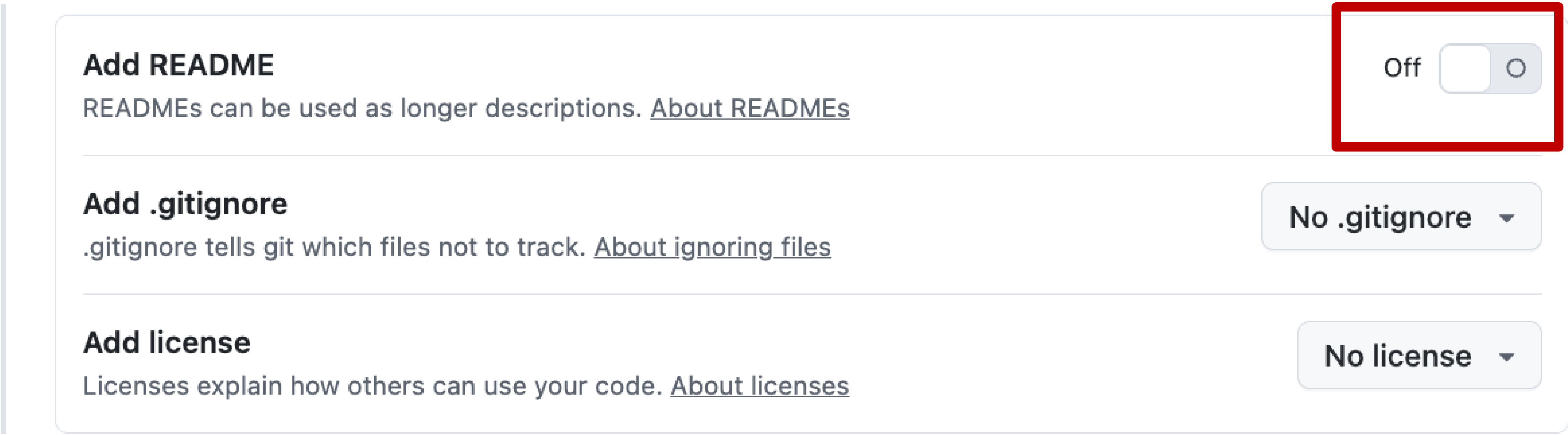
- Write a short description for what this repo is for
- Example: "Test repo for a workshop on using Git in RStudio"
- The descriptions for your research code will be more informative

GitHub: Keep it public



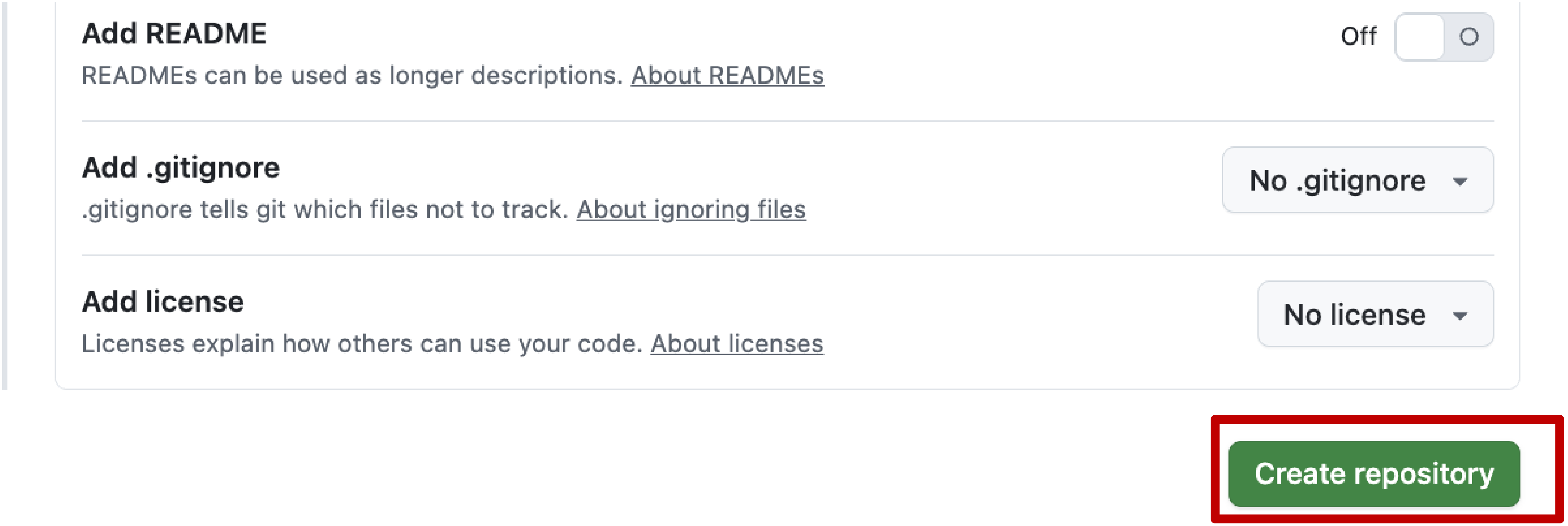
- Keep the repo public for this workshop
- That way we don't have to worry about configuration till we upload changes
- For your research code, you might want to keep it private

GitHub: Add a README file



- A **README file** contains the repo description and any other information that is useful to know about your code.
- Preview of this file will be on the main repo page
- We will edit this file during the workshop to show how to save changes to text files using git.

GitHub: Create repository



- Creates a repository (https://github.com/REPONAME)
- Visible to anyone
- Contains a README with the description you entered previously

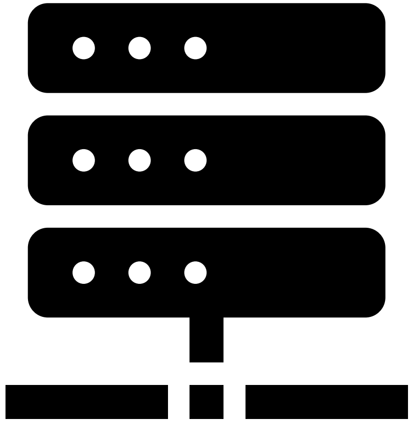
Clone to your computer

Local

GitHub

Clone

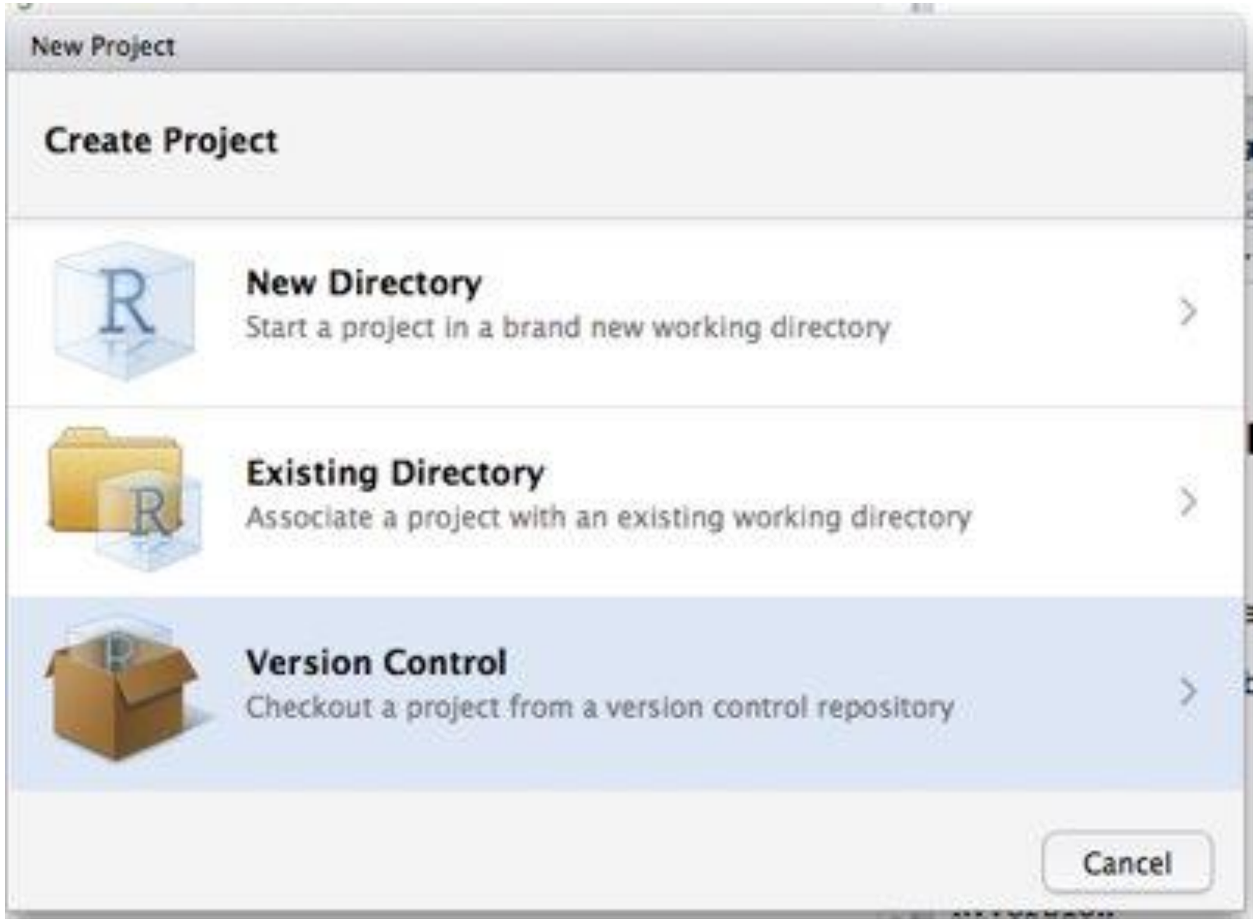
- Download to your computer
- Establish a link between the repo in GitHub and the one on your computer



Remote

RStudio: Make a New Project

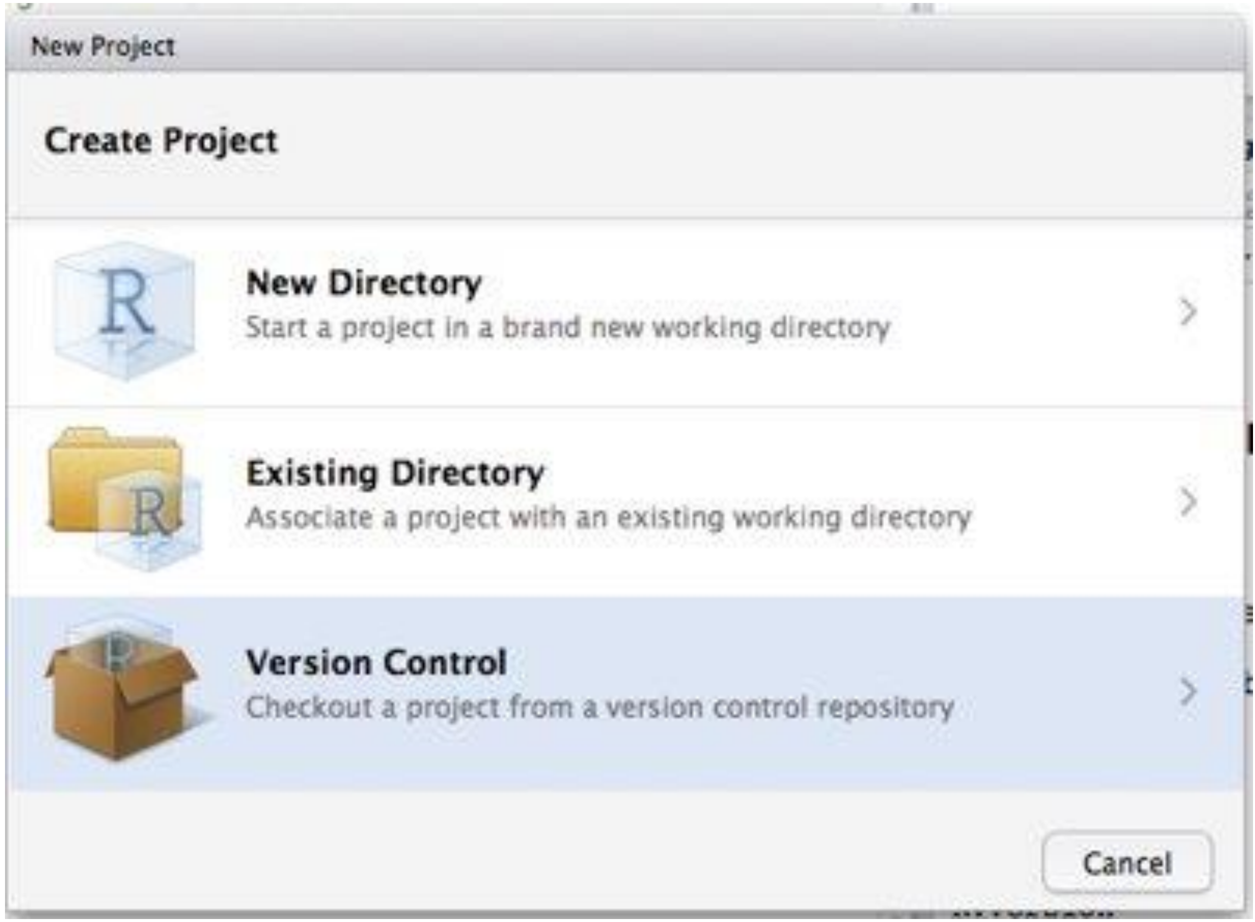
- RStudio projects are very useful for reproducibility
- They also make setting up a GitHub repository from Git very easy



RStudio: Make a New Project

To create a new RStudio Project

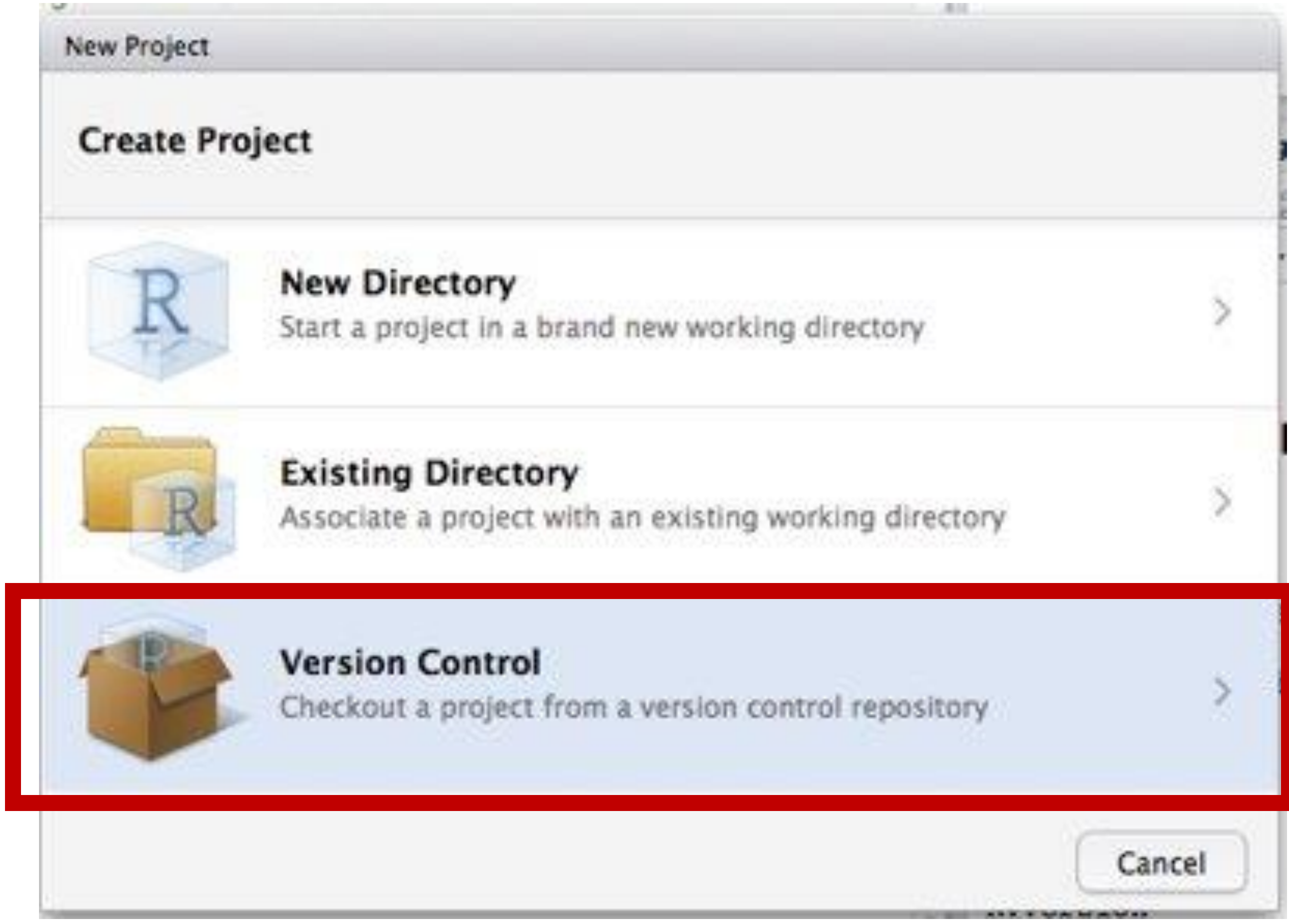
- Select the **File** menu
- Select **New Project**
- A new window will open



RStudio: Make a New Project

To create a new RStudio Project

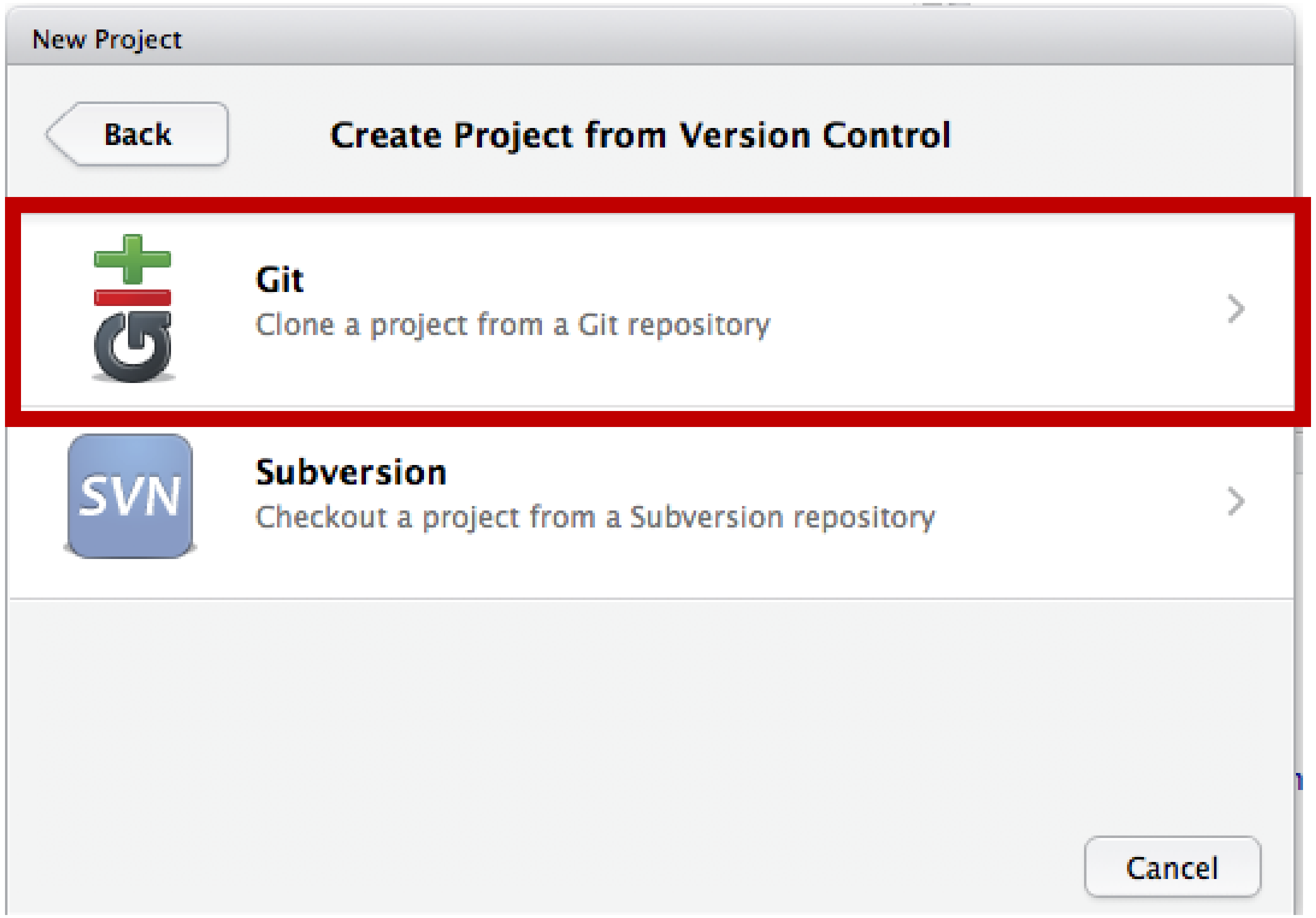
- Select the **File** menu
- Select **New Project**
- A new window will open
- Select **Version Control**



RStudio: Make a New Project

To create a new RStudio Project

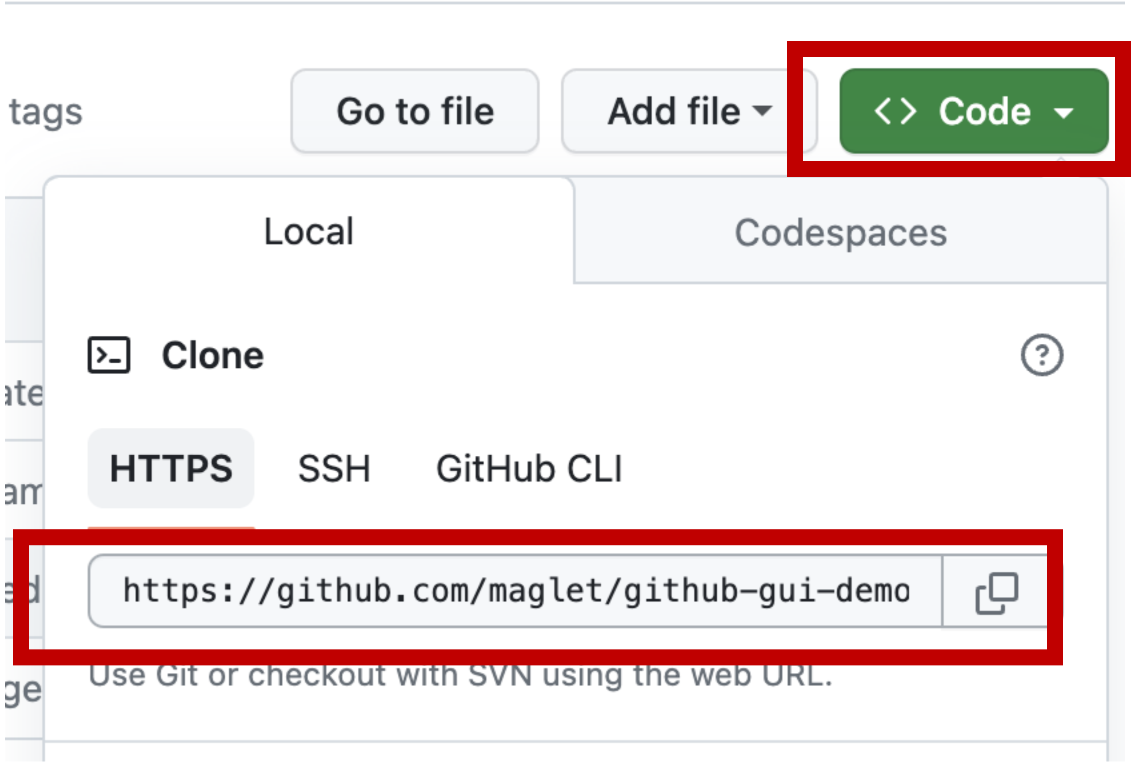
- Select the **File** menu
- Select **New Project**
- A new window will open
- Select **Version Control**
- Select **Git**



RStudio: Make a New Project

To create a new RStudio Project

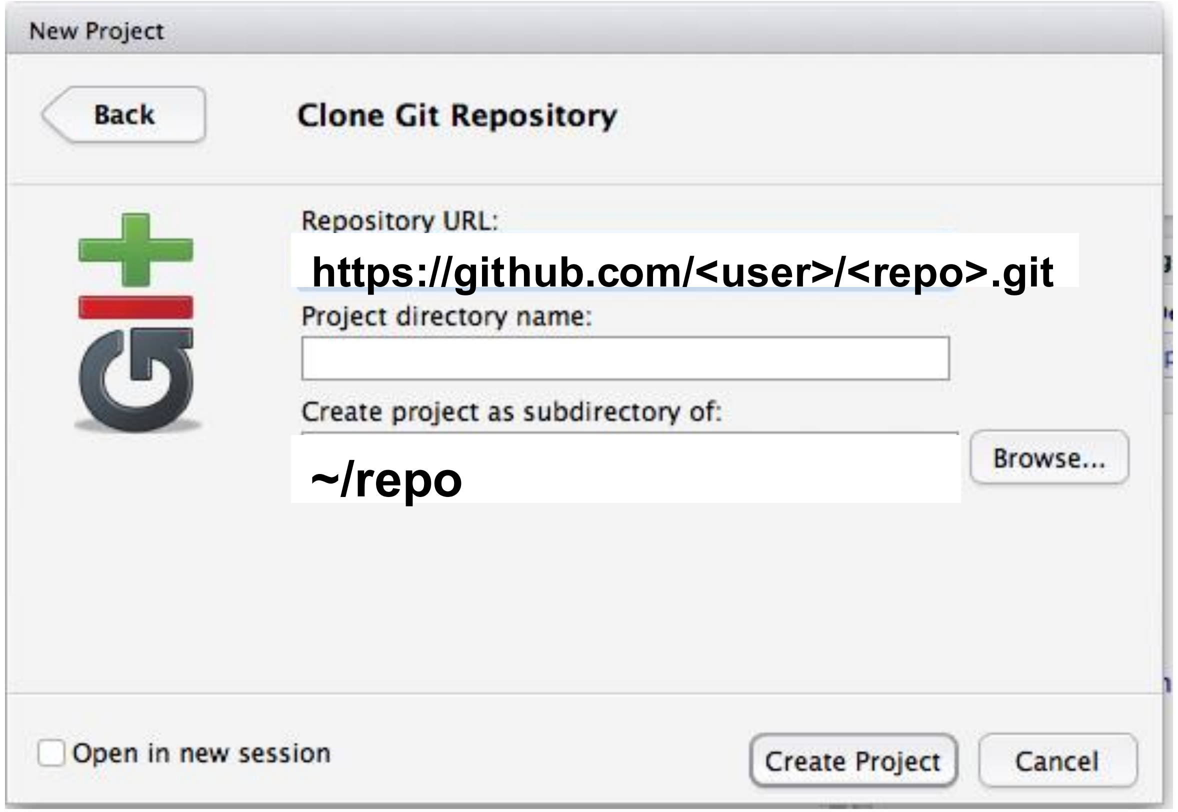
- Select the **File** menu
- Select **New Project**
- A new window will open
- Select **Version Control**
- Select **Git**
- Copy link from GitHub



RStudio: Make a New Project

To create a new RStudio Project

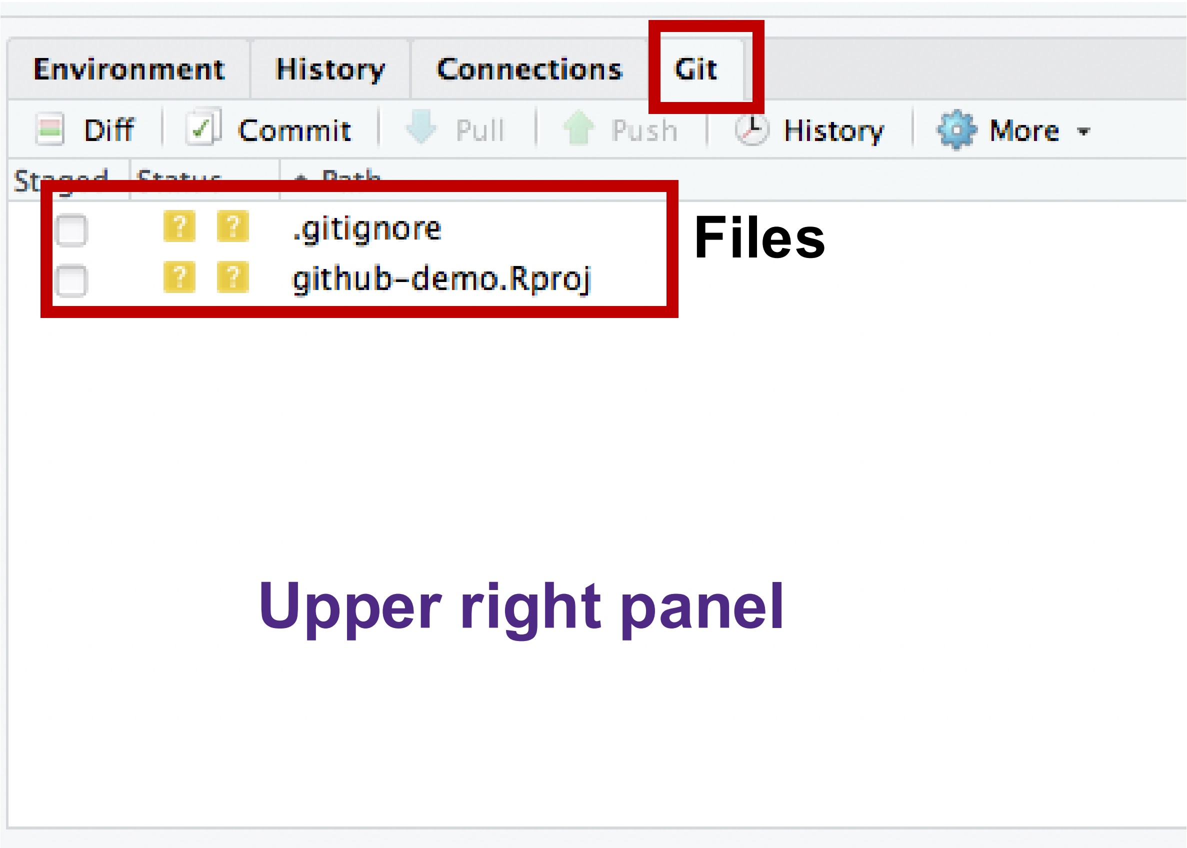
- Select the **File** menu
- Select **New Project**
- A new window will open
- Select **Version Control**
- Select **Git**
- Copy link from GitHub
- Paste into RStudio



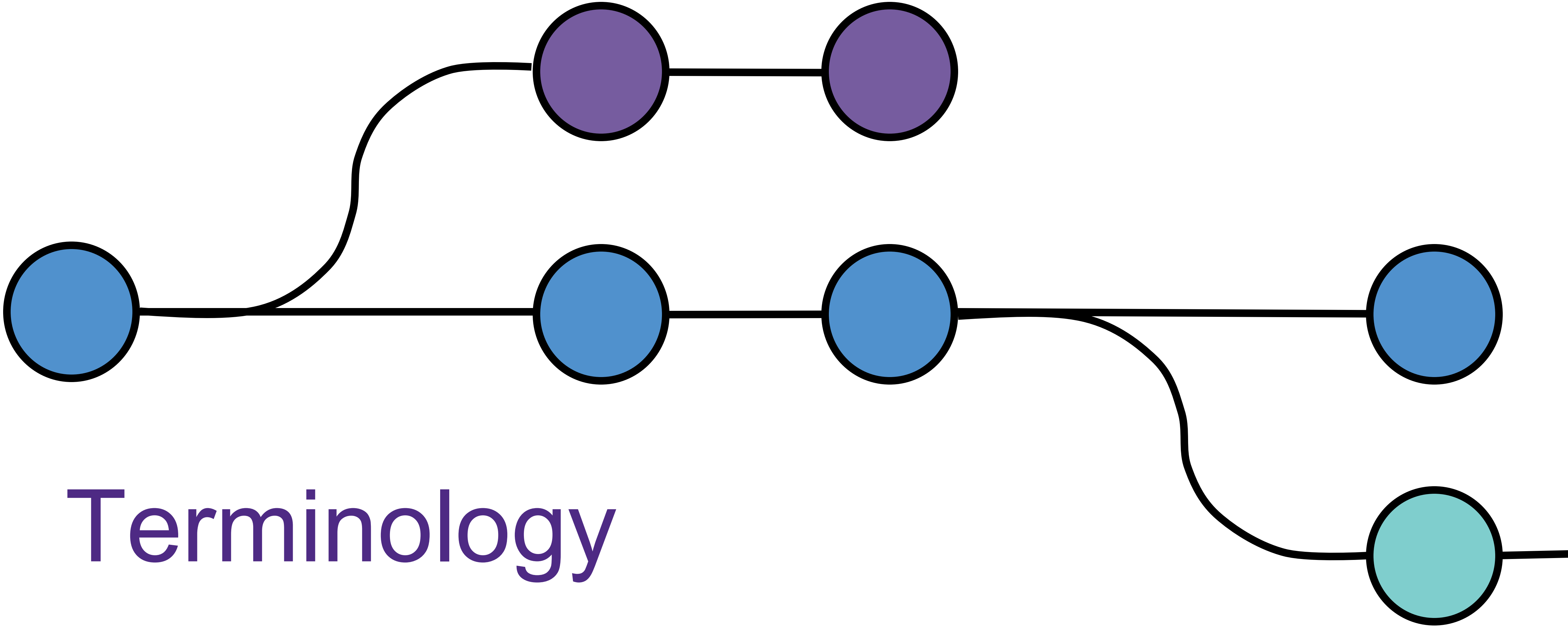
RStudio: Check for git tab

Result: Check for git tab

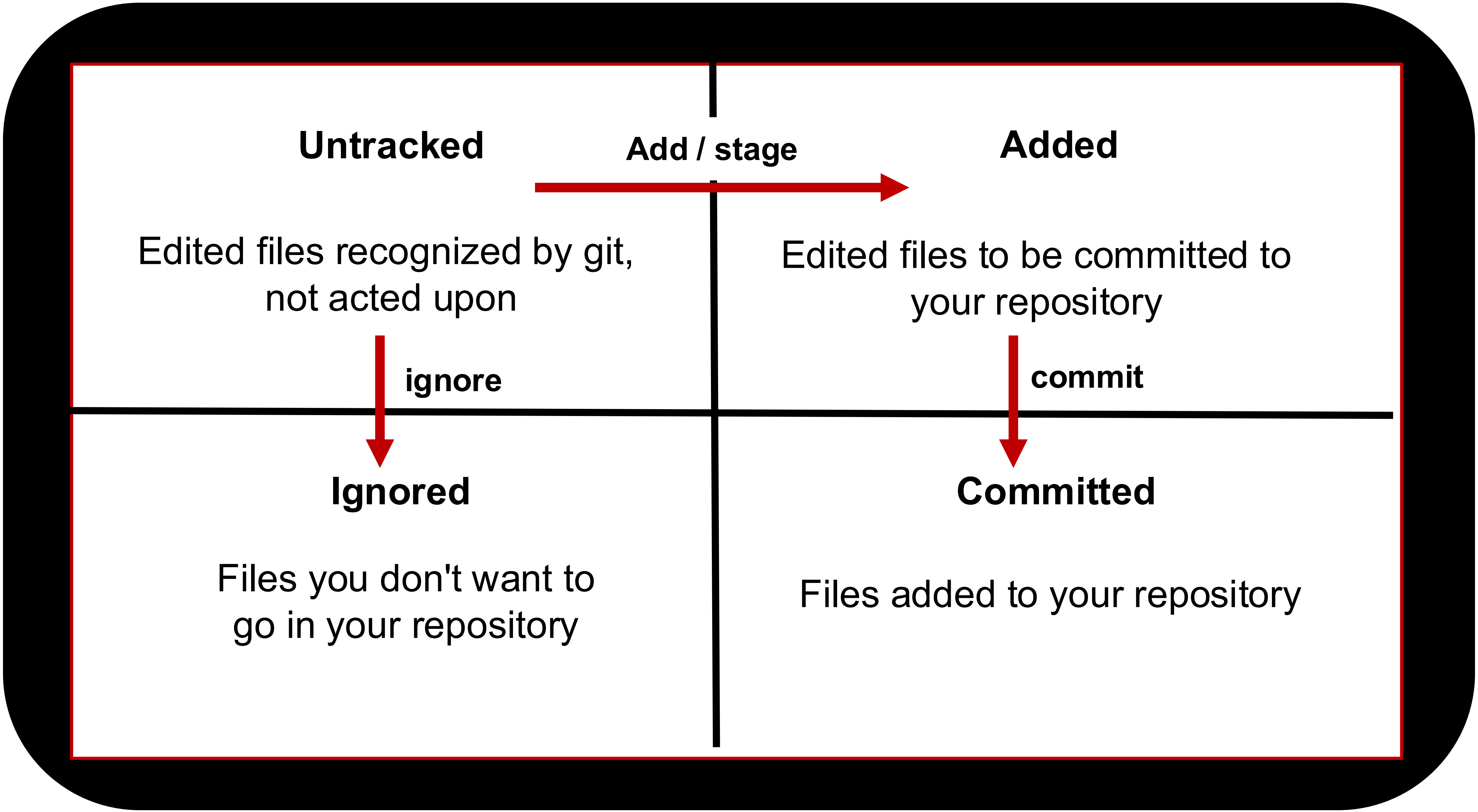
- A new tab will appear in RStudio: Git
- It will show a subset of files in the repository folder
- We'll explain the jargon next (Stage/Add, commit, etc)



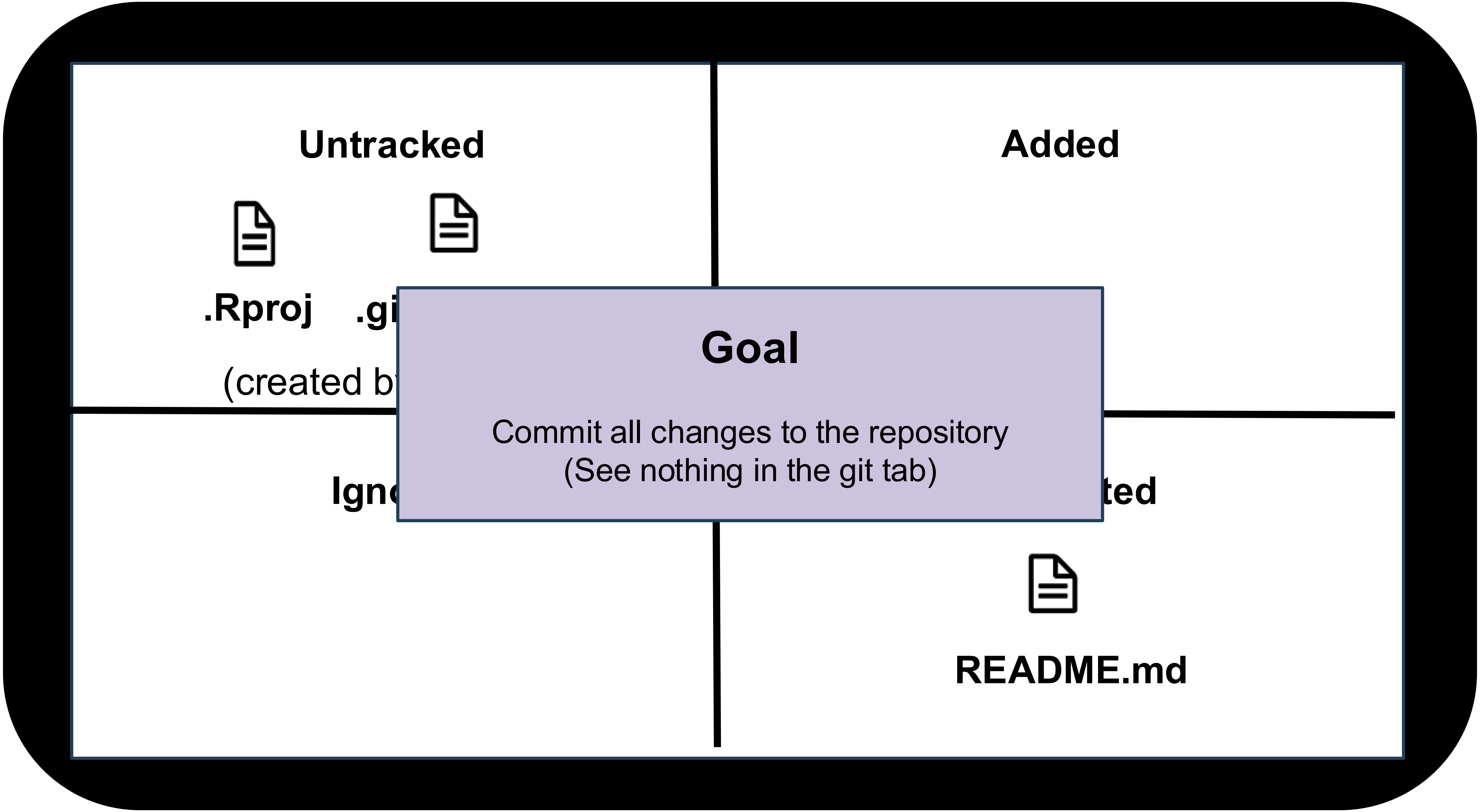
Terminology

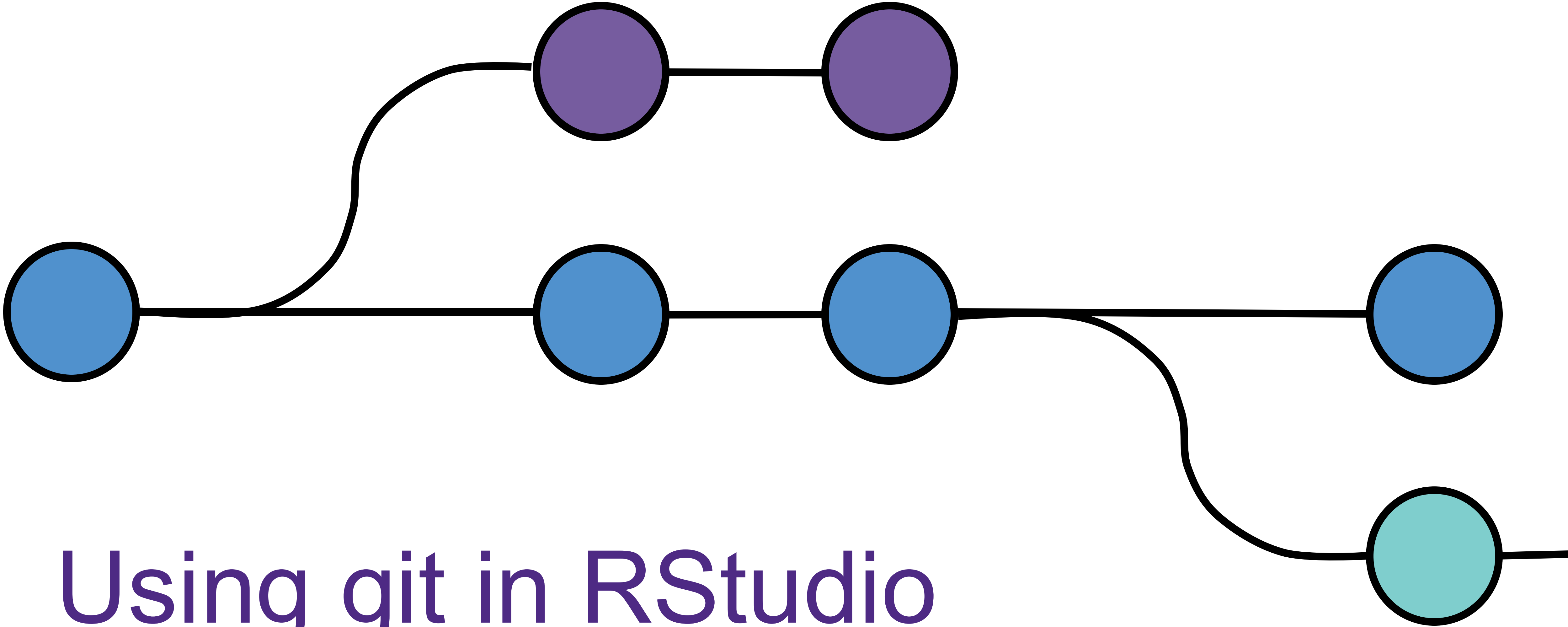


Untracked Edited files recognized by git, not acted upon ?	**Added** Edited files to be committed to your repository A
Ignored Files you don't want to go in your repository (Not seen in git tab)	**Committed** Files added to your repository (Not seen in git tab)


Untracked
Add / stage
Added
Edited files recognized by git,
not acted upon
Edited files to be committed to
your repository
ignore
commit
Ignored
Committed
Files you don't want to
go in your repository
Files added to your repository

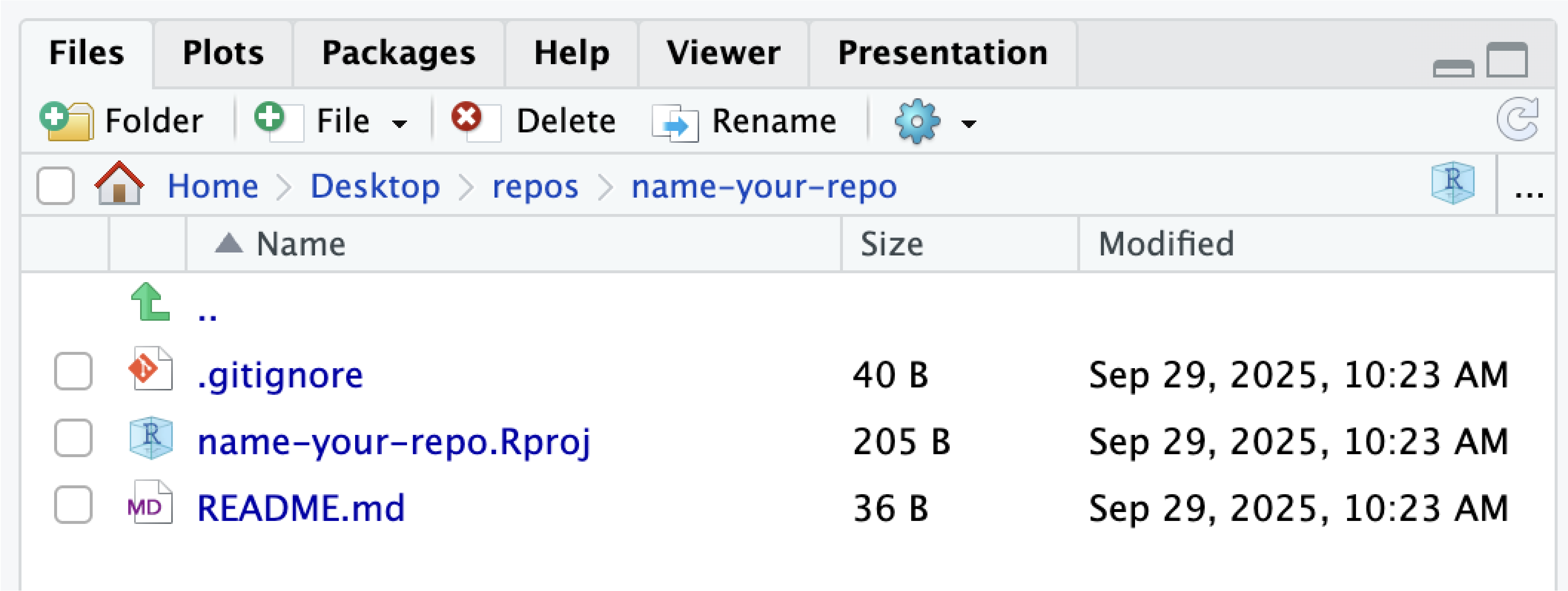

Untracked
.Rproj
.gitignore
(created by RStudio/git)
?
Added
A
Ignored
Committed
README.md


Untracked
Added
.Rproj
.gi
(created b
Ign
ted
README.md
Goal
Commit all changes to the repository
(See nothing in the git tab)



Using git in RStudio

Files in repository folder

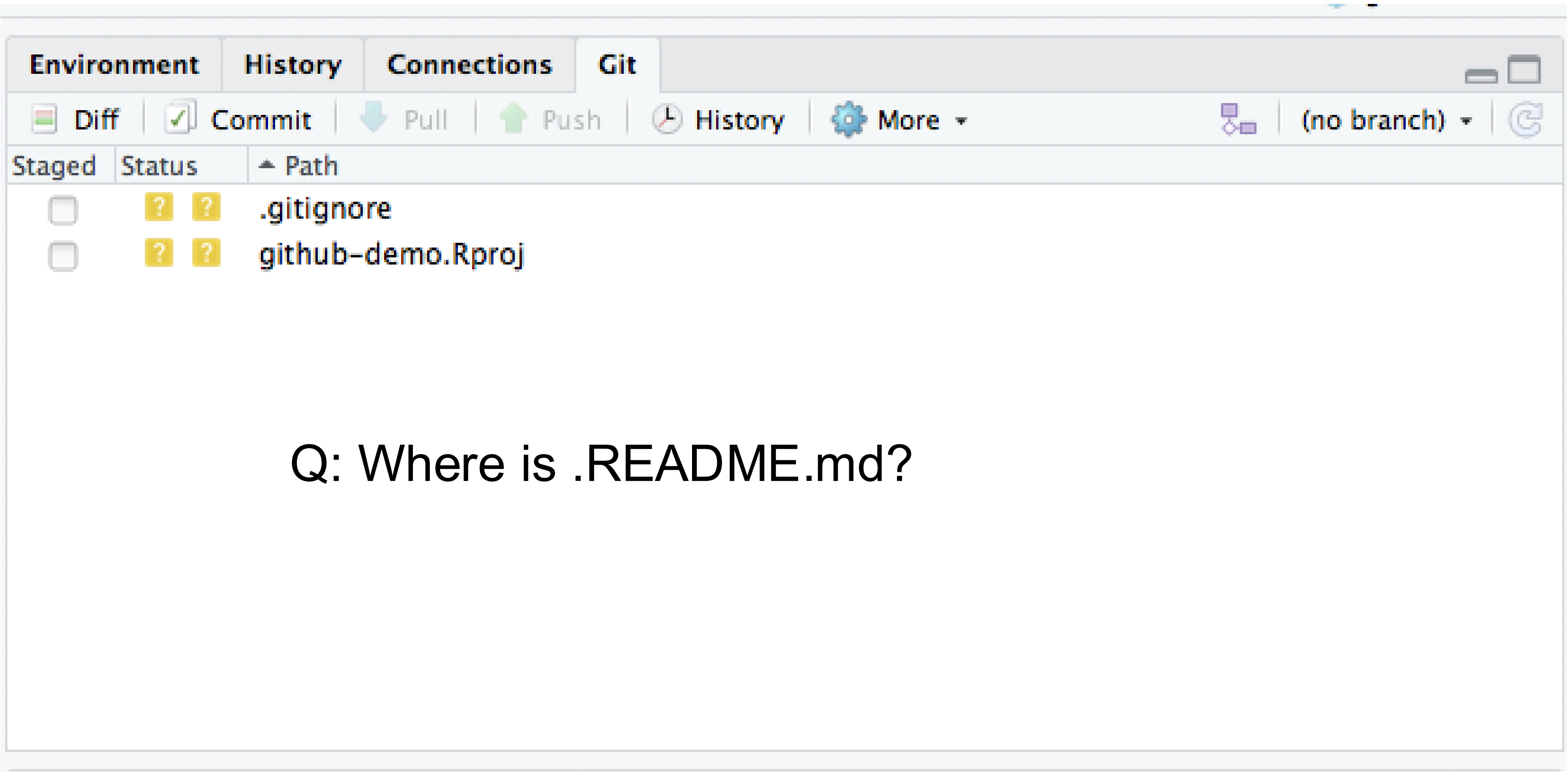


From git

From RStudio

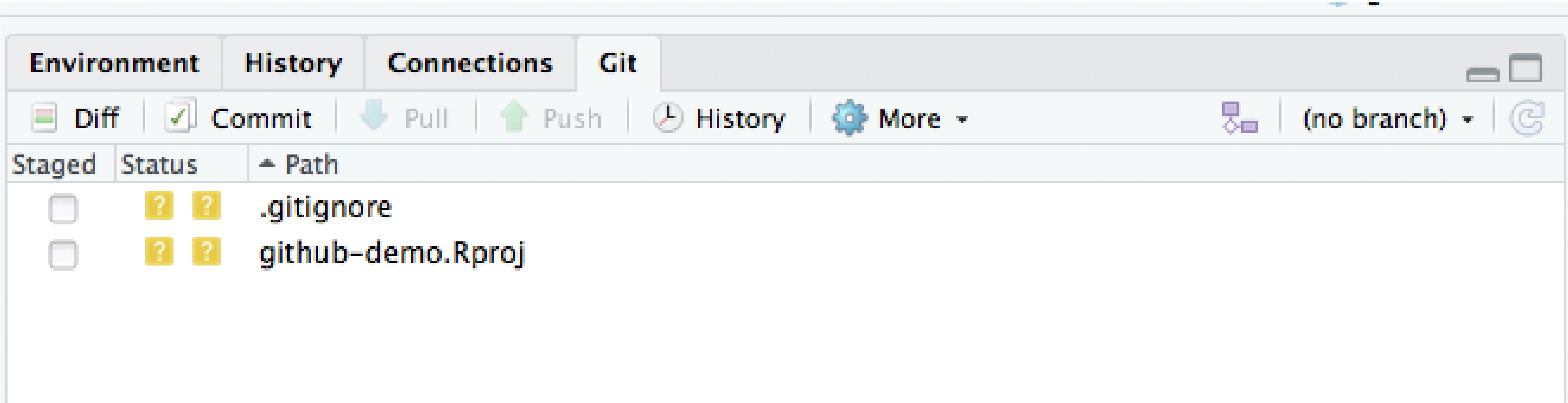
From GitHub

Files with uncommitted changes



Q: Where is .README.md?

Files with uncommitted changes



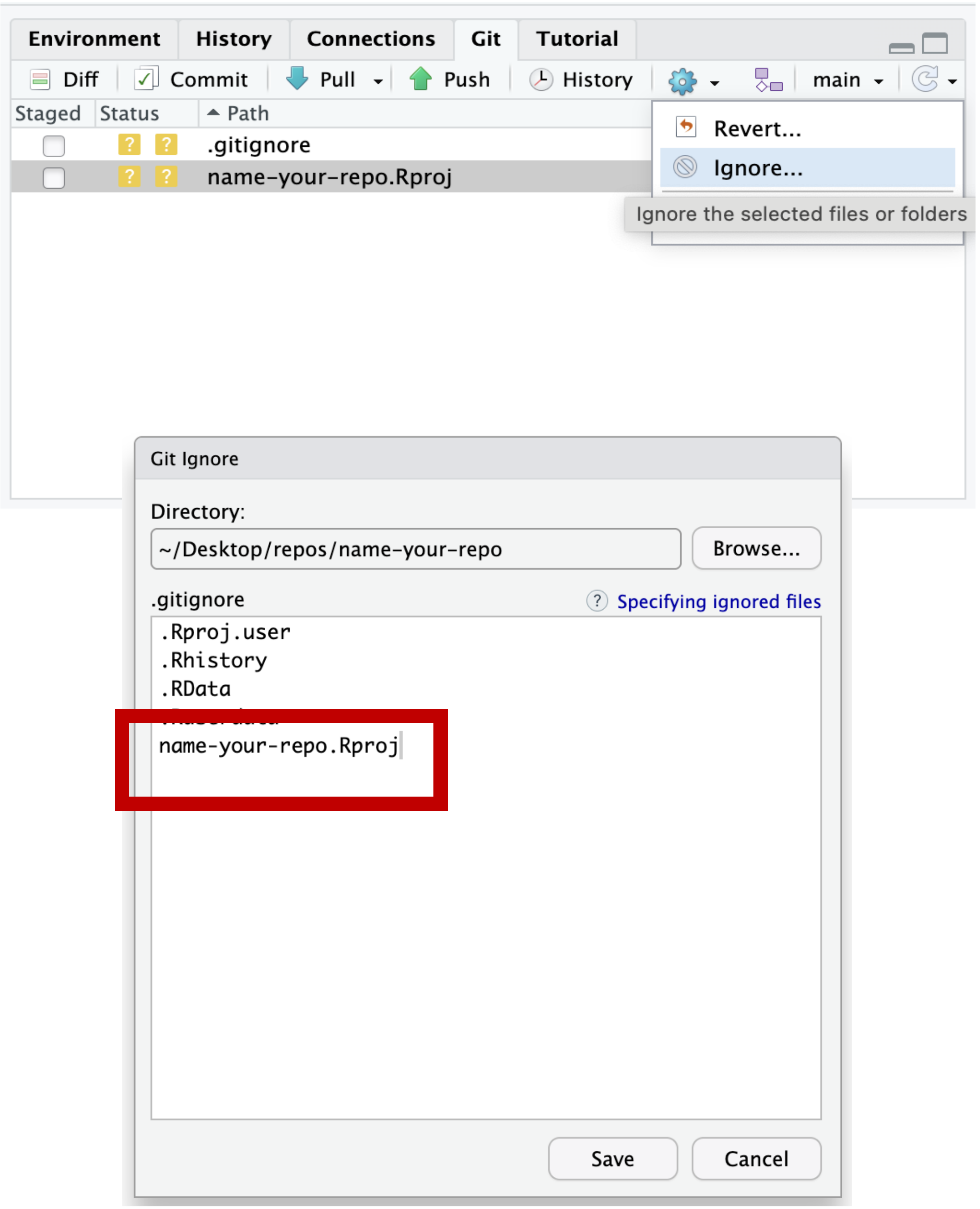
Q: Where is .README.md?

A: No unsaved changes – not in Git window


Untracked
.Rproj
.gitignore
?
Added
A
Ignored
Committed
README.md

Ignoring files

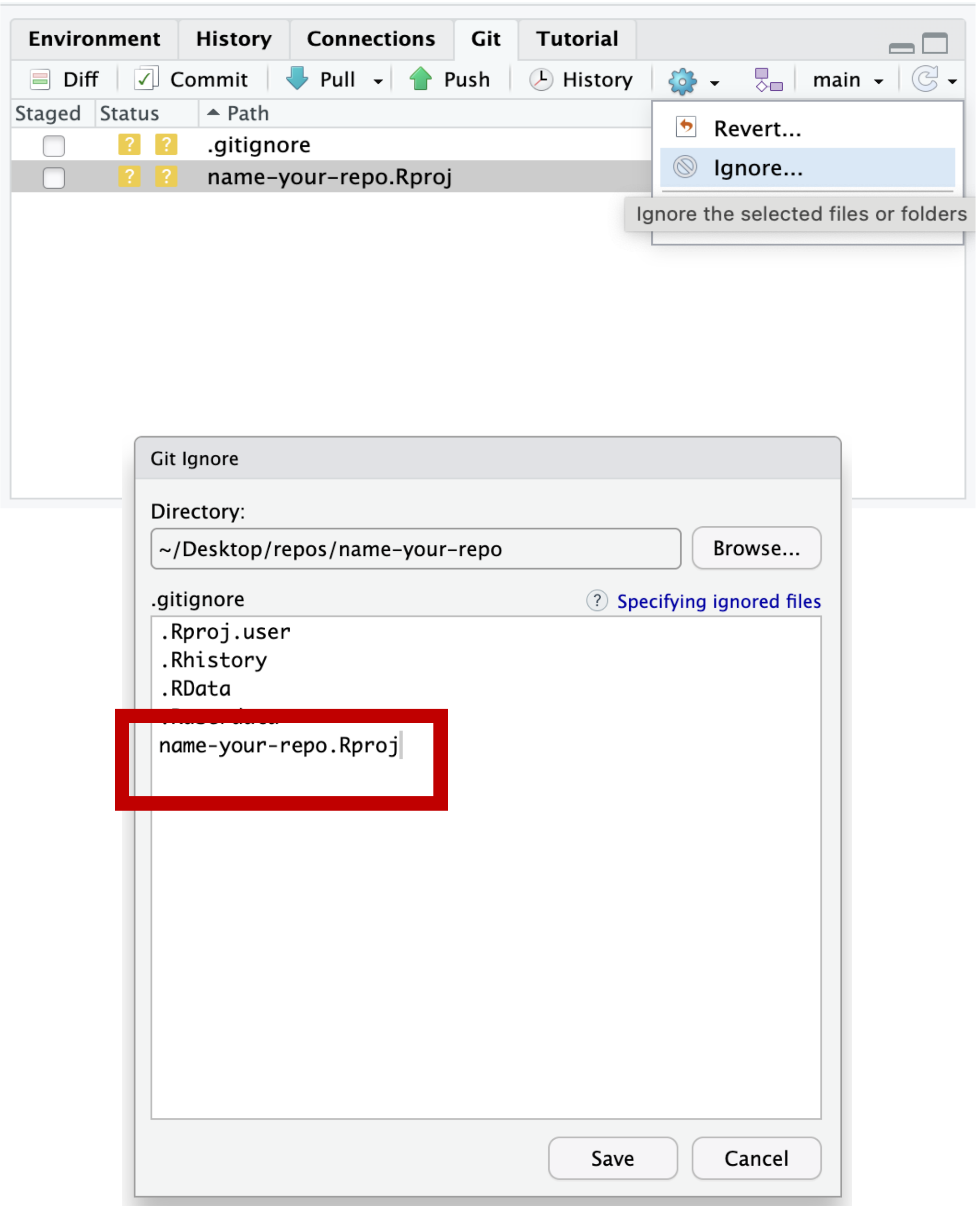
- Git allows you to ignore filesthat you don't want to track
- .Rproj files are a good example, as it's specific to the filesystem on your computer
- Added the R.proj file to the ignore list

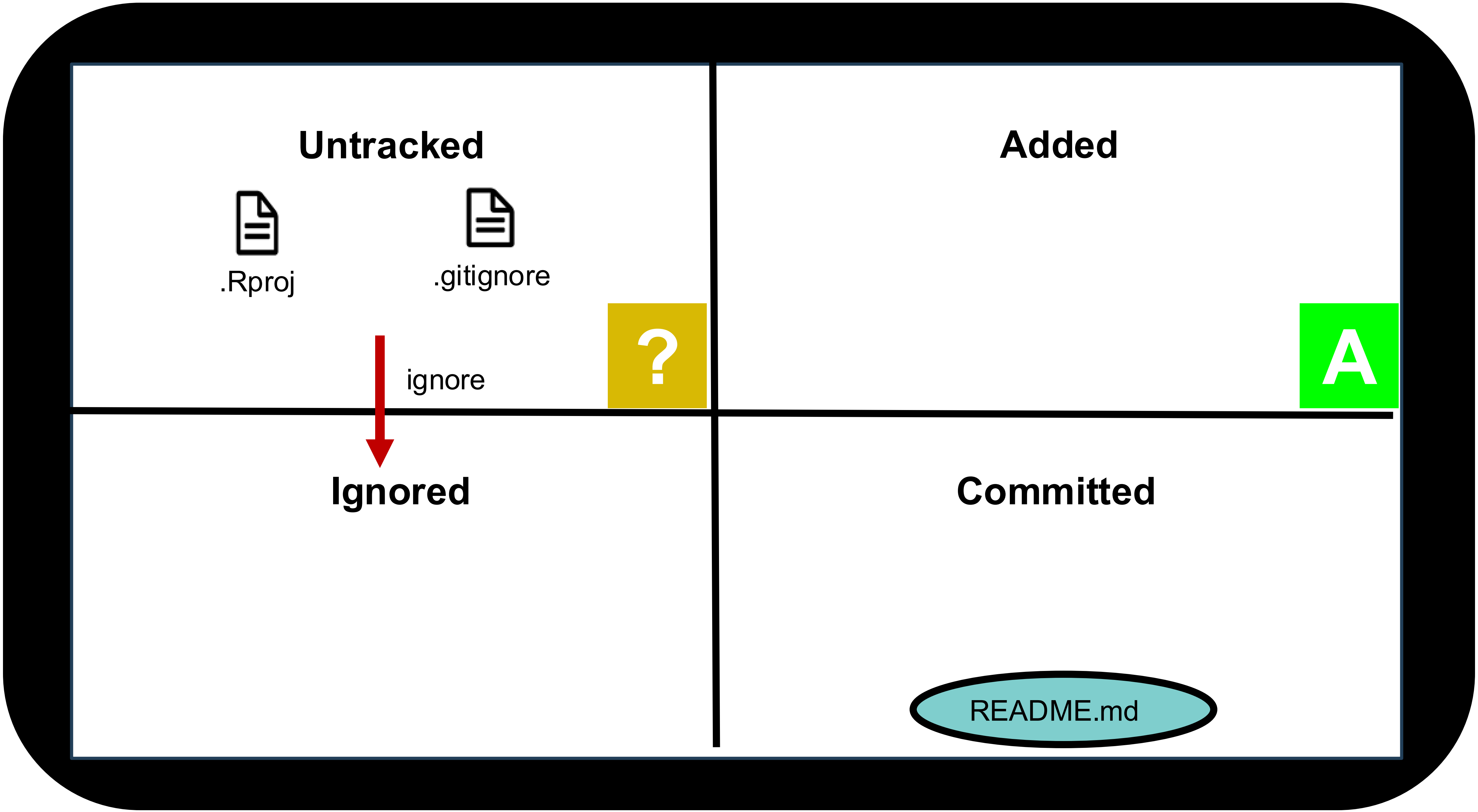


Ignoring files

To ignore a file:

1. Highlight the file you want to ignore
2. Click the gear in the git tab
3. Click ignore – the file name will appear in the ignore list
4. Click Save



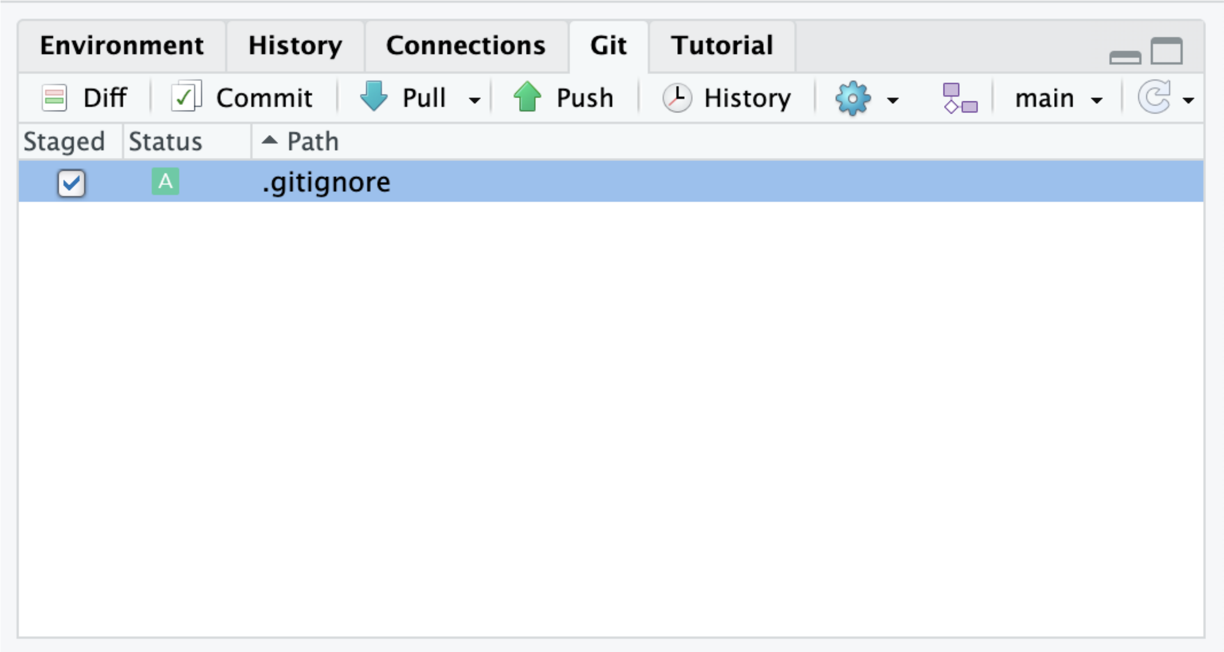

Untracked
.Rproj
.gitignore
ignore
?
Added
A
Ignored
Committed
README.md


Untracked
.gitignore
?
Added
A
Ignored
.Rproj
Committed
README.md

Adding/Staging files

The first step to saving your changes to your repo is to **Add/Stage** the file

- Add is git terminology
- Stage is RStudio terminology
- Adds files to a "Staging area"
- Allows you to combine changes to multiple files into one "Save"



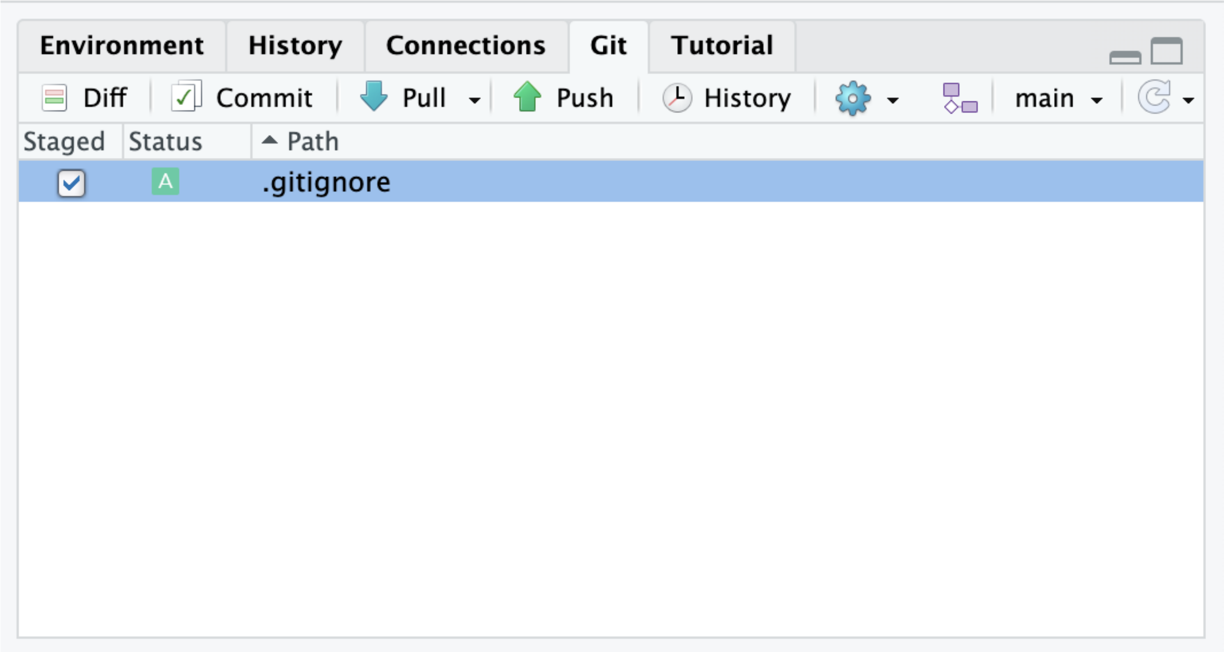
Adding/Staging files

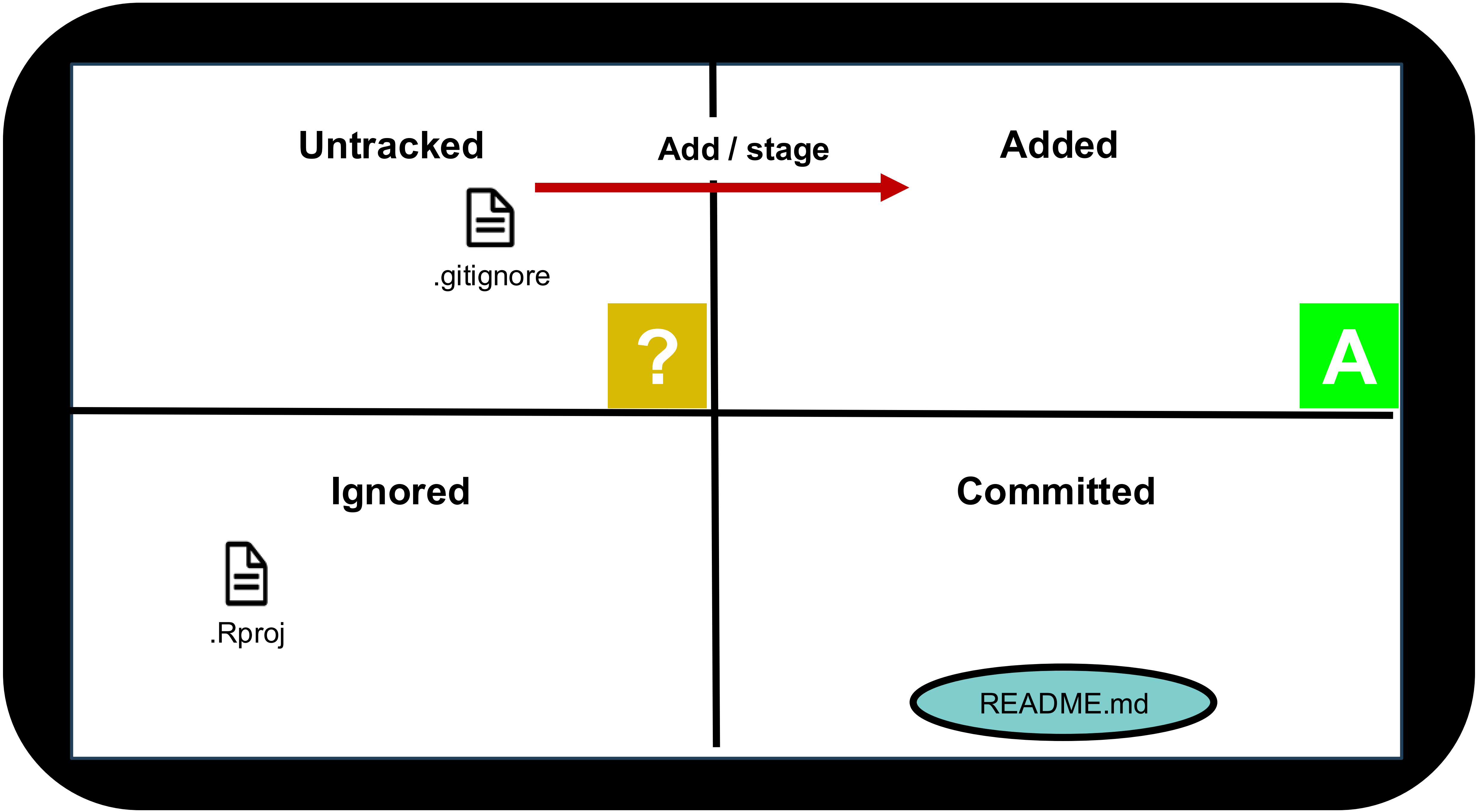
To stage a file:

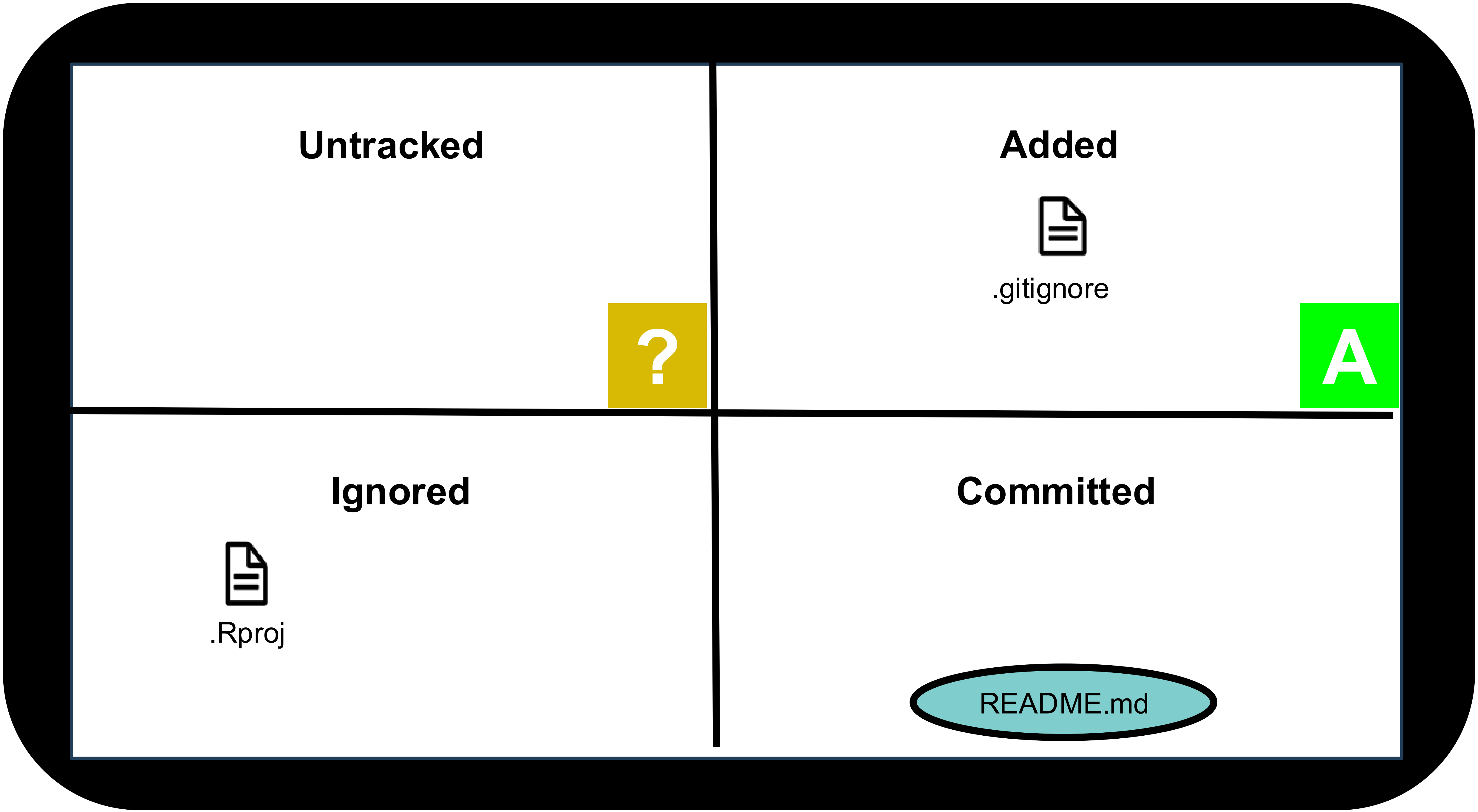
1. Check boxes to add files to the “staging area”

2. **Status**: A for Added

3. Staging area compiles changes to be committed to the repository

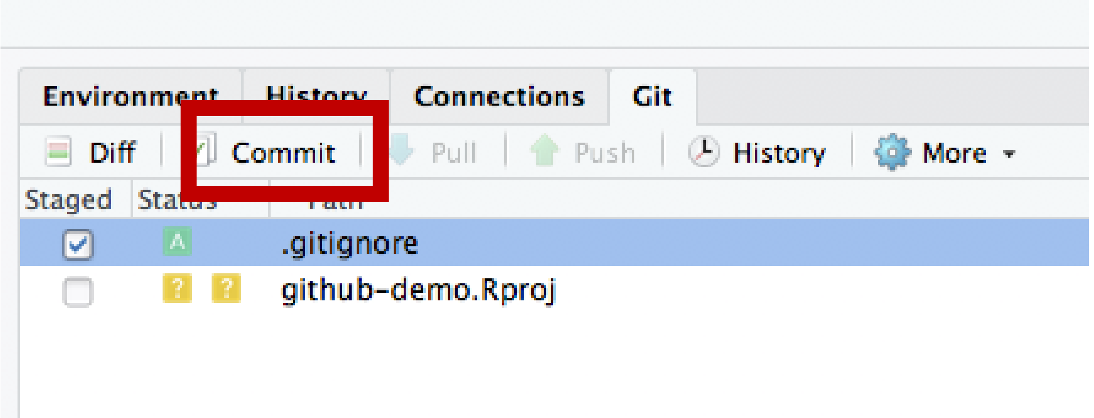



Untracked
Add / stage
Added
.gitignore
?
A
Ignored
Committed
.Rproj
README.md


Untracked
?
Added
.gitignore
A
Ignored
.Rproj
Committed
README.md

Committing files

After adding files to the staging area, you must **commit** them to save them to your repo

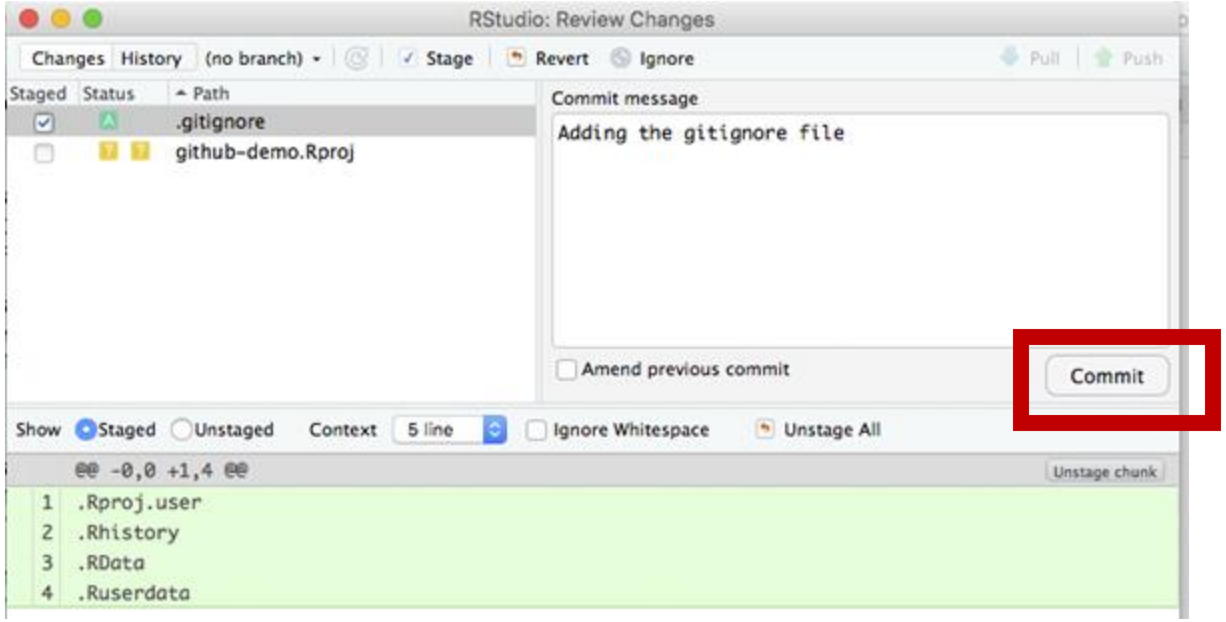


- The **commit** action adds all files from the staging area to the repository
- Git will ask you for a **commit message** that describes what you changed in this commit

Committing files

You will also be able to see what has changed by looking at the bottom of the screen

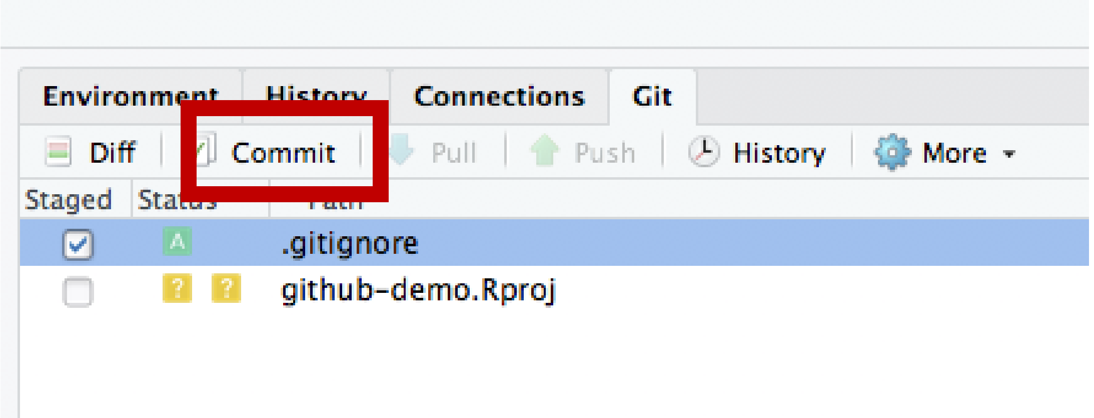
- Text that was added will be in Green
- Text that was removed will be in Red

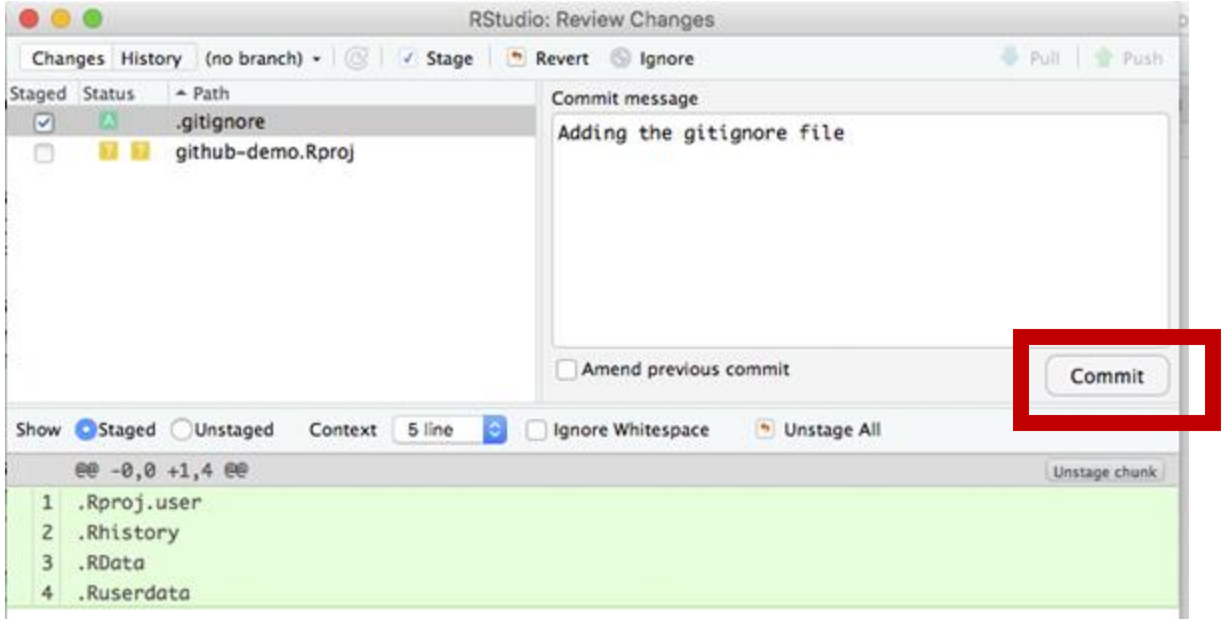


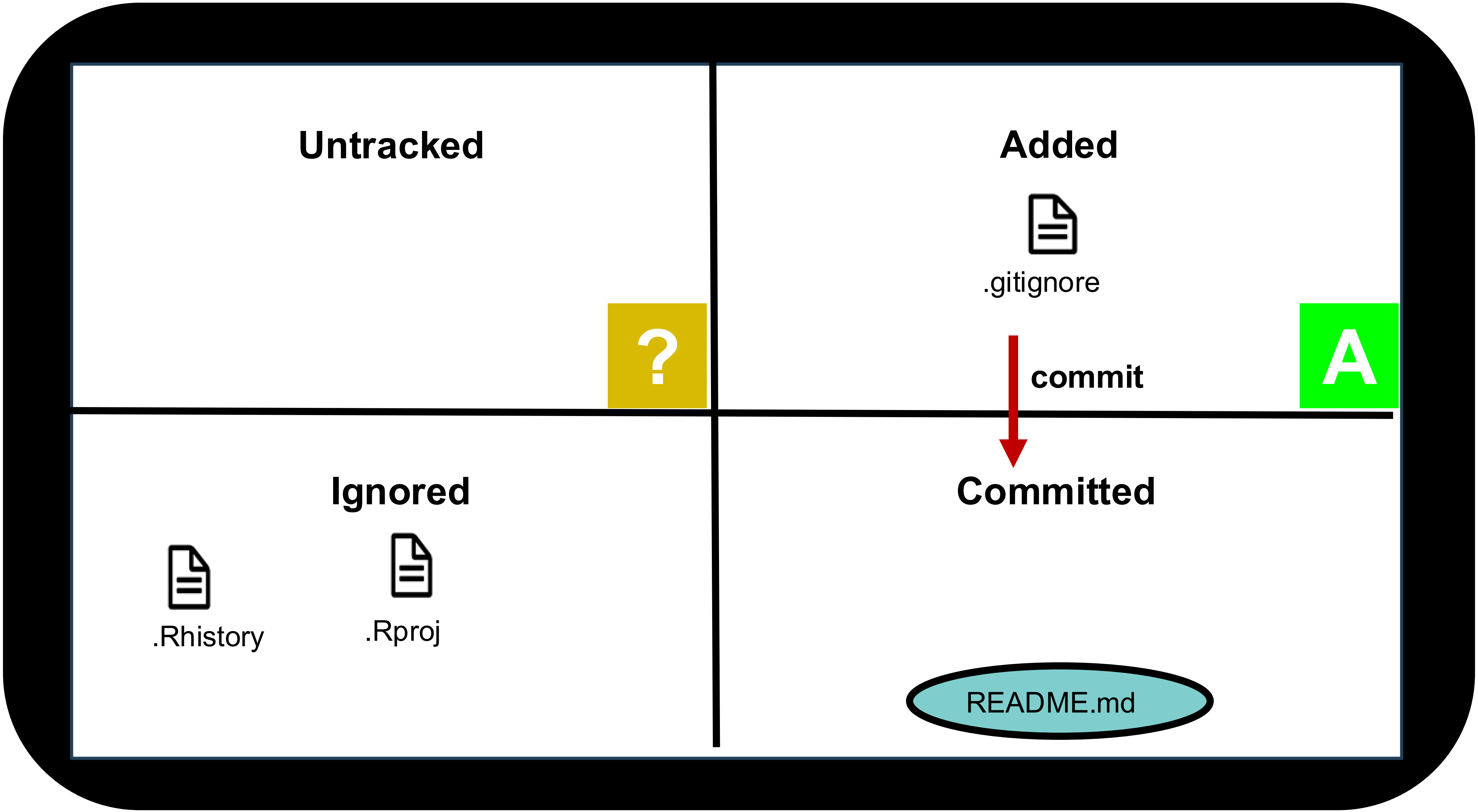
Committing files

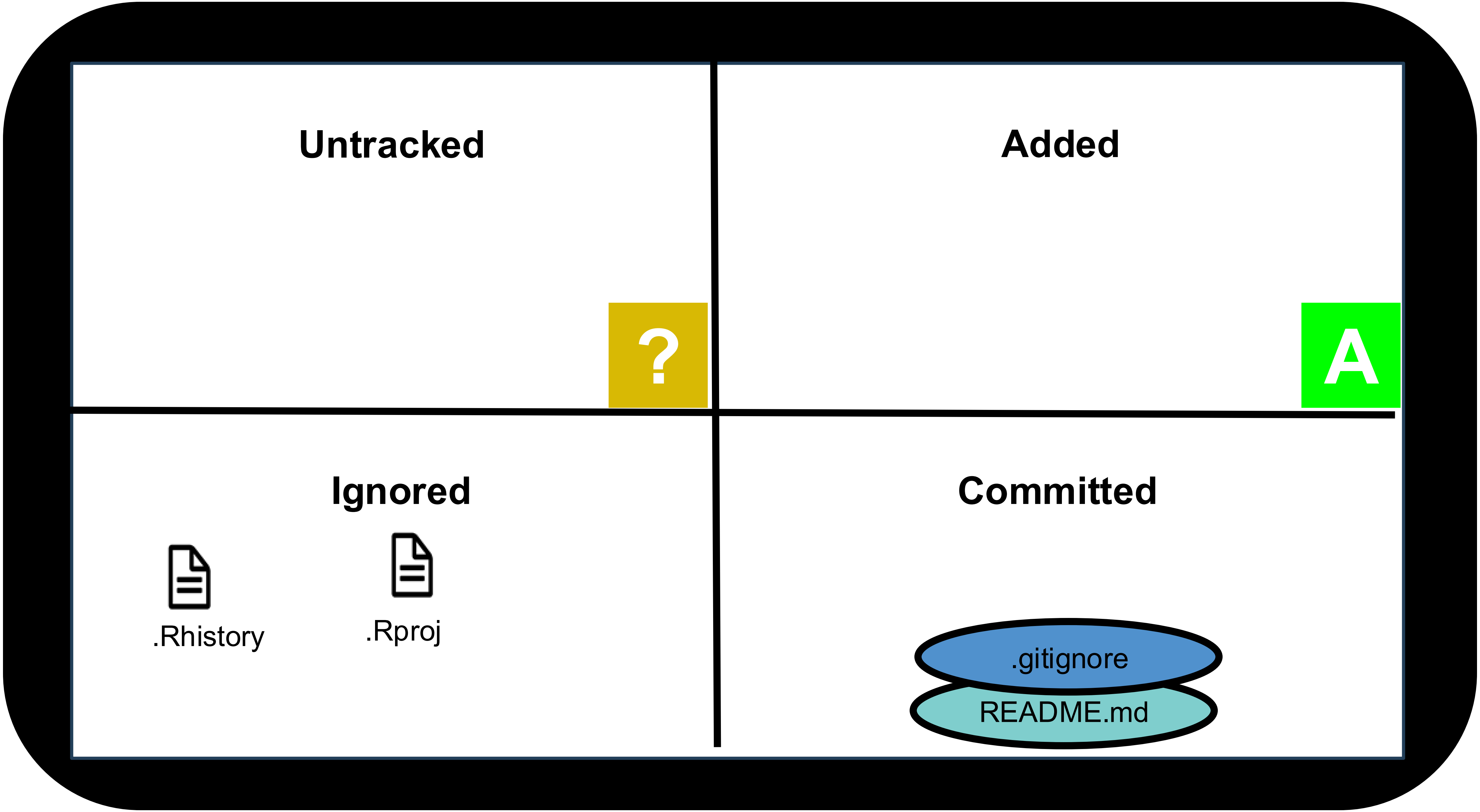
To commit your file(s):

1. Click the **Commit** button in the Git tab
2. Type a description of your changes in the **commit message** box
3. Click **Commit**



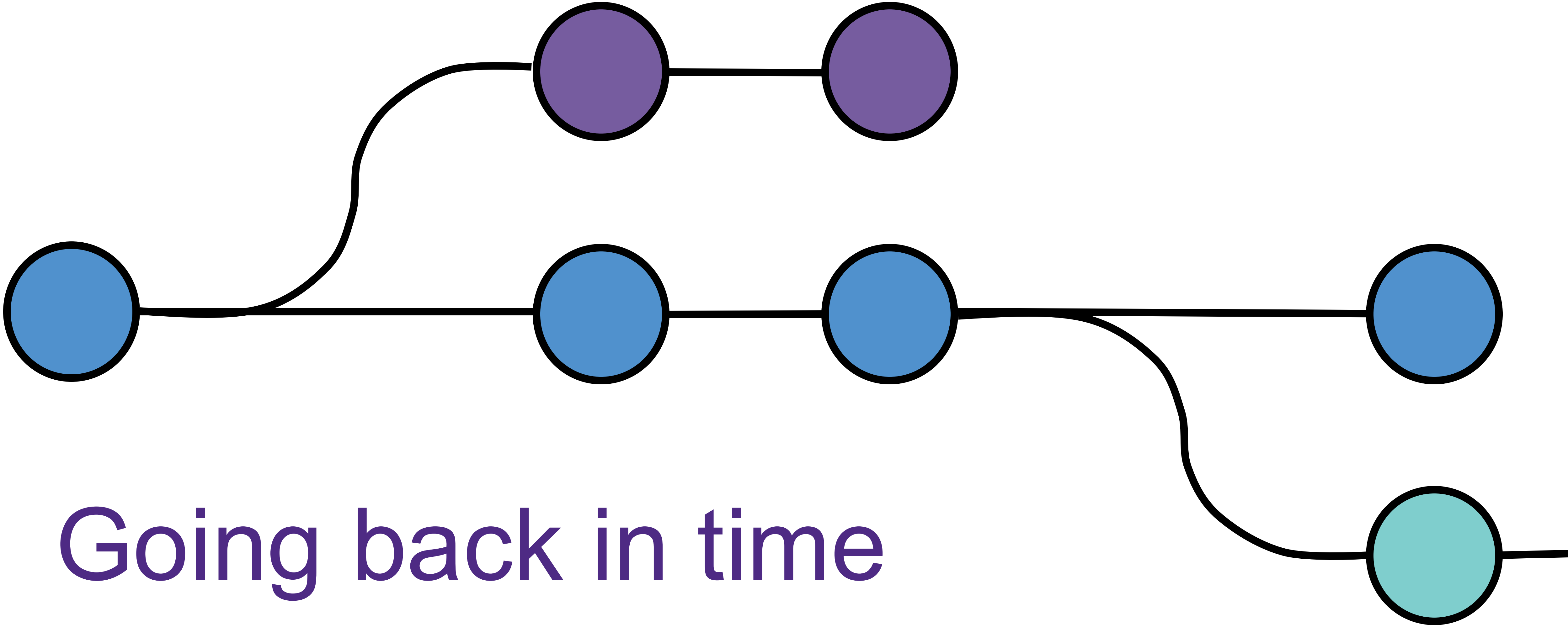



Untracked
?
Added
.gitignore
commit
A
Ignored
.Rhistory
.Rproj
Committed
README.md


Untracked
?
Added
A
Ignored
.Rhistory
.Rproj
Committed
.gitignore
README.md

Exercise:

1. Edit your README.md file
2. **Stage changes: Add** the changes file to the **staging area**
3. **Commit** your staging area to the **repository**

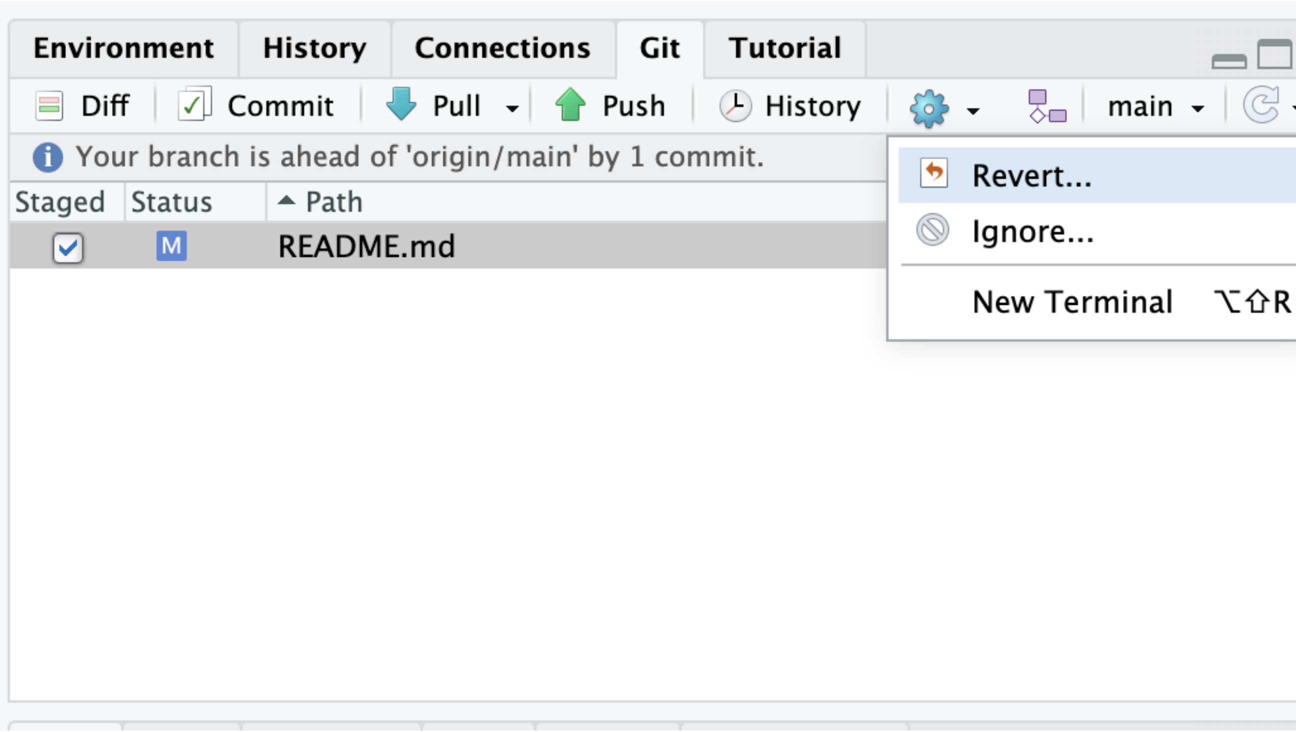


Going back in time

Reverting changes

- Removes all uncommitted changes
  - Unstaged changes
  - Staged changes

- Can’t go back to previous commits within the from R Studio IDE



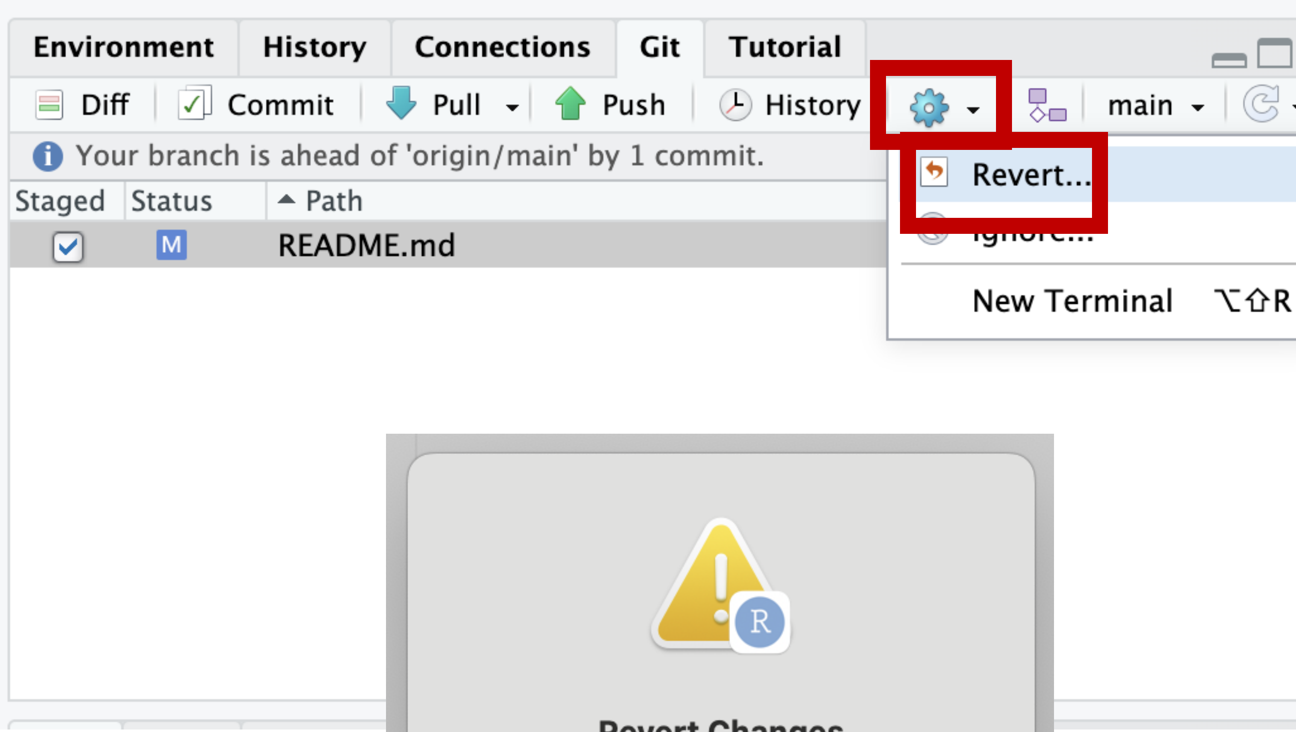
https://intro2r.com/use_git.html#undo

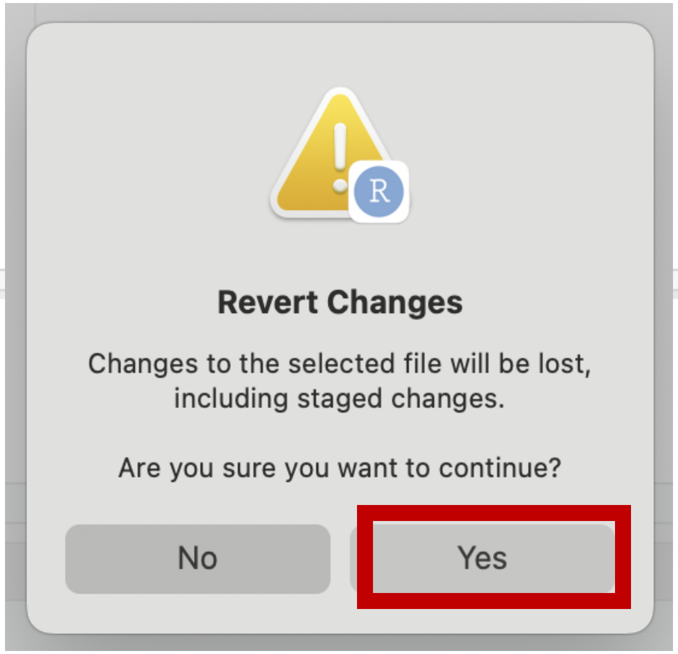
Demo: what can you revert?

1. Add some lines to your README.md file

2. Don't add or commit them > Revert

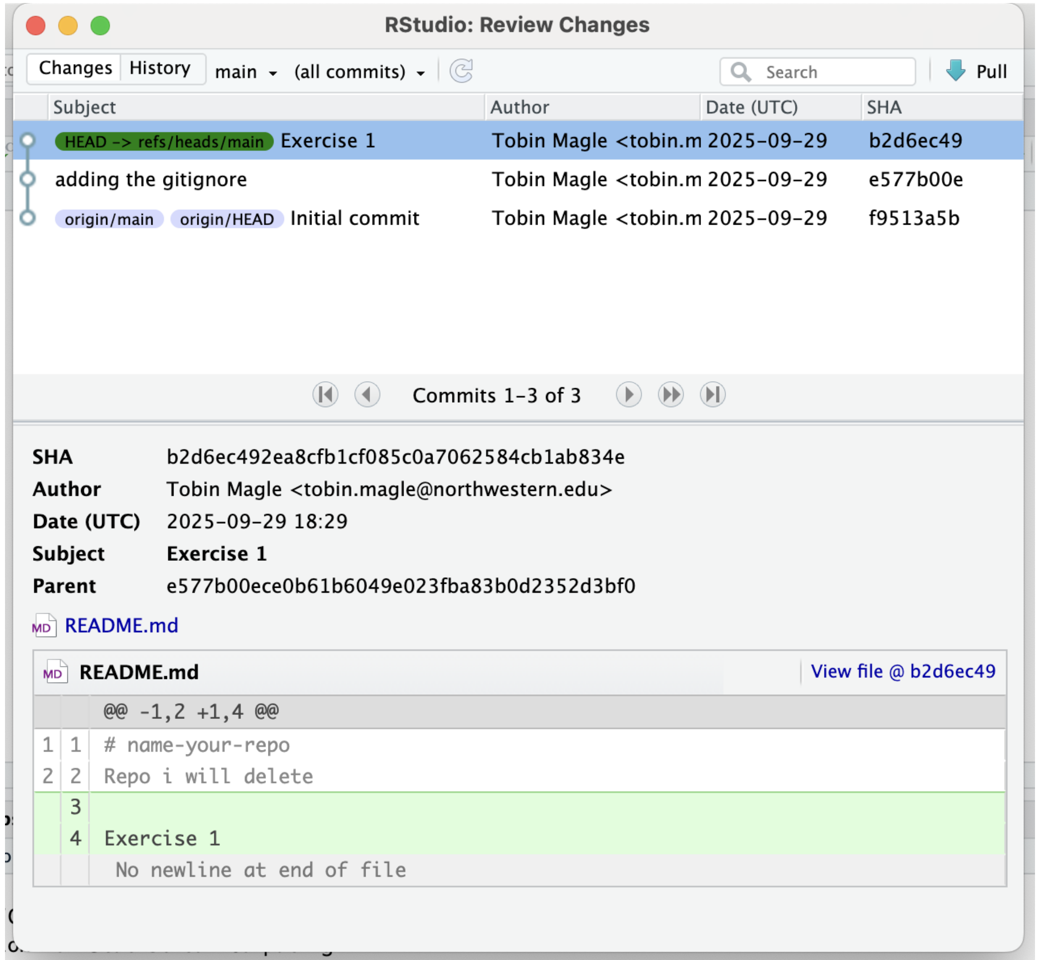
3. Add them, but don't commit them > Revert





Viewing previous commits

- The **History** window allows you to see previous commits
- One line for each commit
  - Commit message
  - Who made the commit
  - When they made it
  - SHA (aka hash) - unique identifier for the commit
- Click on the commit to see what changes were made



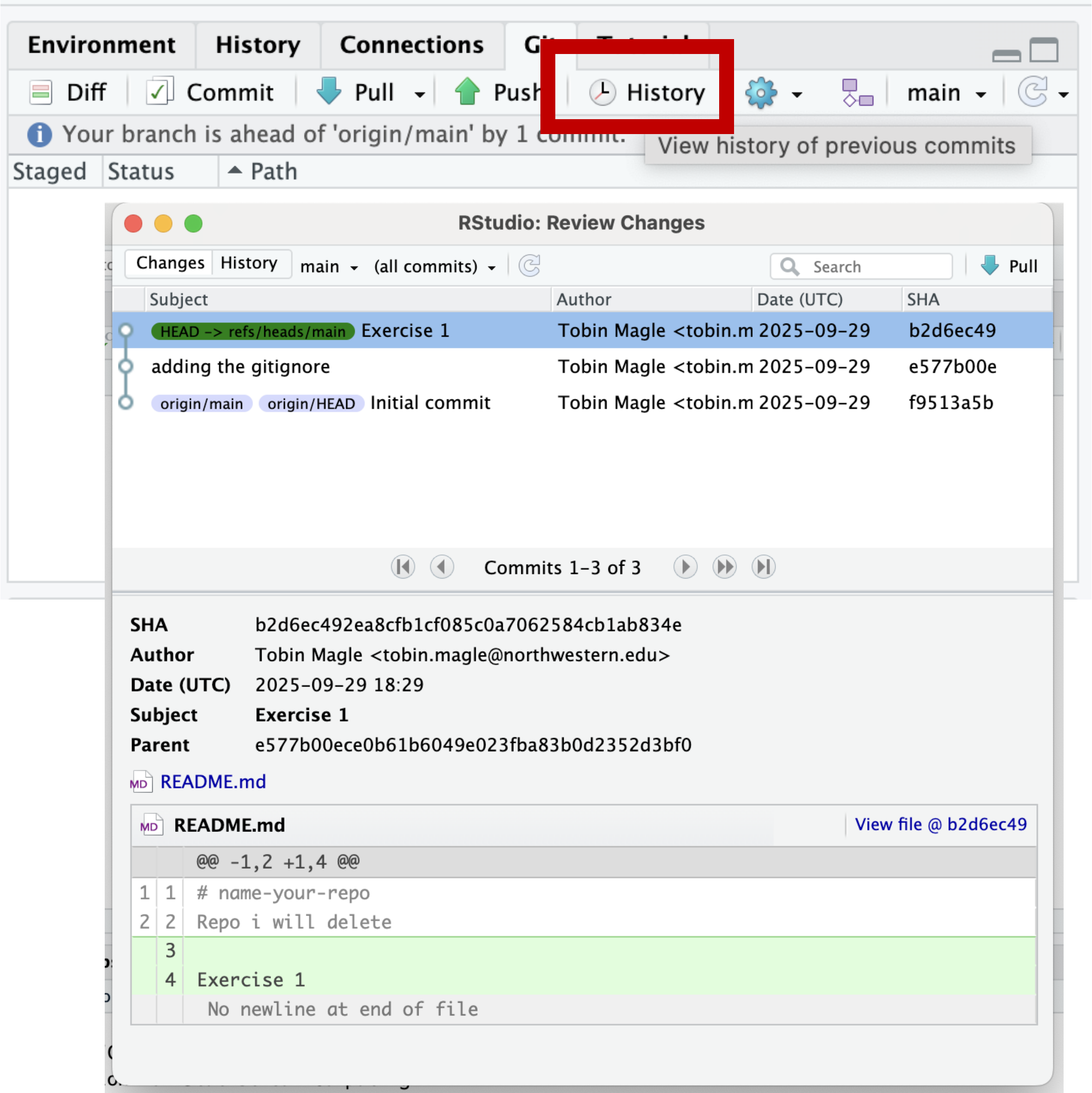
Viewing previous commits

To view previous commits:

1. Click on "History" in the Git tab

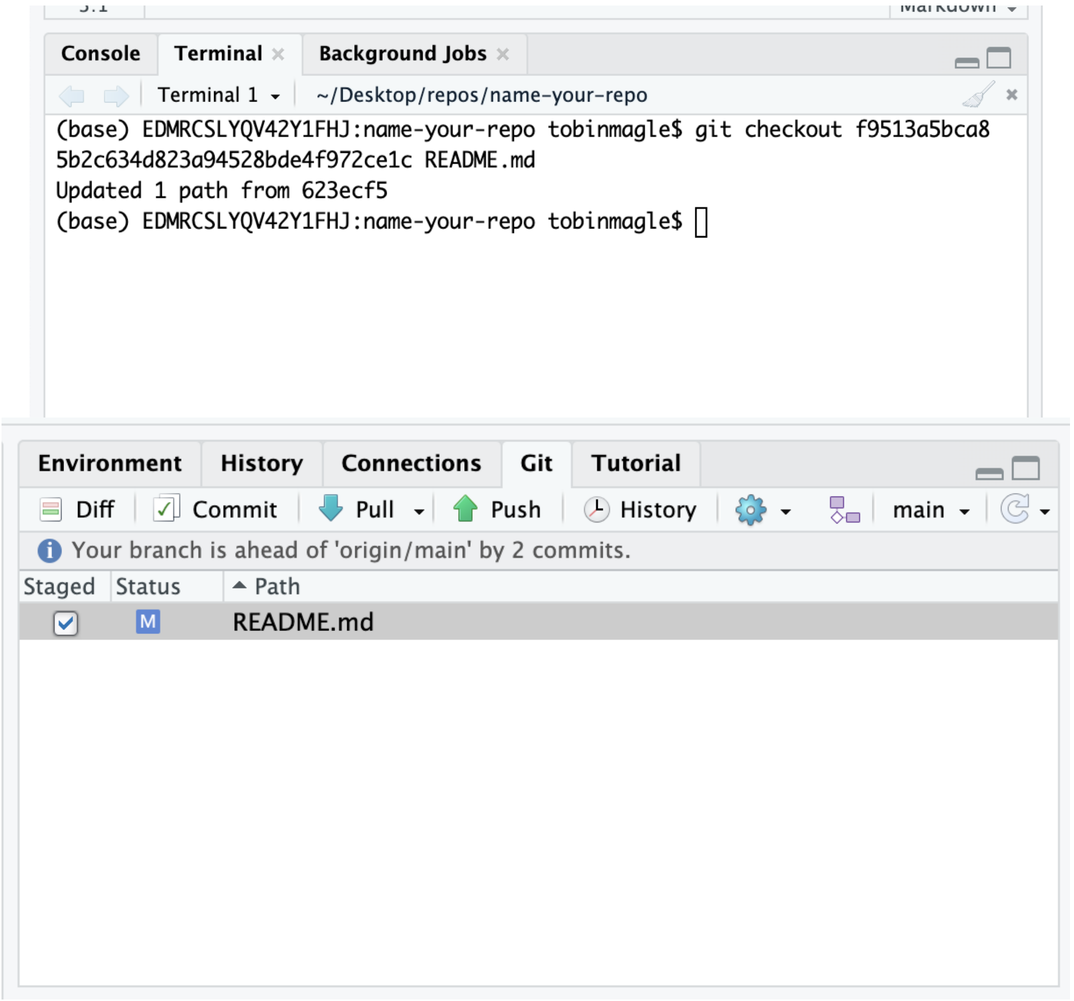
2. A new window will open

3. Click the commit you would like to see details about



Check out a previous version

- Checking out a previous version pulls an old version of a file and puts it in your staging area
- File will show as modified, add and commit like normal
- Need to use the terminal



Check out a previous version

To check out a file from a commit:

1. Copy the SHA hash from the History window

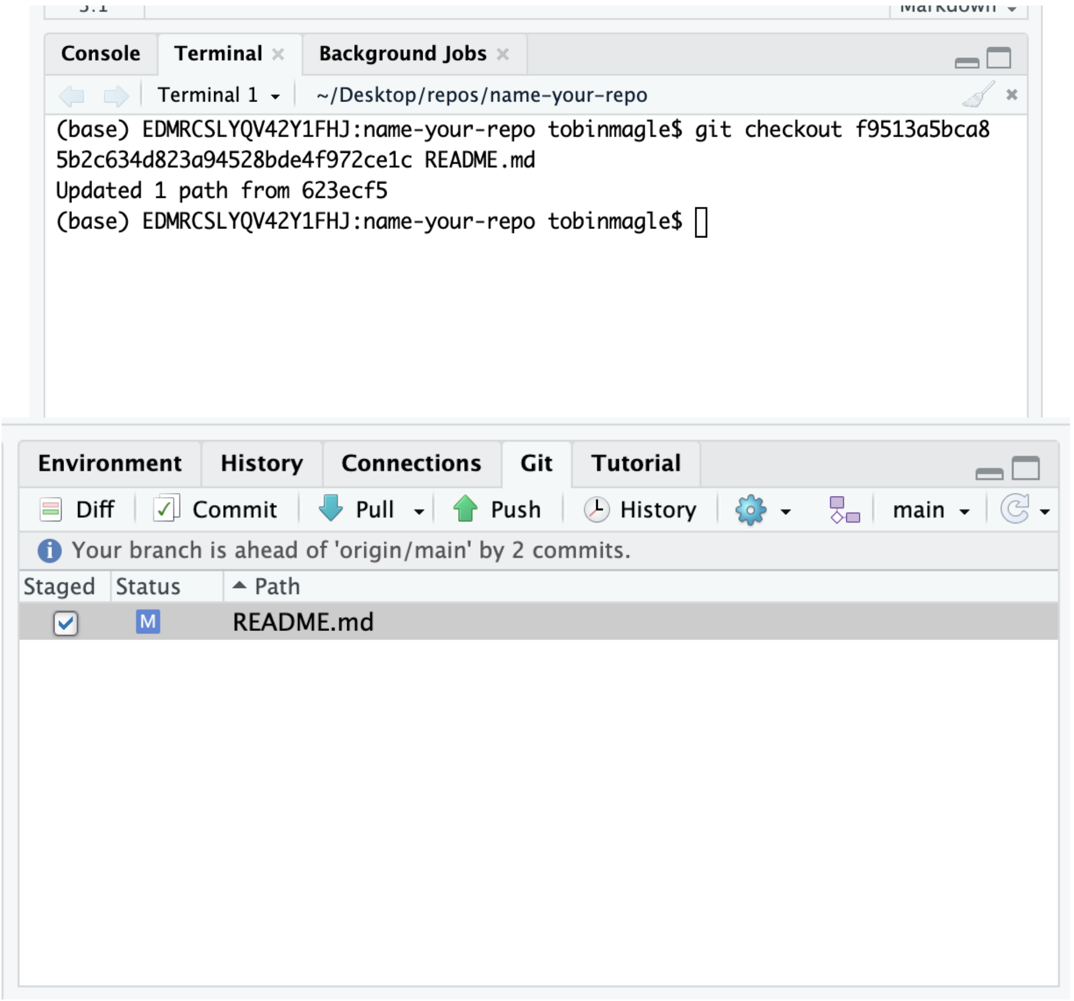
2. Open the terminal in RStudio

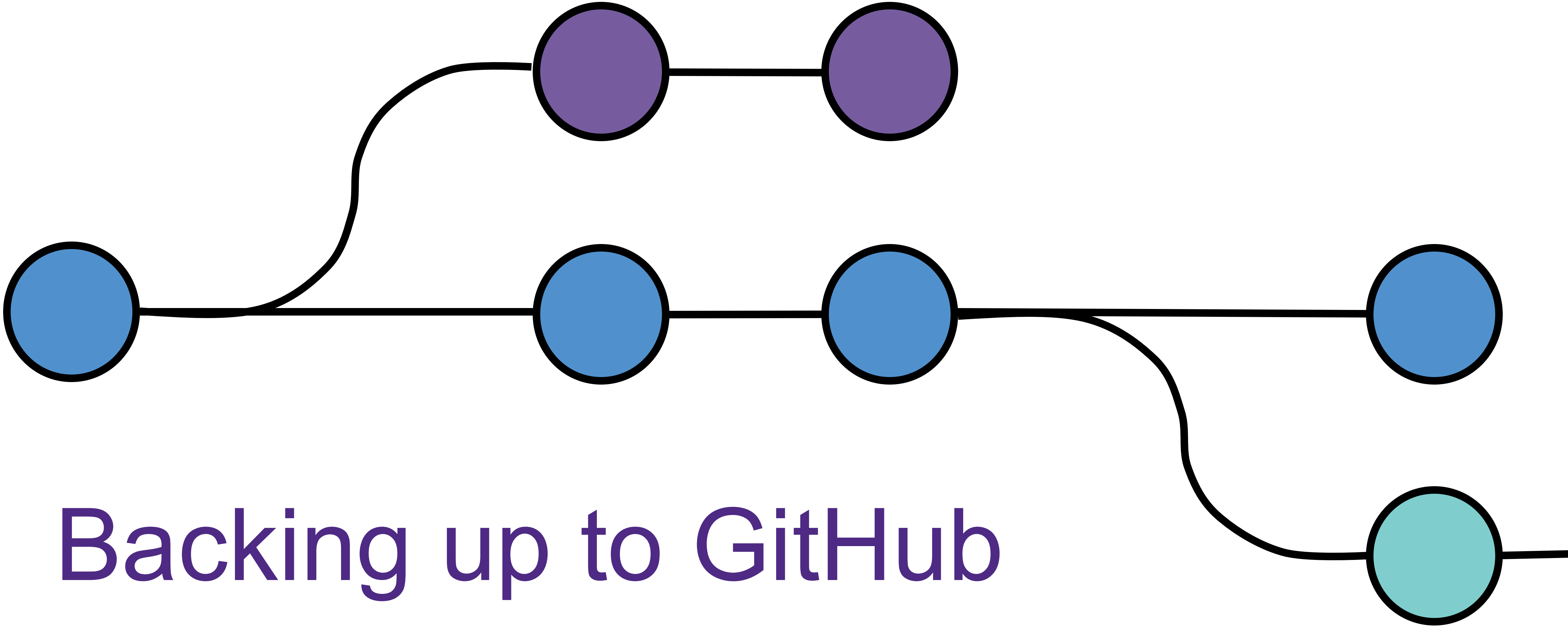
3. Run the following command

`git checkout <SHA hash> <filename>`

4. File will show as modified in the fit window

5. Add and commit if you want to keep the changes

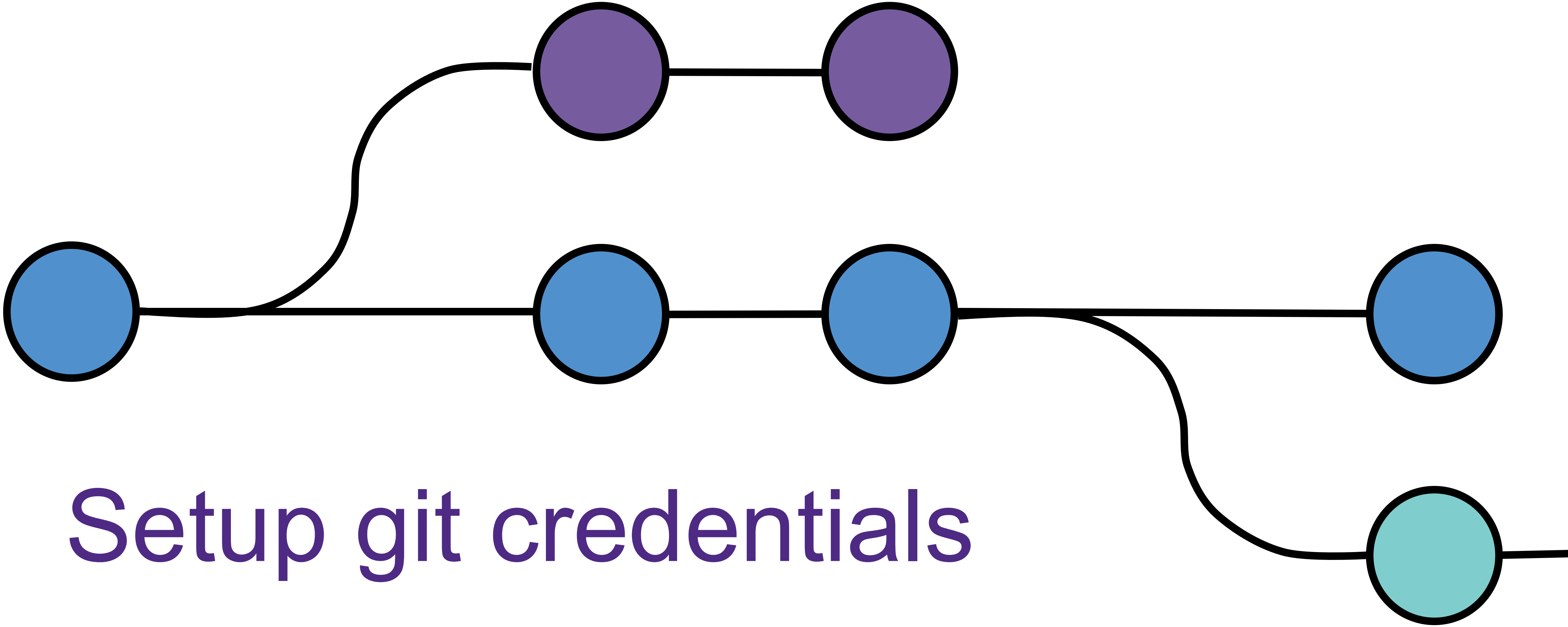




Backing up to GitHub

GitHub best practices

- Store **code and documentation**, not data
- <u>Never</u> store **protected or personal data** even in a private repo
- Store **credentials** (private keys or tokens) in a separate file outside of your repository
  - ie, not in your code
- Add a **personal email** to your GitHub account, so you can use it when you leave Northwestern



Setup git credentials

Configure git on your computer

- Authentication in GitHub has gotten complicated lately

- The `usethis` package "automates repetitive tasks that arise during project setup and development"

- We will use this package to
    - Configure your git user on your computer
    - Set up access credentials in GitHub

Configure your git user on your computer

In RStudio console

1. Open RStudio console

2. Install and load `usethis`

```
install.packages("usethis")
library(usethis)
```

3. Set git username and email
   - Commit history shows username
   - **Use your GitHub account email**

```
usethis::use_git_config(user.name="Jane Doe",
user.email="jane@example.org")
```

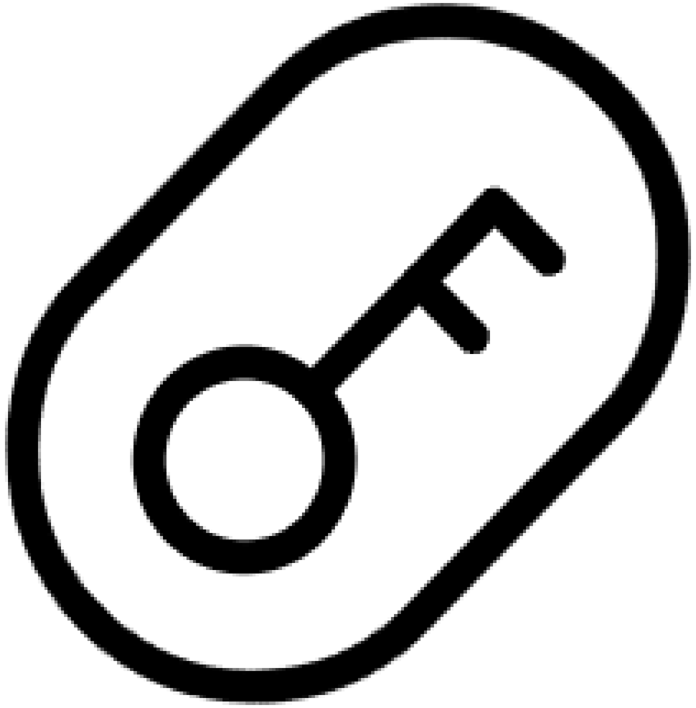
4. Set GitHub authentication

```
usethis::create_github_token()
```

Authenticating into GitHub

Step 1: Generate a GitHub token

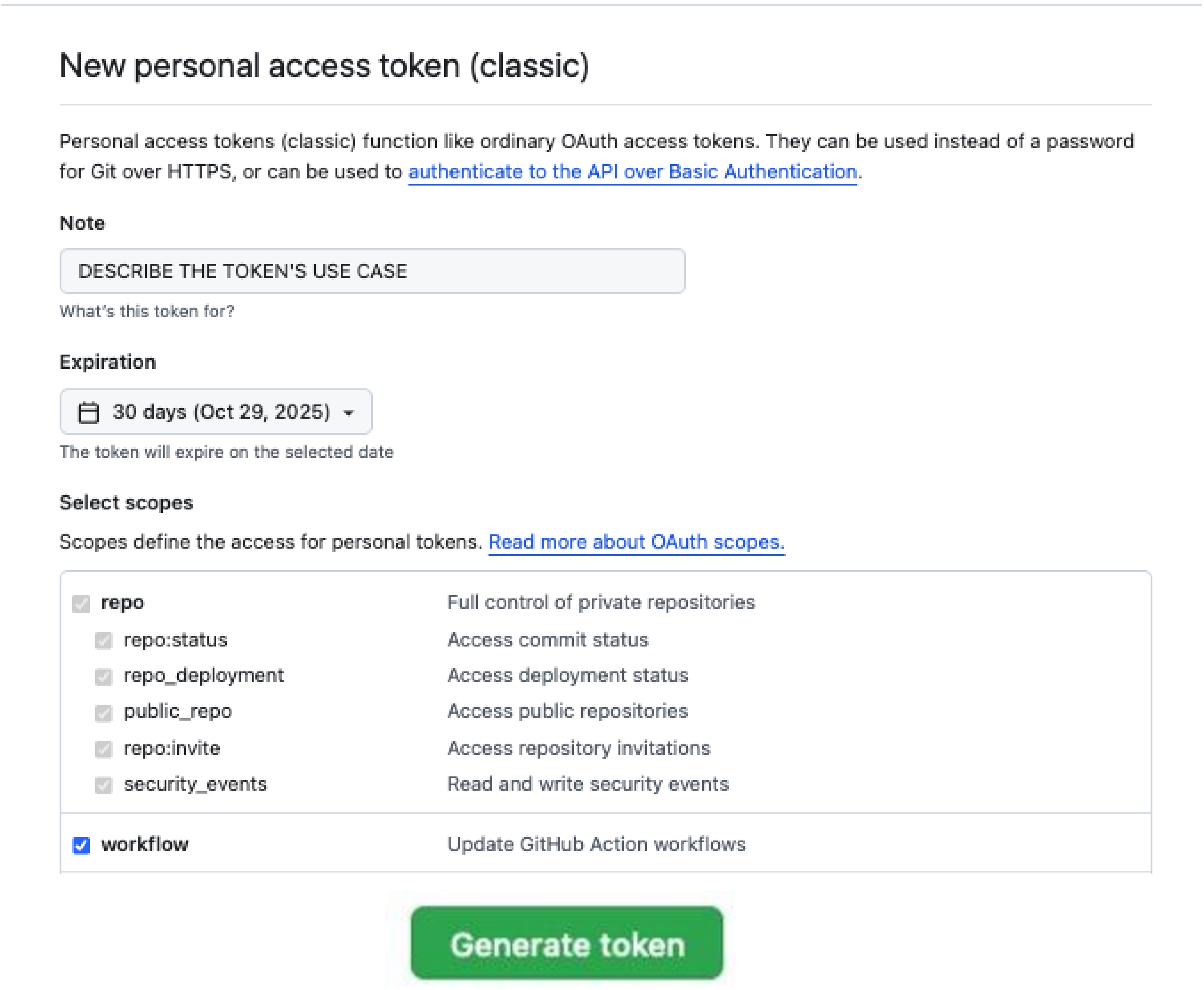
- Good for security
- Use the token instead of your password
- Access tokens allow you to give different levels of access than you have as a user
- Tokens expire
- You can revoke access to tokens



Generate a GitHub token

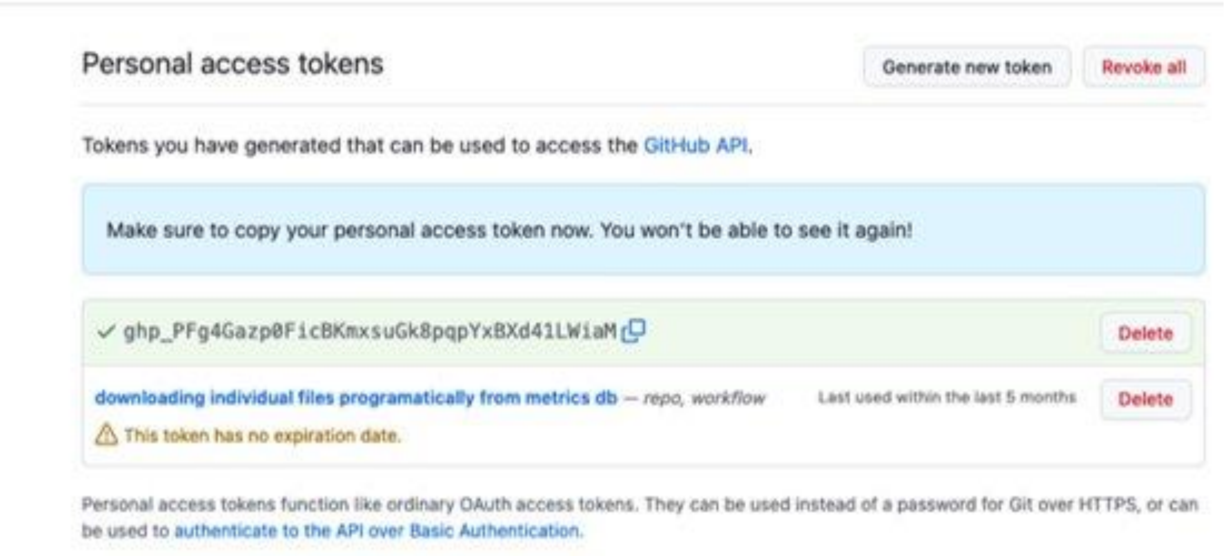
To generate a GitHub token

1. Run `create_github_token()`
2. A webpage will open: New personal access token (classic)
3. Options pre-selected to let you push and pull from a GitHub repo
4. Scroll down and click **Generate Token**



Generate a GitHub token

1. Run `create_github_token()`
2. A webpage will open: New personal access token (classic)
3. Options pre-selected to let you push and pull from a GitHub repo
4. Scroll down and click **Generate Token**
5. A new page will open with the token. Copy it!

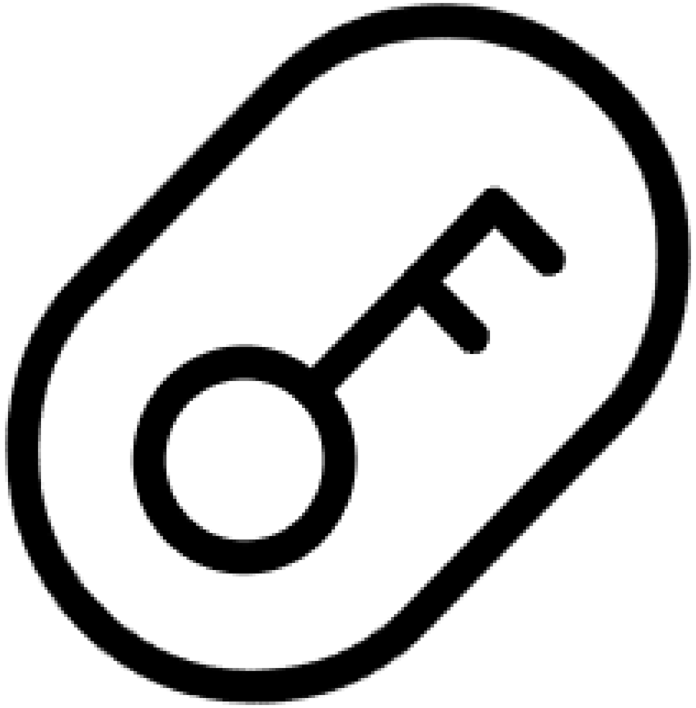


Authenticating into GitHub

Step 1: Generate a GitHub token

Step 2: Use the token to connect the repository on your computer to your repository on GitHub

- Use the token from Step 1 to establish the connection



Log in to GitHub

To log in to GitHub:

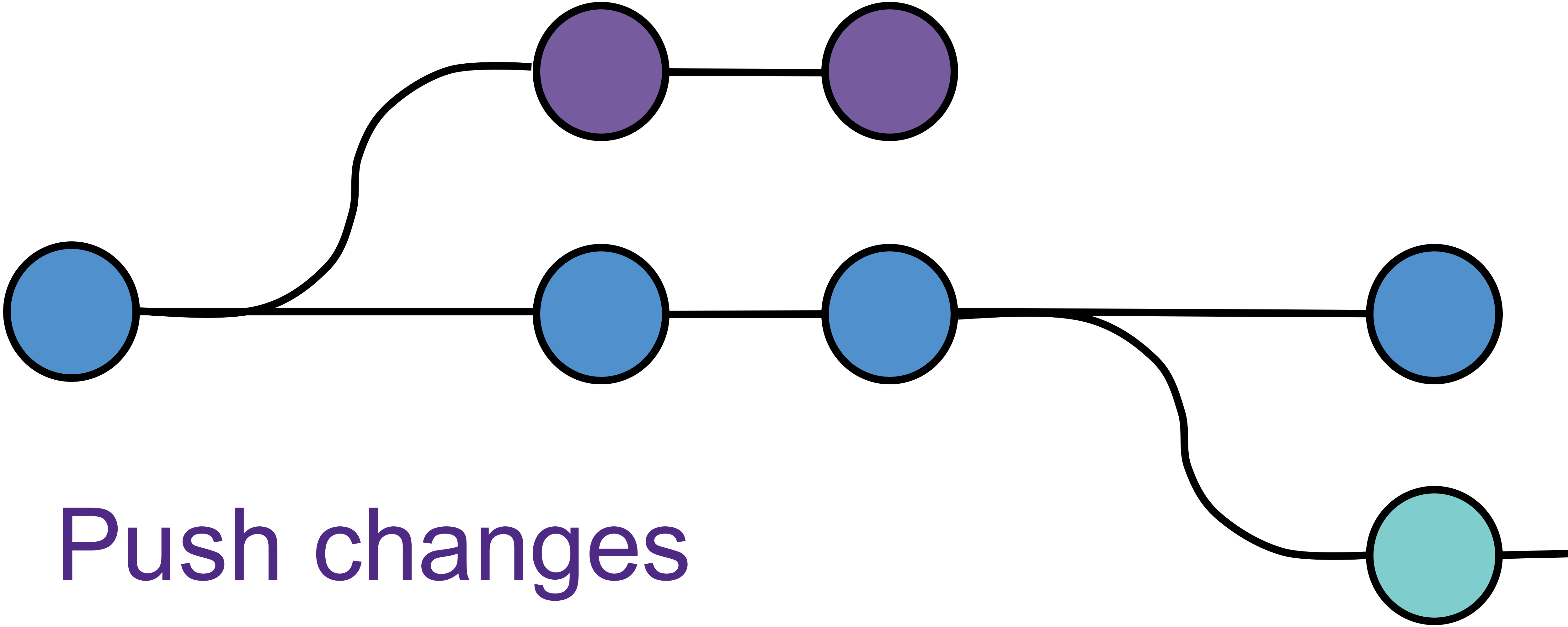
1. Run the `gitcreds_set()` command
2. Paste in the token when asked
3. If you don't have GitHub credentials, it will ask you to for the Token
4. If you had existing credentials you will see 3 options (Pick 2 to use new Token):

```
1: Abort update with error, and keep
the existing credentials
2: Replace these credentials
3: See the password / token
```

5. Git saves these credentials for you

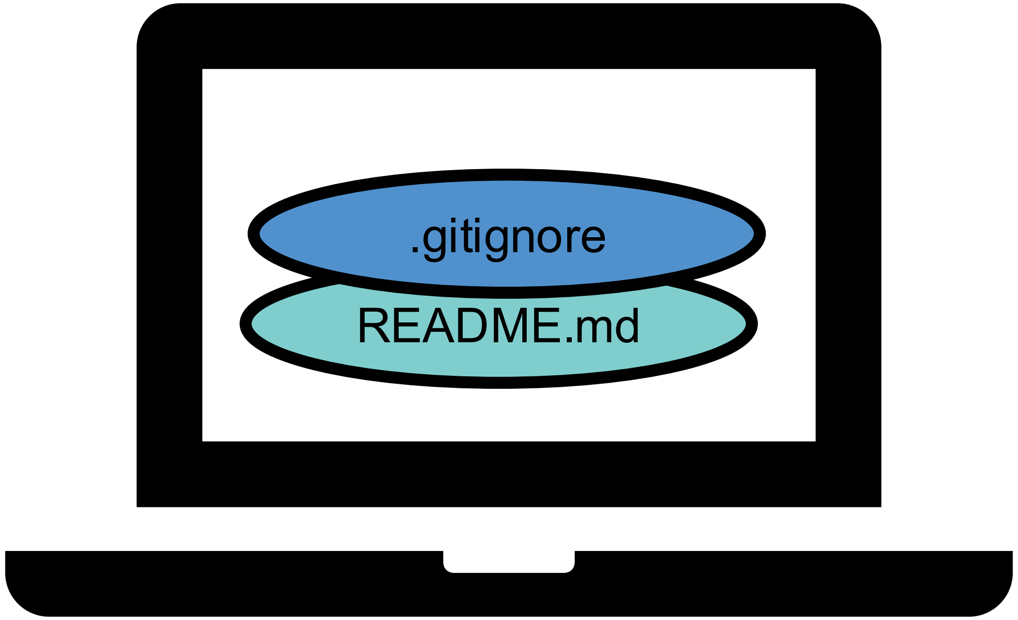
In RStudio console

```
gitcreds::gitcreds_set()
```

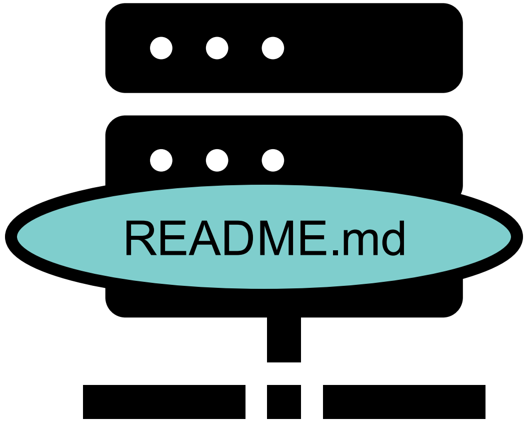


Push changes

Local changes not backed up

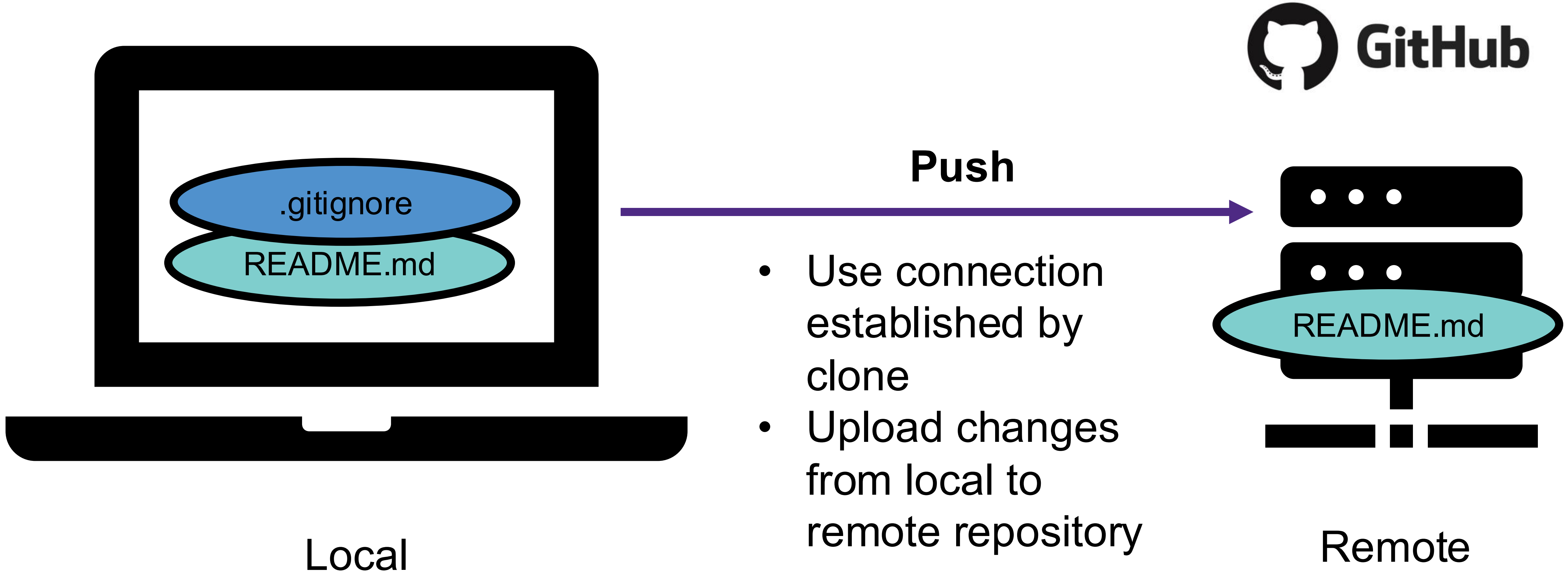


Local



Remote

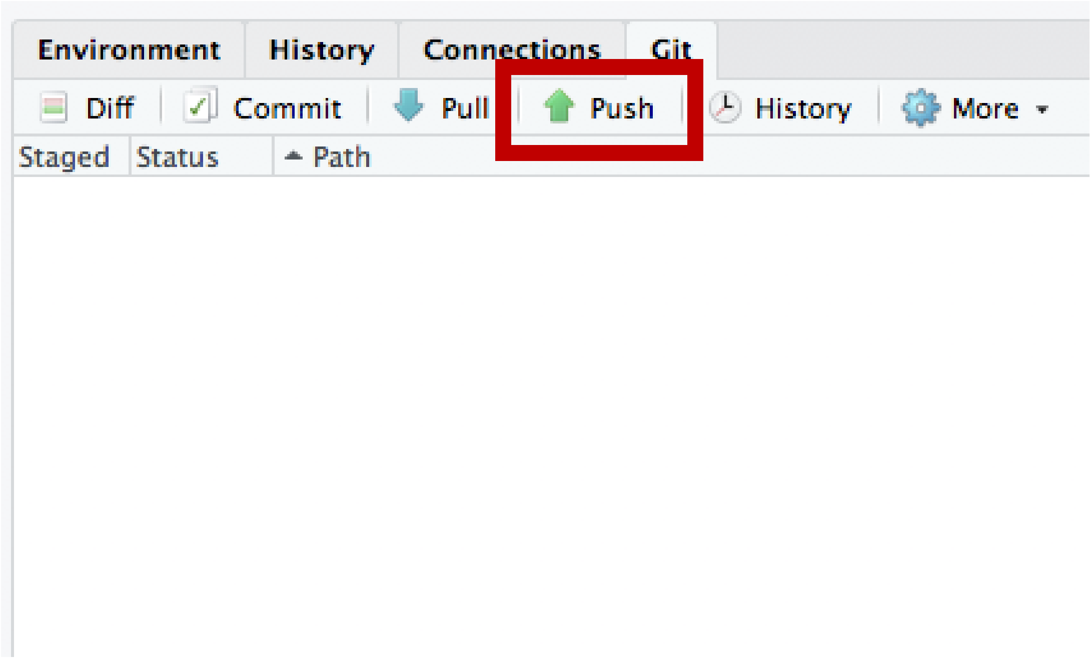
Push to remote



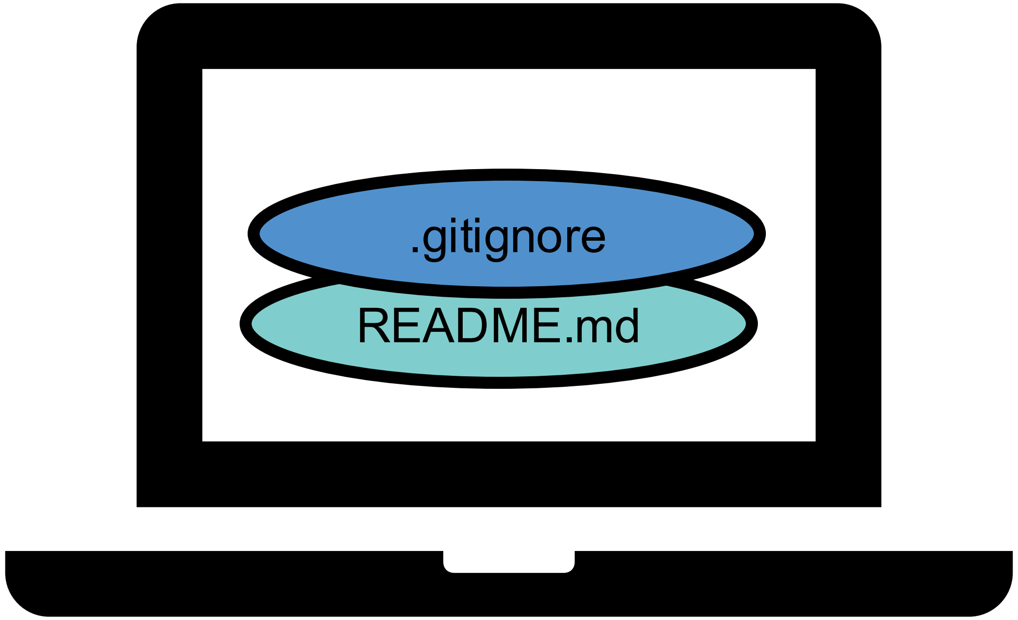
- Use connection established by clone
- Upload changes from local to remote repository

Push

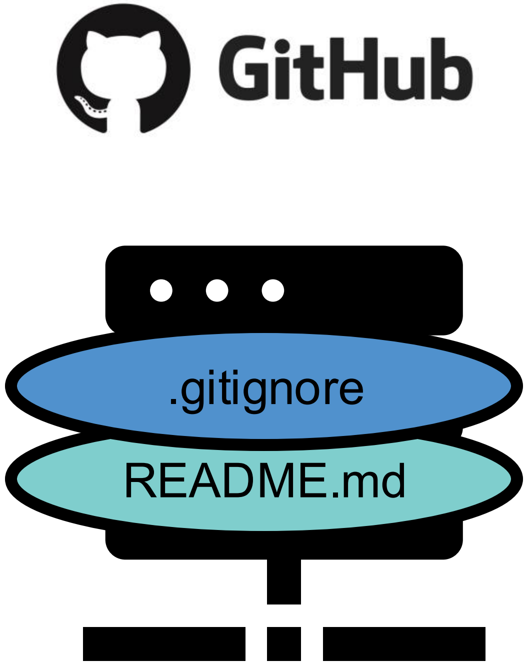
- Save local changes to remote repository
- Changes must be committed before pushing
- Requires that you have authentication set up on your machine



Changes pushed

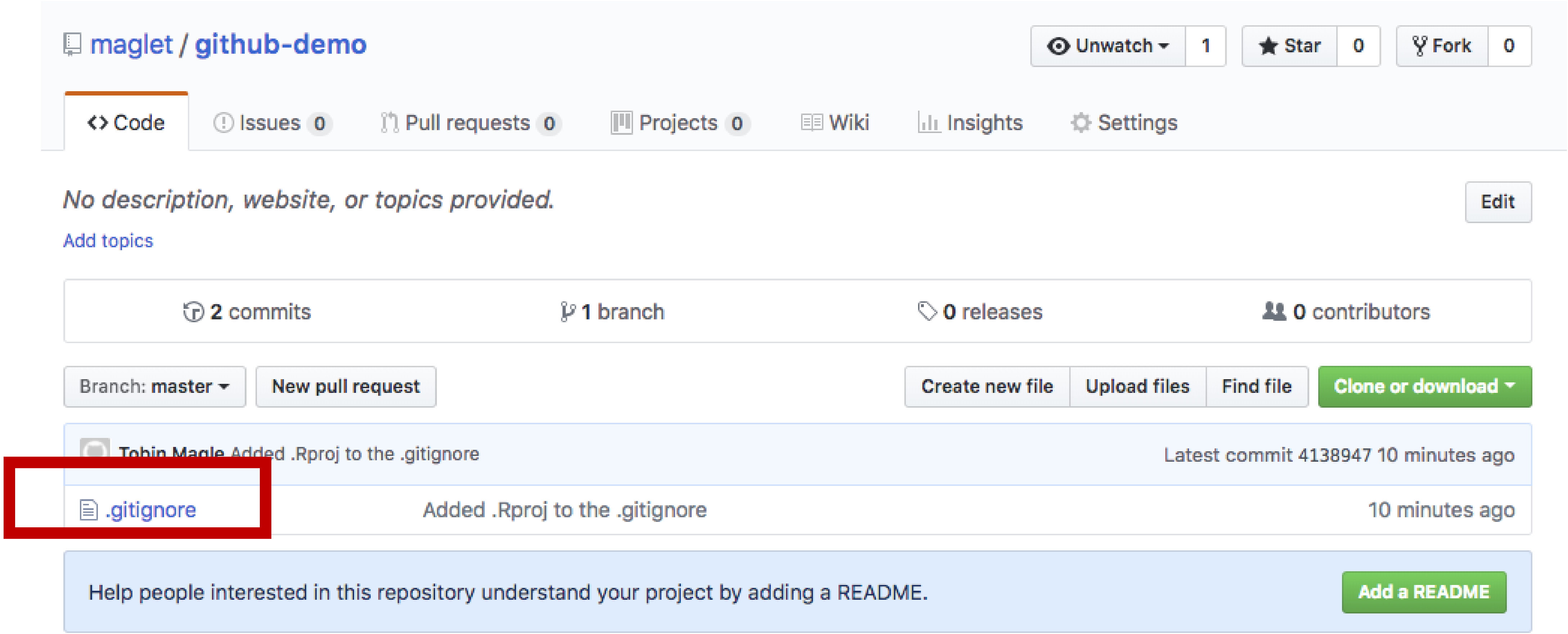


Local



Remote

Files in remote



Practice:

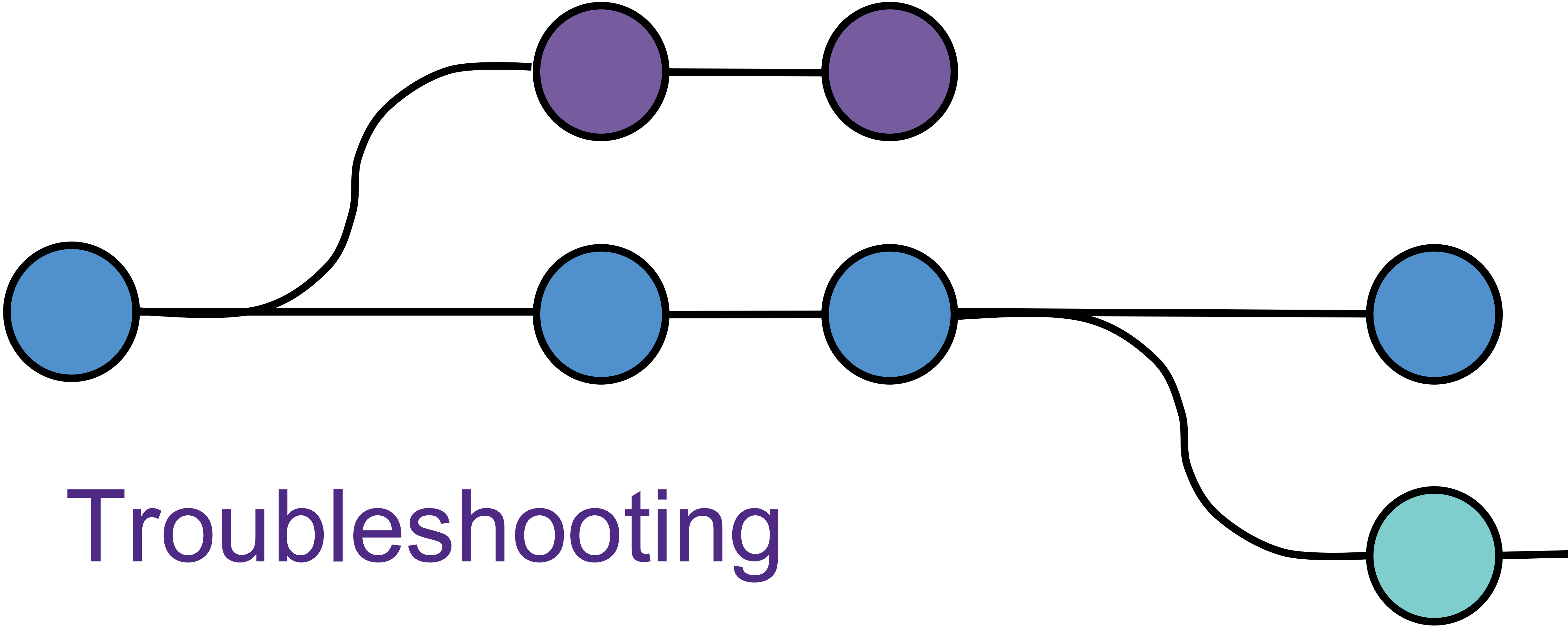
- Make a new R script
- Save it to the R project directory
- Add some R code
- Do the steps necessary to get this file into your GitHub repo

Need help?

Workshop materials: https://github.com/nuitrcs/git-RStudio

- Request a consult: bit.ly/rcdsconsult

- Other Resources
  - RCDS: Written Setup Instructions
  - RCDS: Other ways to set up Git repos in R
  - What git is and how it works: Git for Humans
  - Why you should use git: Academic benefits of using git
  - How to use git with R and RStudio: Happy git with R



Troubleshooting

Q: I don't see the git tab
A: Check that RStudio sees git

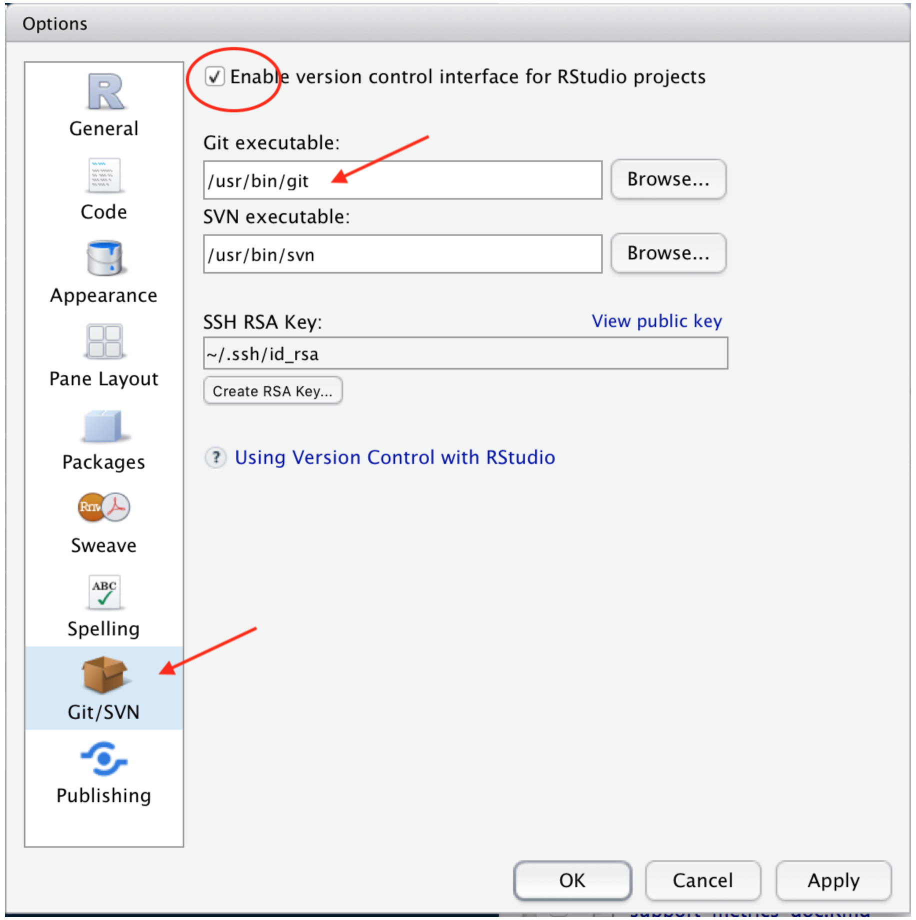
Mac: RStudio > Preferences

Windows: Tools > Global Options

Mac: type

which git

in the Terminal to get the path



Git workflow